

Neural Networks II

Kaiqi Zhao

Harbin Institute of Technology, Shenzhen



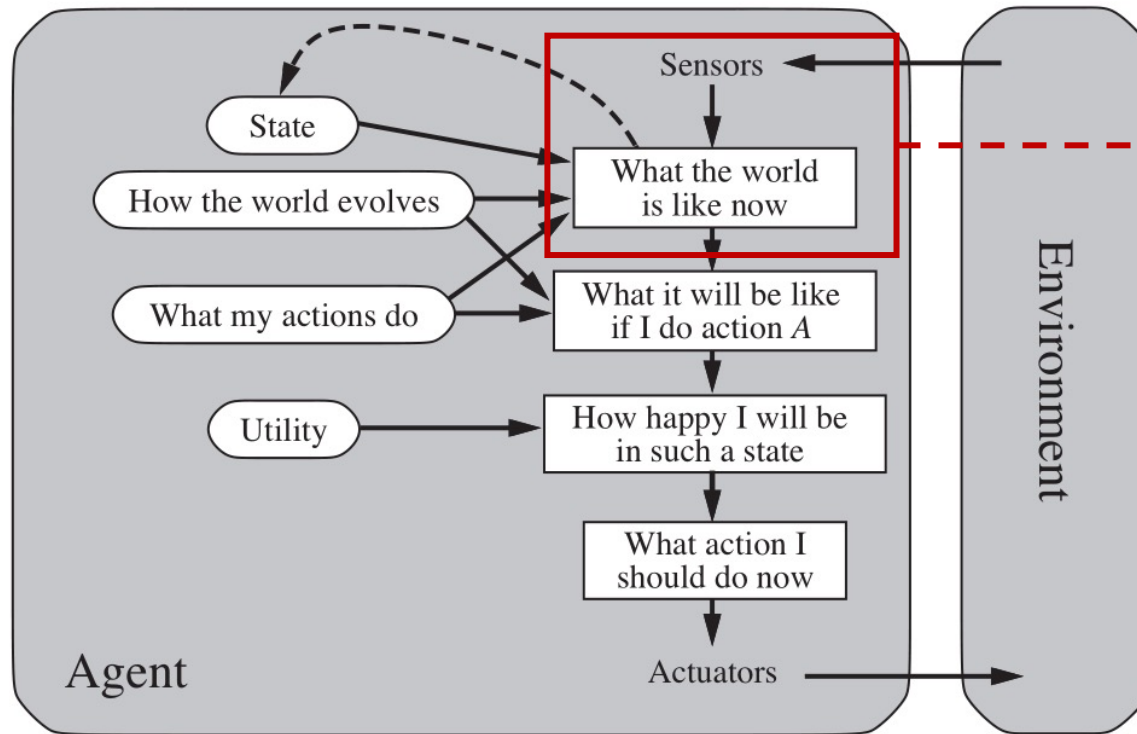
Outline

- Perception with neural networks:
 - Convolutional Neural Networks (CNNs)
- Reasoning with neural networks:
 - Recurrent Neural Networks (RNNs)
 - Attentions and Transformers

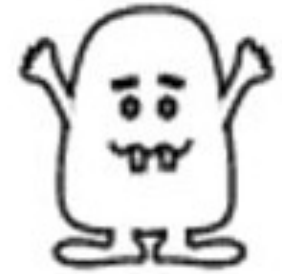
Convolutional Neural Networks

Perception of Agents

Consider the perception problem of an Agent



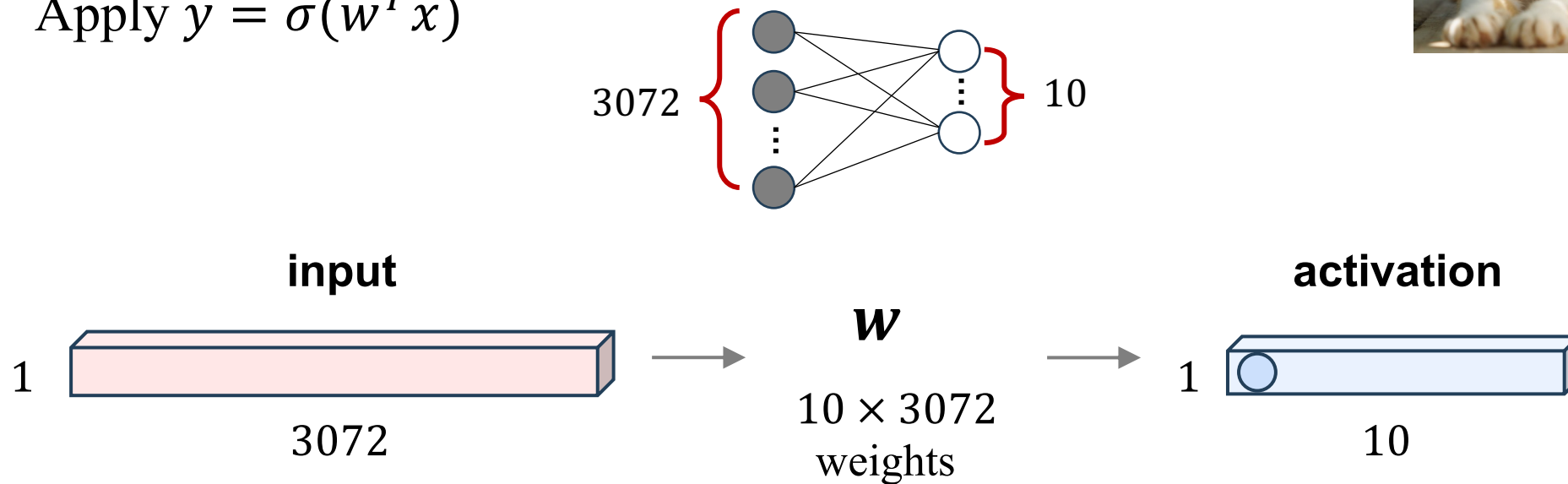
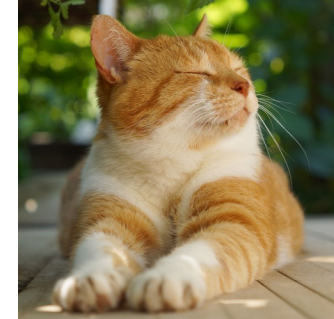
What is this?
Cat!



What is this?
Wumpus :(

First attempt: Fully Connected Network

- Consider classifying an image to one of 10 classes
- For a 32x32 resolution image with 3 channels (R, G, B)
- Flatten the $32 \times 32 \times 3$ tensor to 3072×1 vector x
- Apply $y = \sigma(w^T x)$



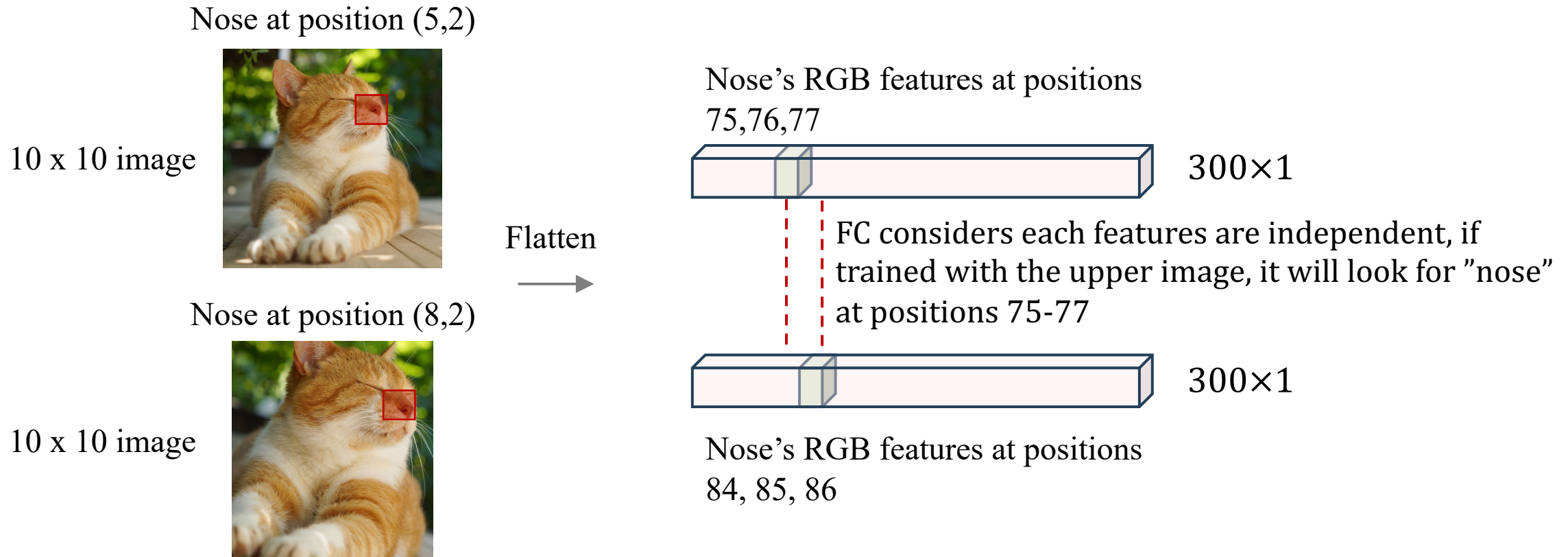
First attempt: Fully Connected Network

- Problem of a simple fully connected network
 - **Losing Spatial Structure:** Nearby pixels are highly correlated (edges, textures, shapes). FC networks flatten the image, destroying the 2D neighbourhood relationships.



First attempt: Fully Connected Network

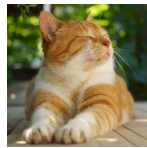
- Problem of a simple fully connected network
 - **No spatial invariance**: Slight changes to the angle or position will make great difference.



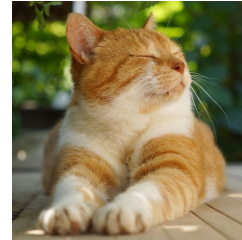
First attempt: Fully Connected Network

- Problem of a simple fully connected network
 - **Exploding Weights**: The number of weights explodes on high-resolution images.
 - Slow training, and massive overfitting.

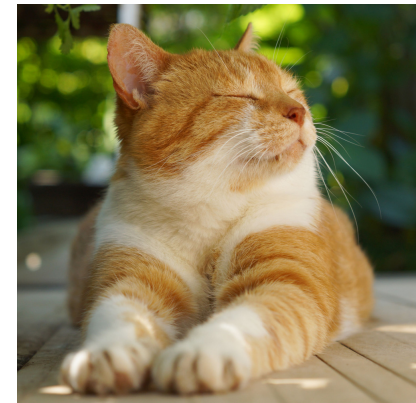
Suppose the first layer
has 500 neurons



Input size: $32 \times 32 \times 3 \approx 3k$
First layer weights $\#weights \approx 1.5m$



$320 \times 320 \times 3 \approx 300k$
 $\#weights \approx 150 m$



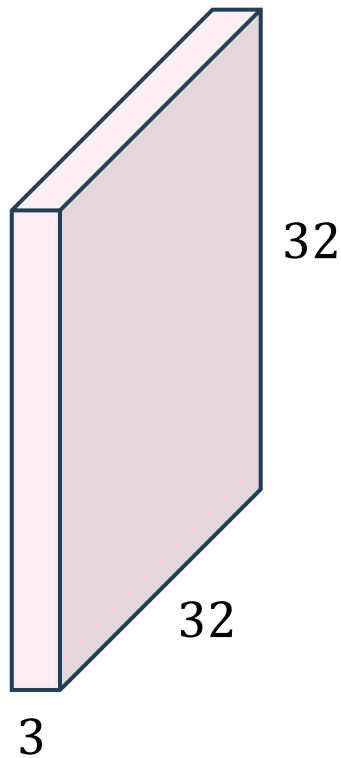
$3200 \times 3200 \times 3 \approx 30m$
 $\#weights \approx 15 b$

Convolution Neural Nets

- Addresses the three problems of FC networks
 - **Preserving Spatial Structure:** Work on the original image dimensions without flattening
 - **Handling spatial invariance & reducing weights:** Each time focus on a small “region” of the image

Convolution

$32 \times 32 \times 3$ image



$5 \times 5 \times 3$ filter



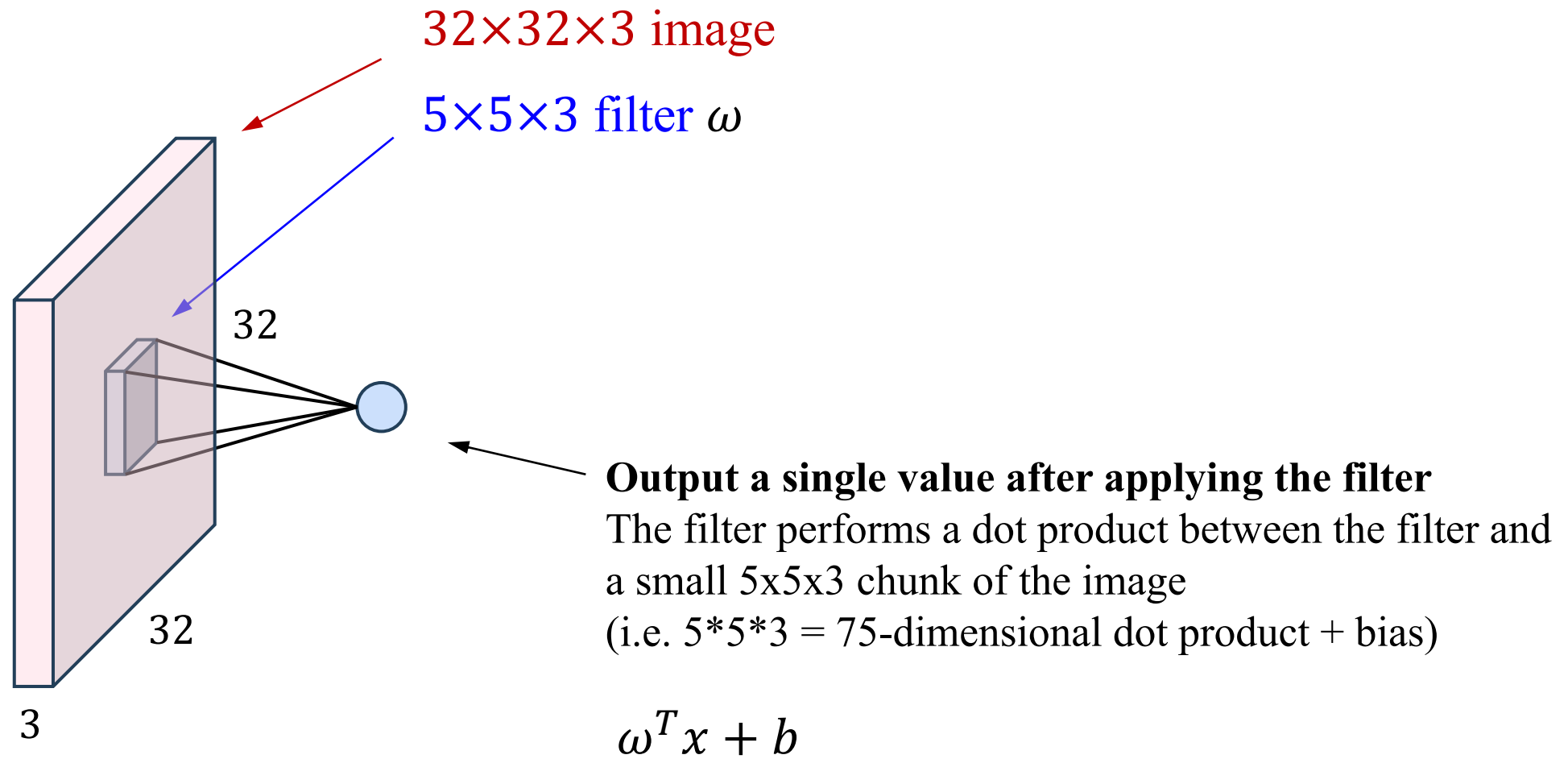
Introduce small filters that always extend the full depth of the input volume

Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

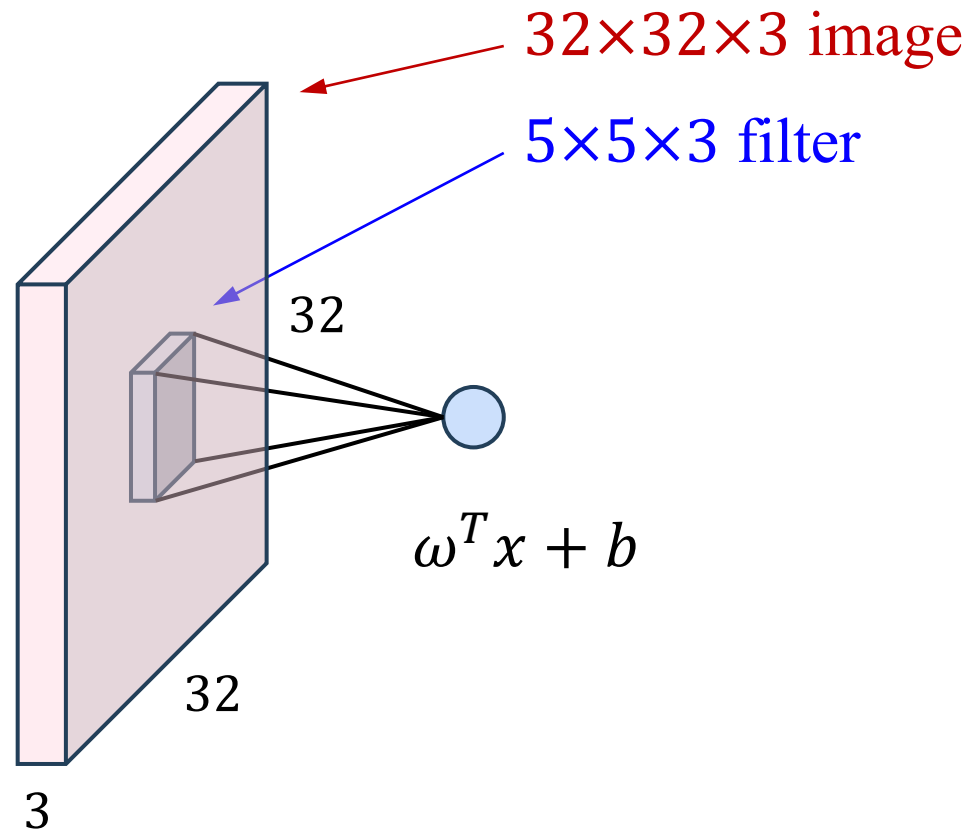
- 1. Local connectivity**
- 2. Parameter sharing across regions!**

$32 \times 32 \times 3$ image $\rightarrow$ preserve spatial structure

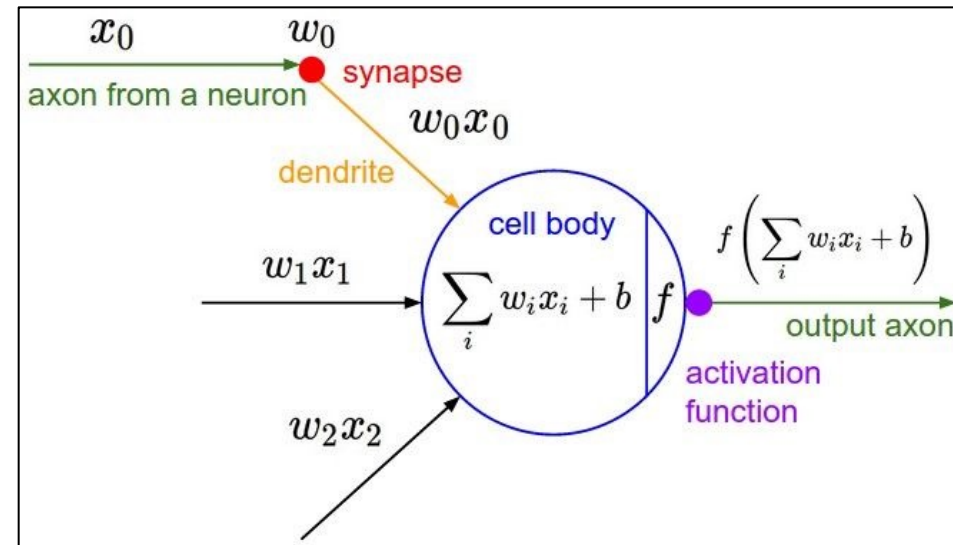
Convolution



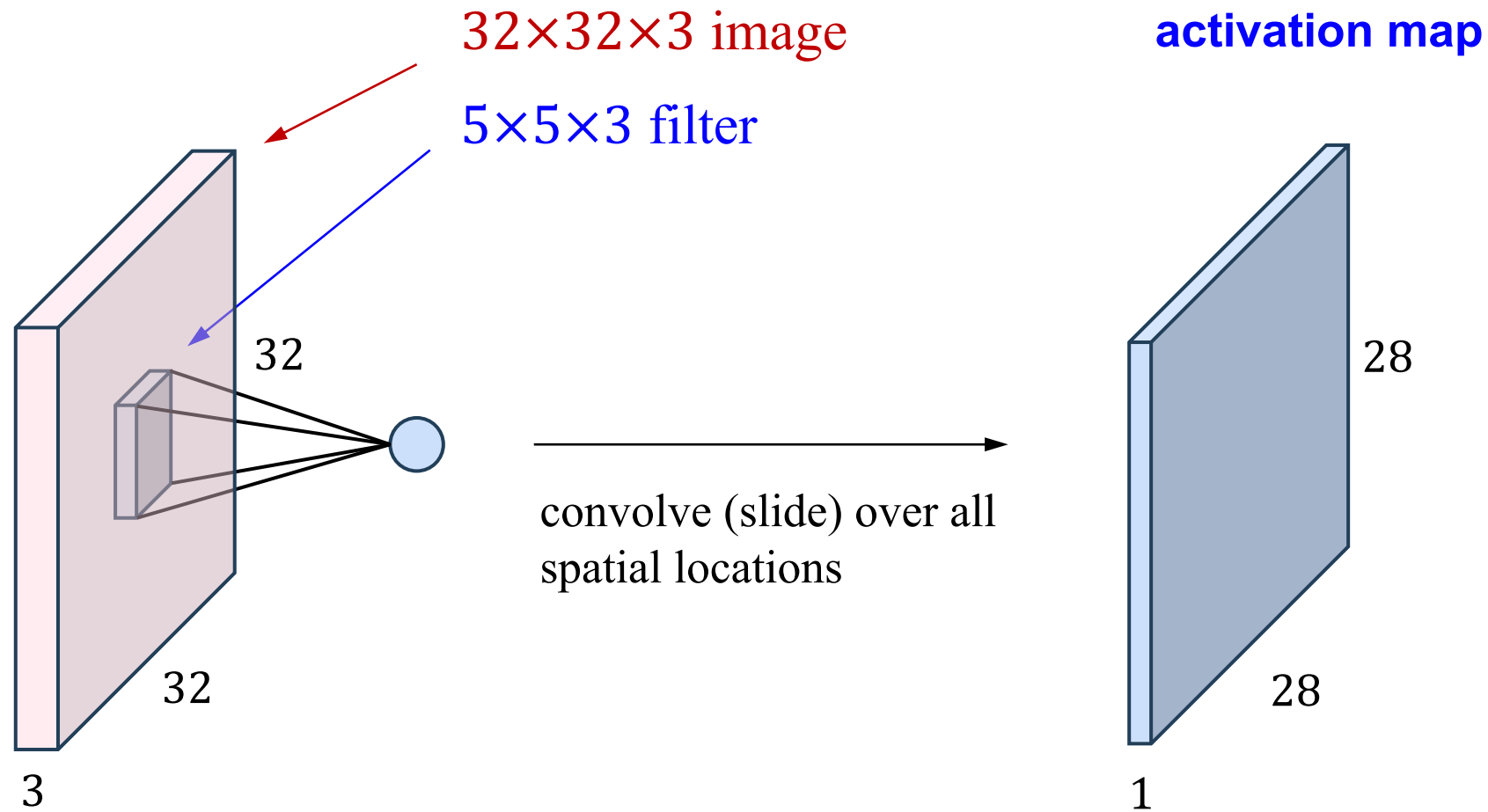
Convolution



Each filter can be considered as a neuron that captures local connectivity...

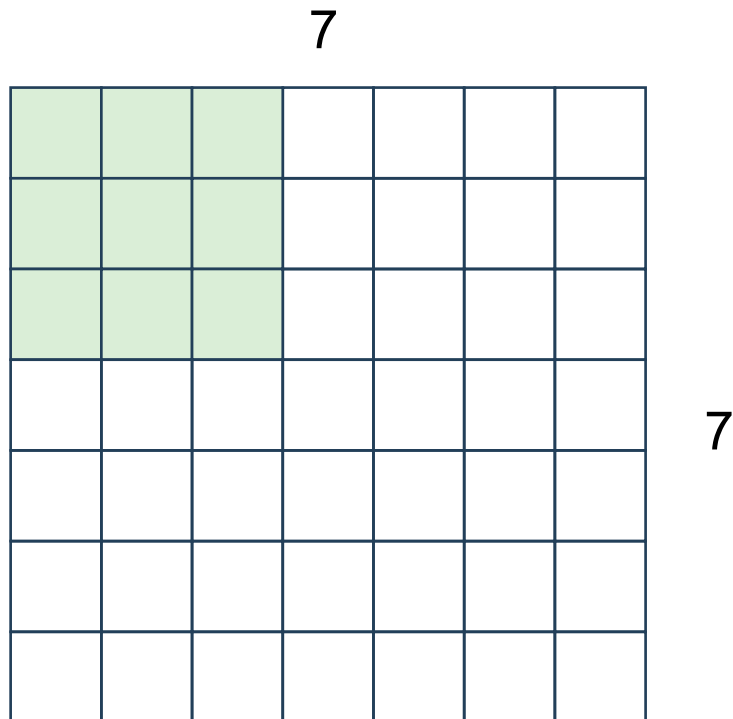


Convolution Layer



Convolution Layer

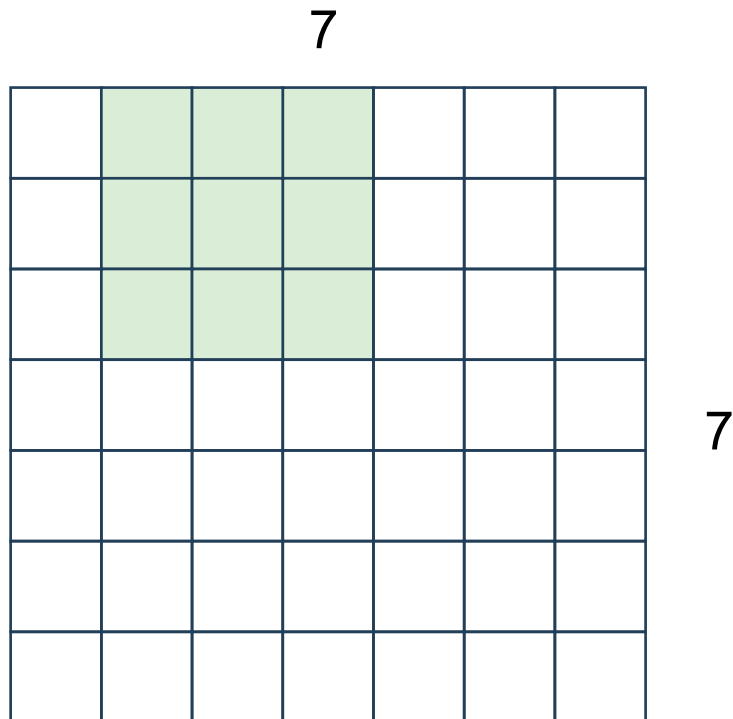
A toy example (look at spatial dimensions)



7×7 input (spatially)
assume 3×3 filter

Convolution Layer

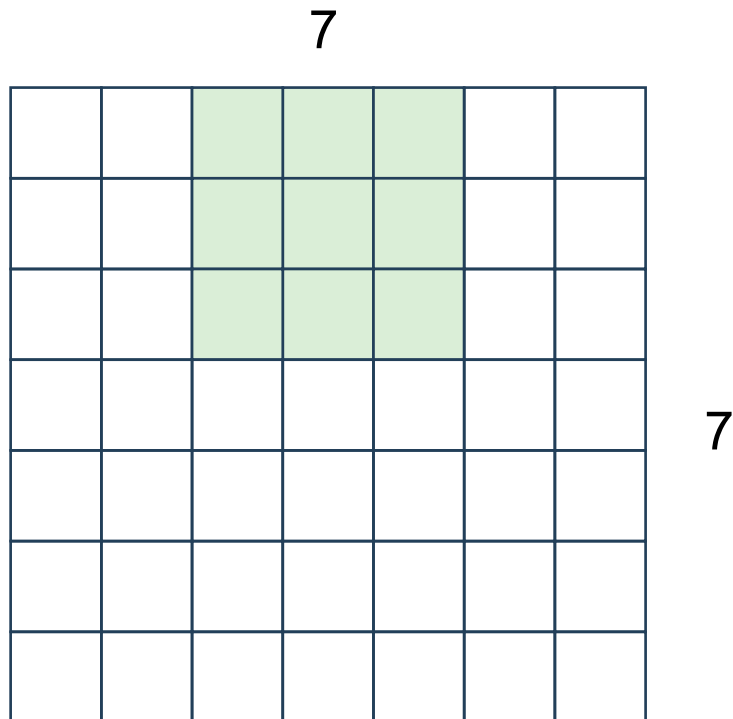
A toy example (look at spatial dimensions)



7×7 input (spatially)
assume 3×3 filter

Convolution Layer

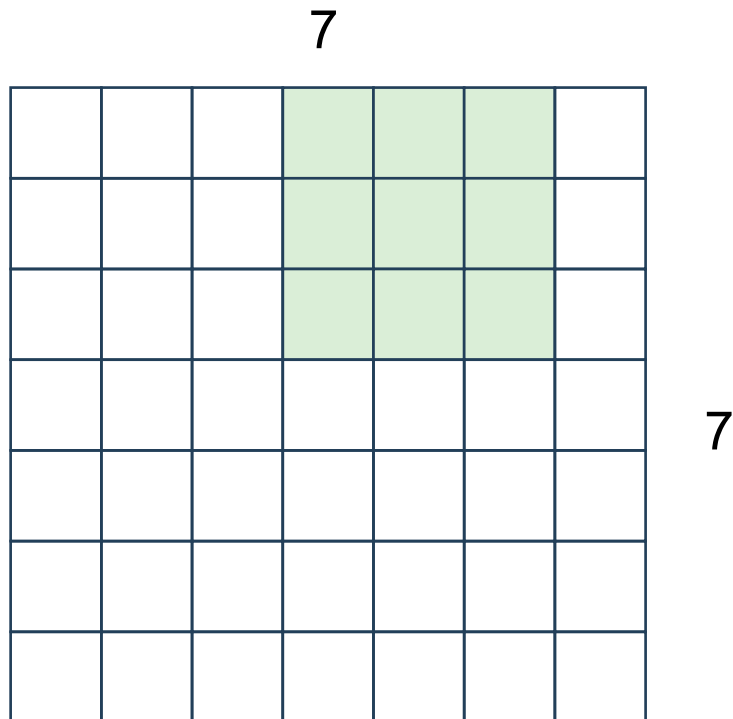
A toy example (look at spatial dimensions)



7×7 input (spatially)
assume 3×3 filter

Convolution Layer

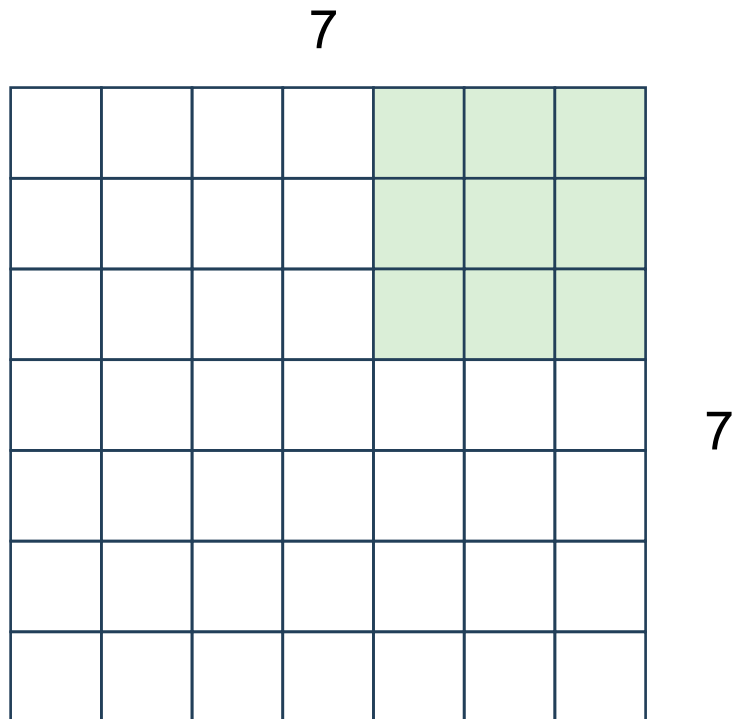
A toy example (look at spatial dimensions)



7×7 input (spatially)
assume 3×3 filter

Convolution Layer

A toy example (look at spatial dimensions)

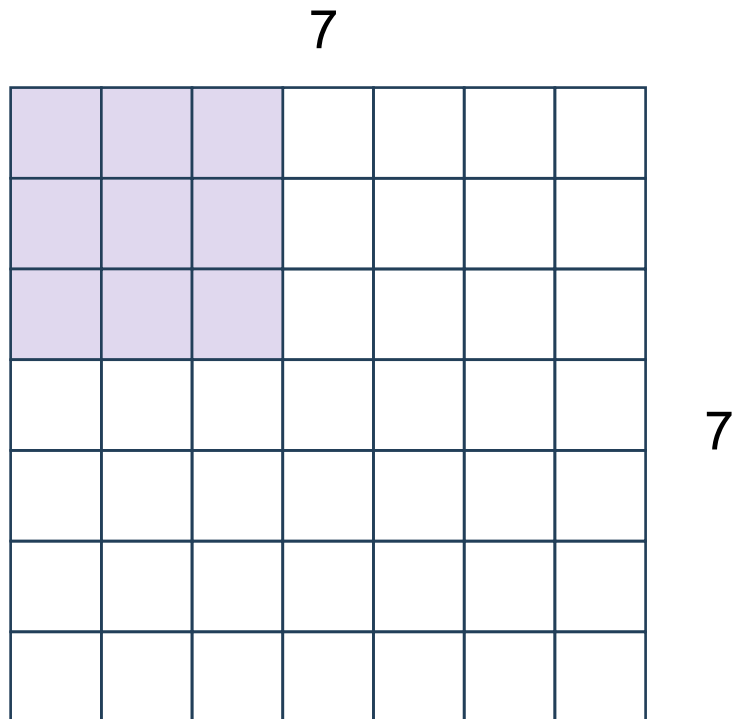


7×7 input (spatially)
assume 3×3 filter

$\Rightarrow 5 \times 5$ output

Convolution Layer

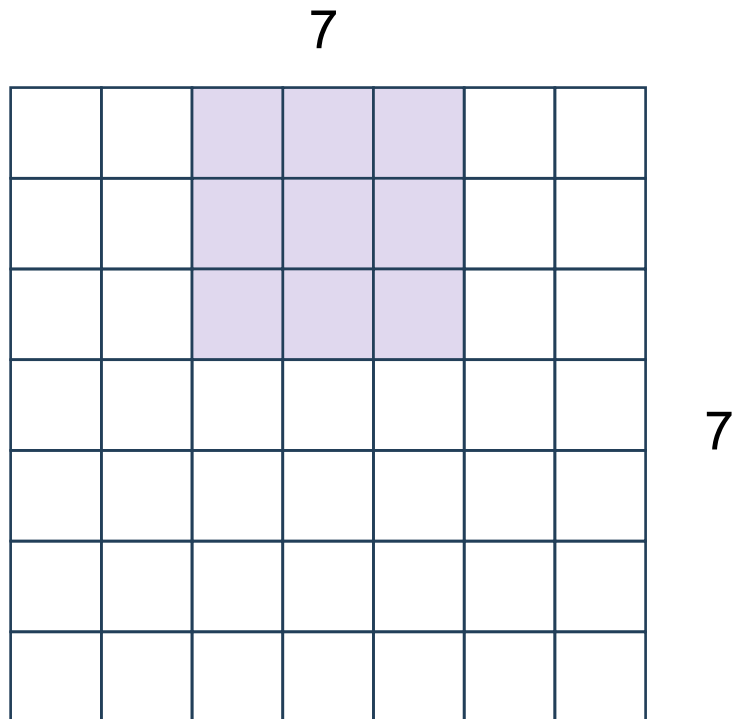
A toy example (look at spatial dimensions)



7×7 input (spatially)
assume 3×3 filter
applied **with stride 2**

Convolution Layer

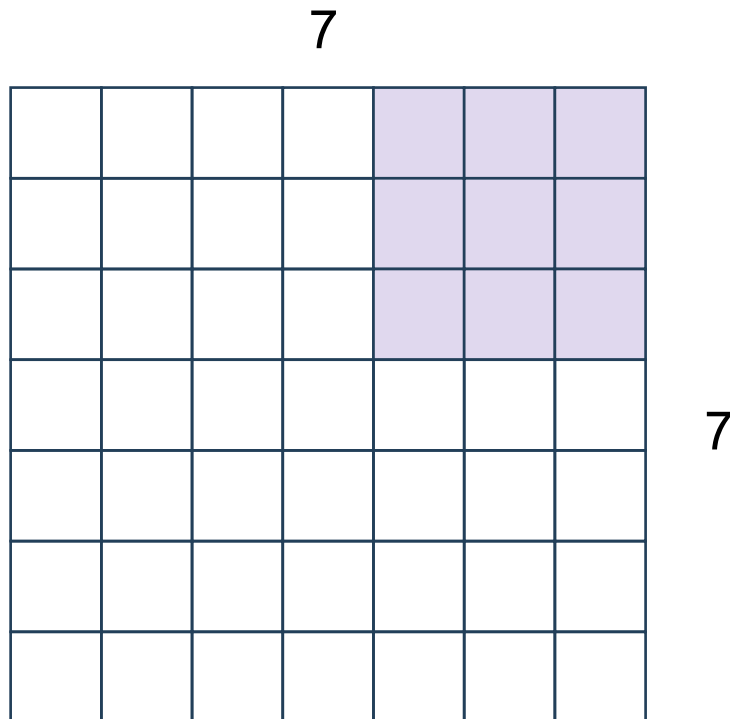
A toy example (look at spatial dimensions)



7×7 input (spatially)
assume 3×3 filter
applied **with stride 2**

Convolution Layer

A toy example (look at spatial dimensions)

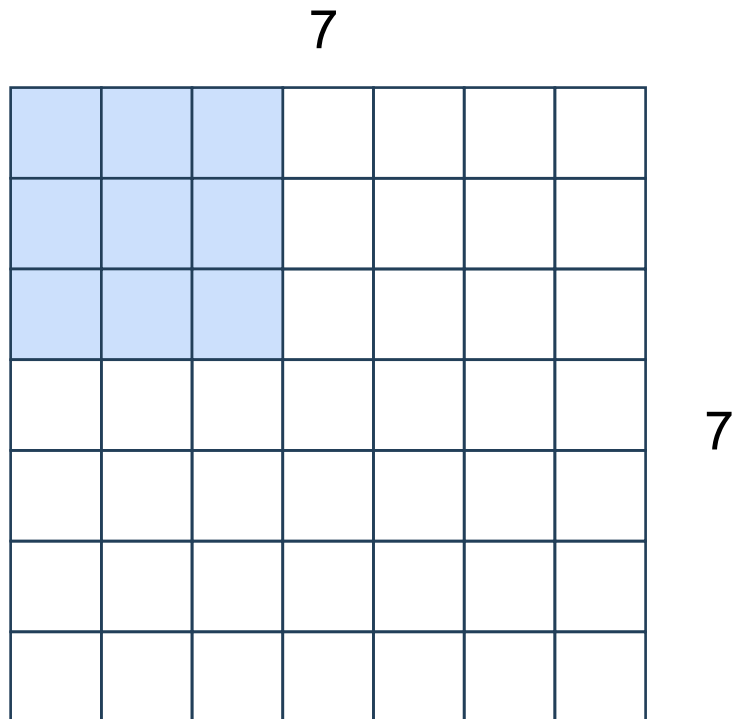


7×7 input (spatially)
assume 3×3 filter
applied **with stride 2**

$\Rightarrow 3 \times 3$ output

Convolution Layer

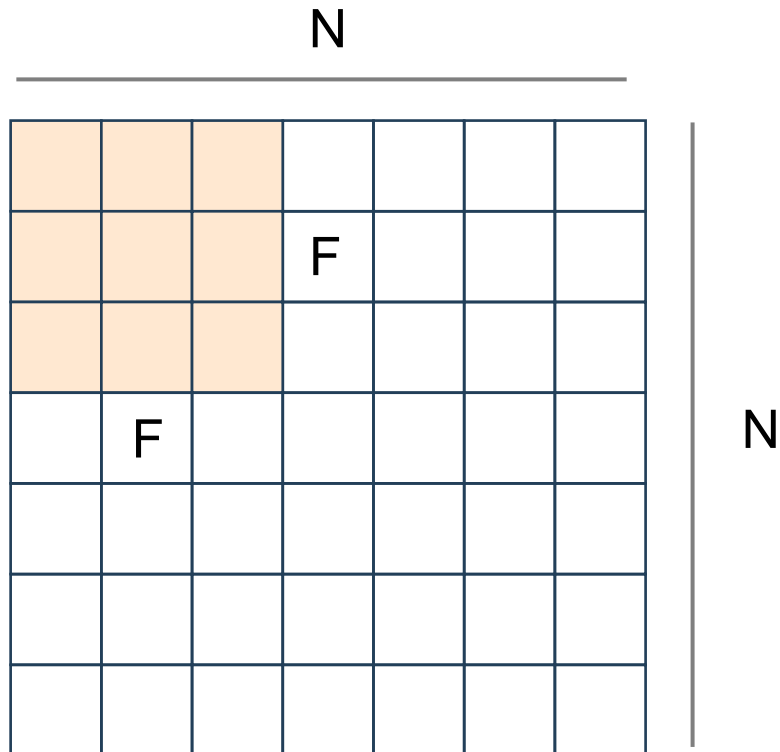
A toy example (look at spatial dimensions)



7×7 input (spatially)
assume 3×3 filter
applied **with stride 3**?

doesn't fit!
cannot apply 3×3 filter on
 7×7 input with stride 3.

Convolution Layer



Output size:
 $(N - F) / \text{stride} + 1$

e.g. $N = 7, F = 3$:

stride 1 $\Rightarrow (7 - 3) / 1 + 1 = 5$

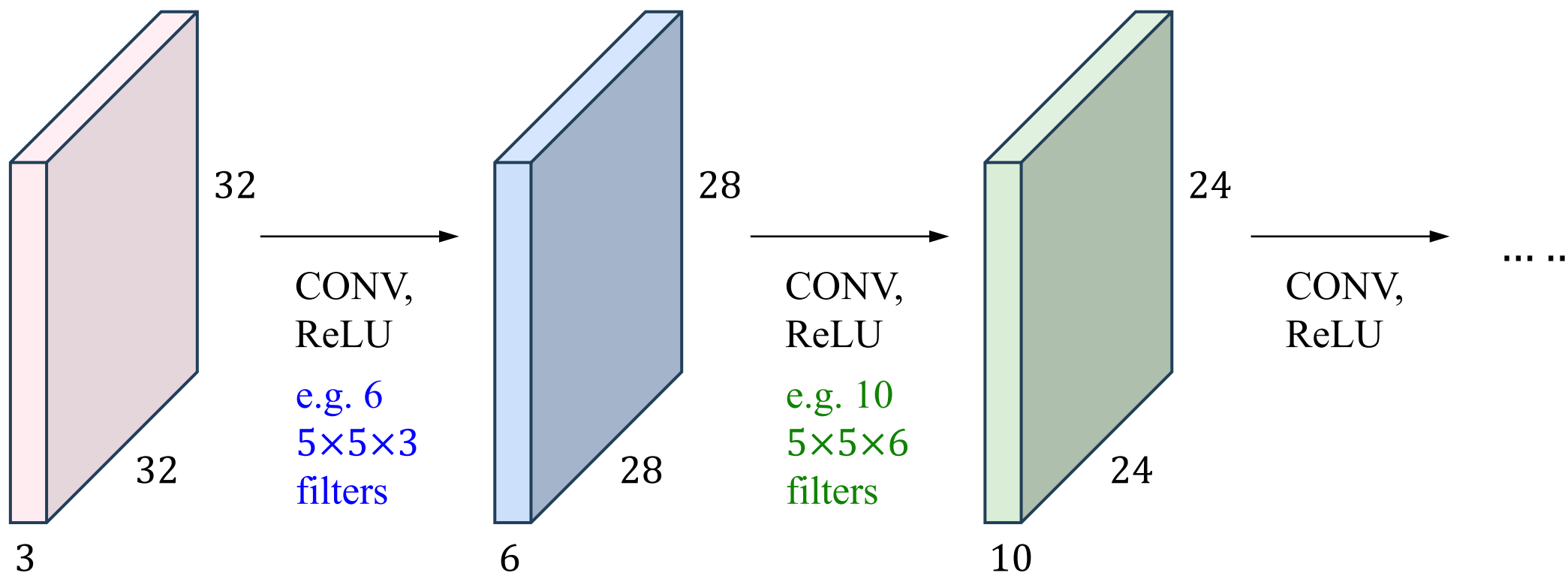
stride 2 $\Rightarrow (7 - 3) / 2 + 1 = 3$

stride 3 $\Rightarrow (7 - 3) / 3 + 1 = 2.33$

Convolution Layer

The feature map will shrink after each convolution.

5×5 filters applied on 32×32 input image shrinks spatially after each convolution layer ($32 \rightarrow 28 \rightarrow 24 \dots$).



Convolution Layer

Padding: It is common to pad the border with “0”

0	0	0	0	0	0			
0								
0								
0								
0								

E.g. input 7×7

3×3 filter, applied with **stride 1**

Pad with 1 pixel border $\Rightarrow$ what is the output?

No padding: $(N - F)/\text{stride} + 1$

Padding P pixels: $(N + 2P - F)/\text{stride} + 1$

7×7 output!

Commonly, stride is set to 1.

To preserve the size spatially, filters of size $F \times F$ will need zero-padding with $(F - 1)/2$.

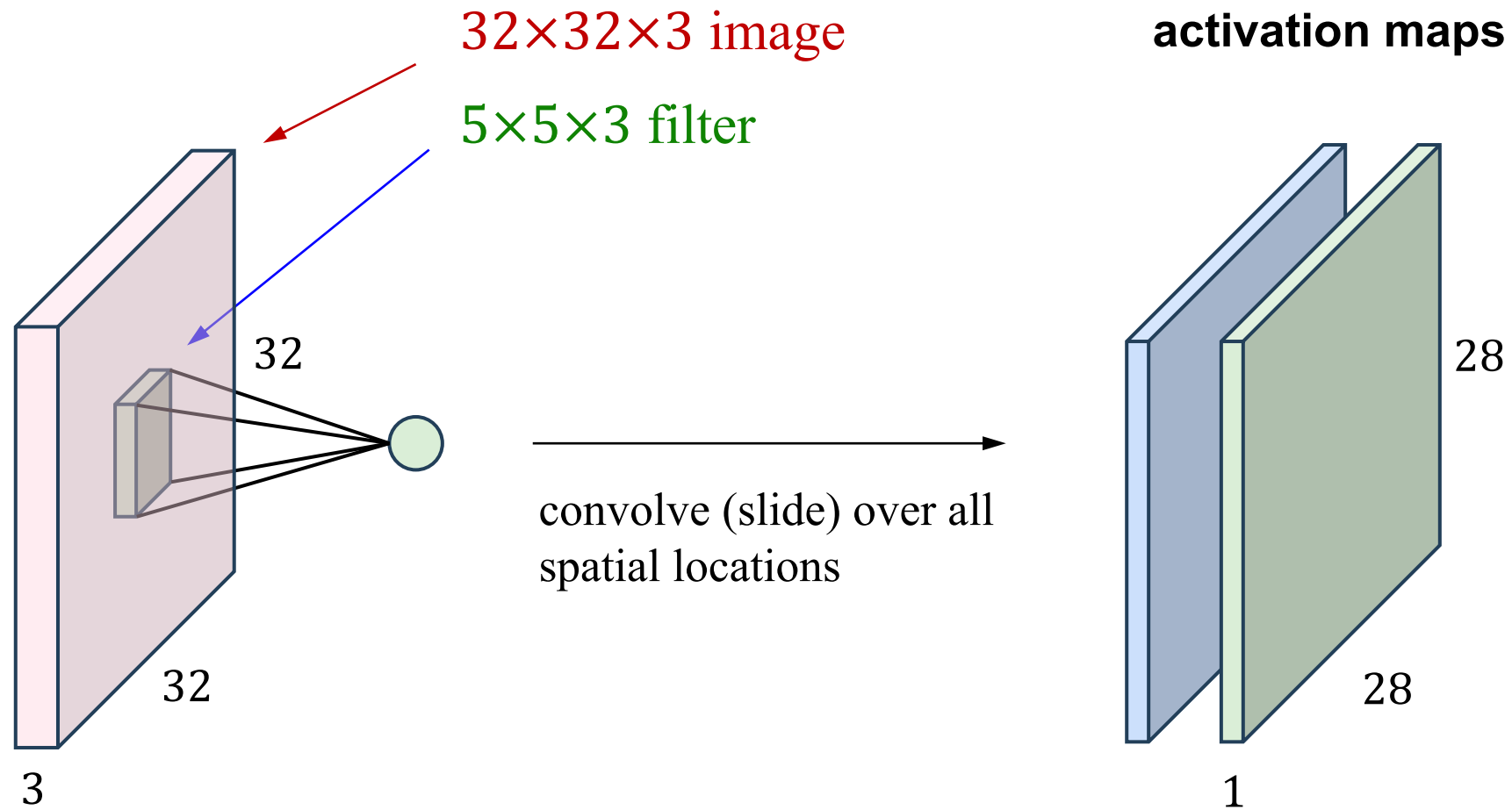
E.g. $F = 3 \Rightarrow$ zero pad with 1

$F = 5 \Rightarrow$ zero pad with 2

$F = 7 \Rightarrow$ zero pad with 3

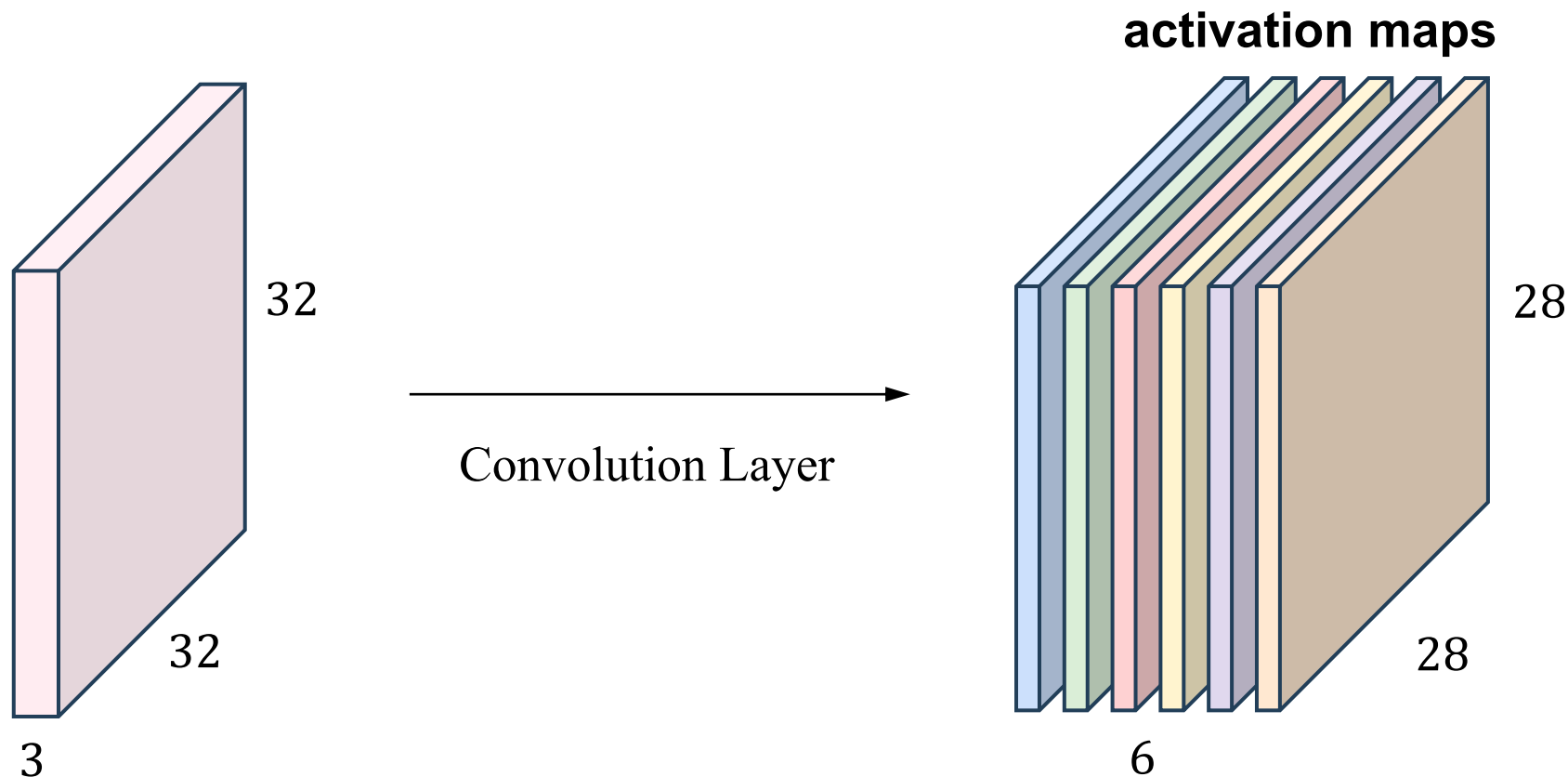
Convolution Layer

Consider a second, **green** filter



Convolution Layer

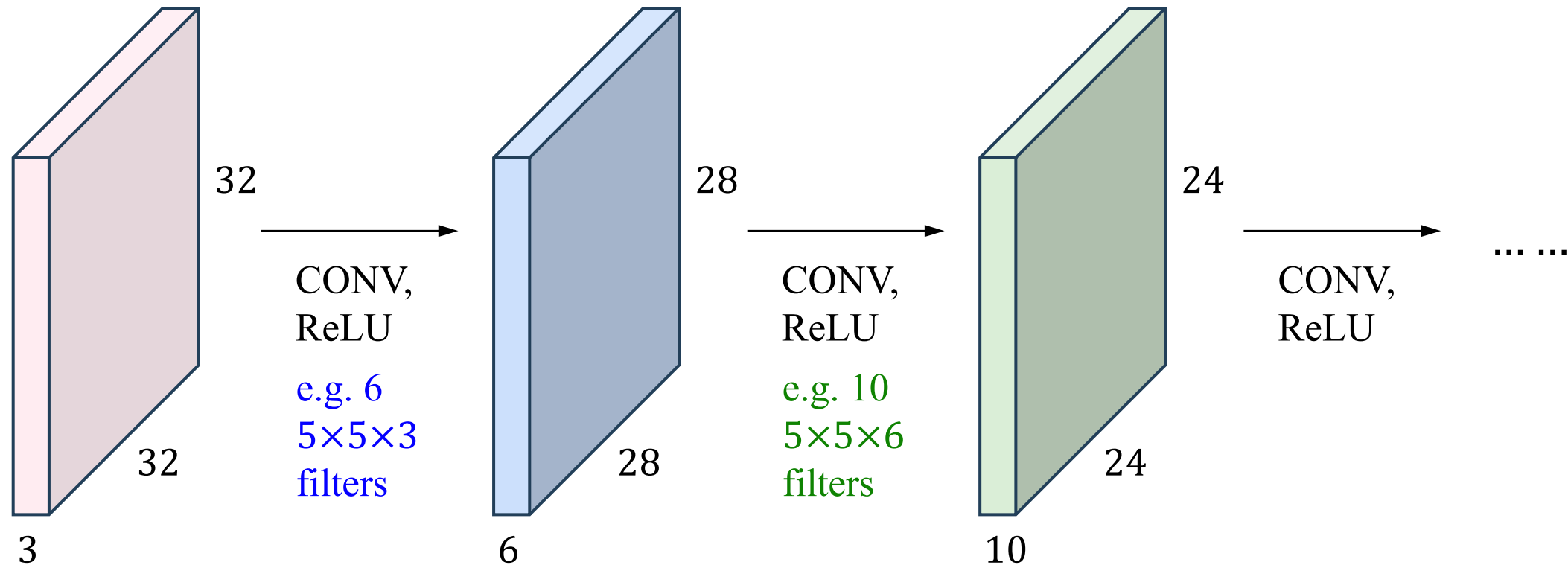
For example, if we had 6 5×5 filters, we'll get 6 separate activation maps:



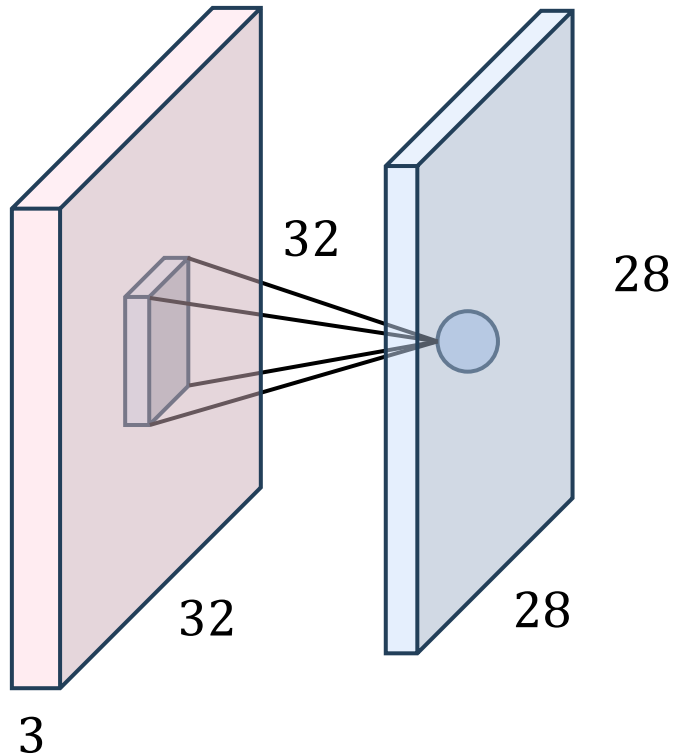
We stack these up to get a “new image” of size $28 \times 28 \times 6$!

Convolution Layer

ConvNet is a sequence of Convolution Layers with activation functions



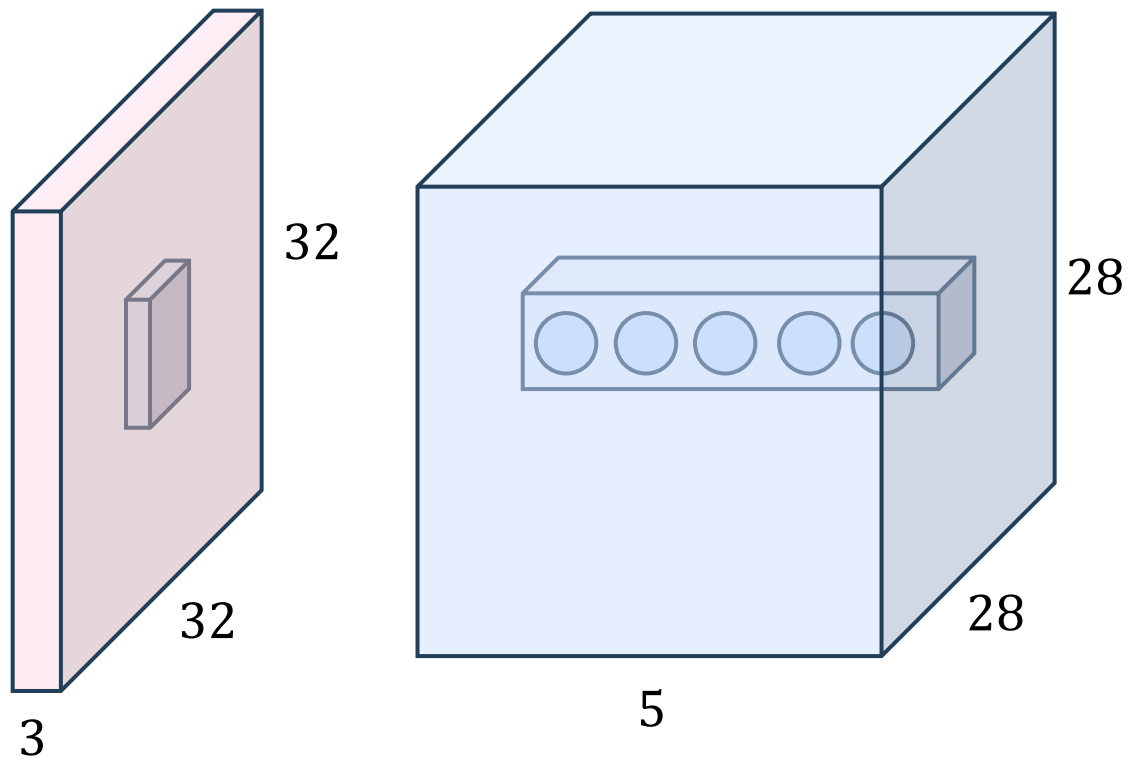
Receptive field



1. Each neuron is connected to a small region in the input
2. All of them share parameters

“5×5 filter” → “5×5 receptive field for each neuron”

Receptive field



E.g. with 5 filters, CONV layer consists of neurons arranged in a 3D grid ($28 \times 28 \times 5$)

There will be 5 different neurons all looking at the same region in the input volume

Convolution layer: summary

Let's assume input is $W_1 \times H_1 \times C$

Conv layer needs 4 hyperparameters:

- Number of filters **K**
- The filter size **F**
- The stride **S**
- The zero padding **P**

This will produce an output of $W_2 \times H_2 \times K$

where:

- $W_2 = (W_1 - F + 2P)/S + 1$
- $H_2 = (H_1 - F + 2P)/S + 1$

Number of parameters: $K \cdot (F^2 C + 1)$

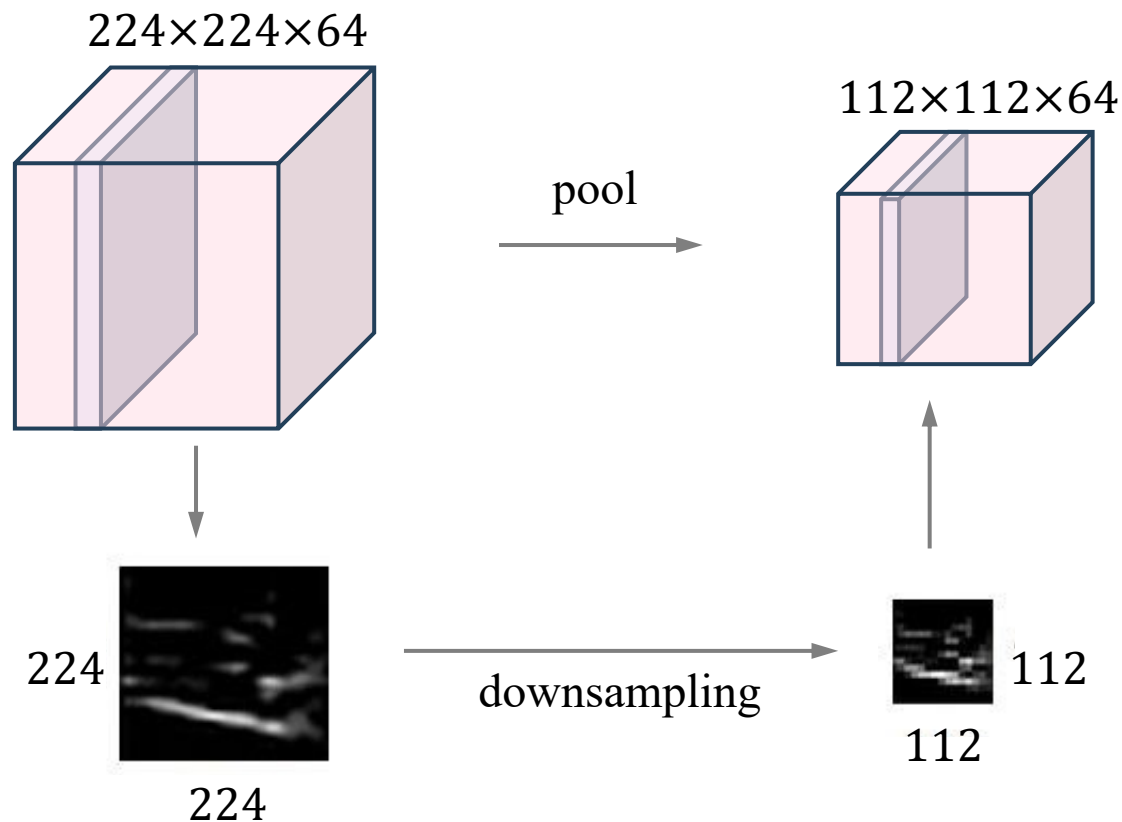
Common settings:

K = (powers of 2, e.g. 32, 64, 128, 512)

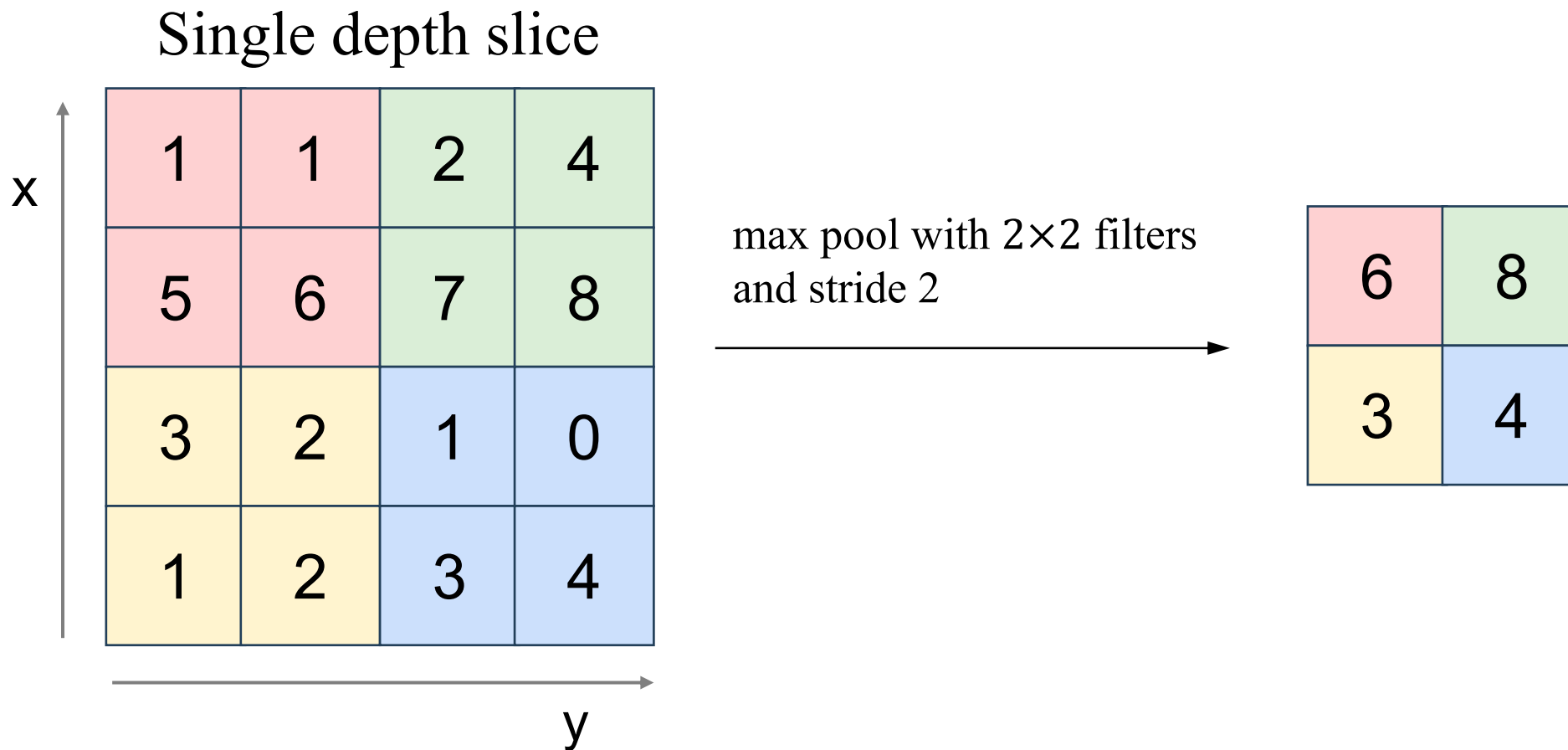
- $F = 3, S = 1, P = 1$
- $F = 5, S = 1, P = 2$
- $F = 5, S = 2, P = \text{whatever fits}$
- $F = 1, S = 1, P = 0$

Pooling layer

- Reduce the representation size
- Operates over each activation map independently:



MAX POOLING



Pooling layer: summary

Let's assume input is $W_1 \times H_1 \times C$

Pooling layer needs 2 hyperparameters:

- The spatial extent F
- The stride S

This will produce an output of $W_2 \times H_2 \times C$ where:

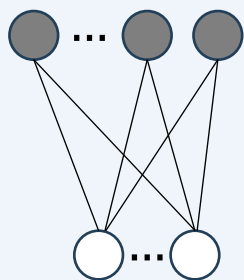
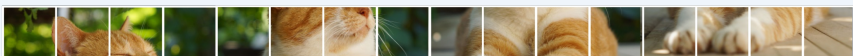
- $W_2 = (W_1 - F)/S + 1$
- $H_2 = (H_1 - F)/S + 1$

Number of parameters: 0

Summary of CNNs

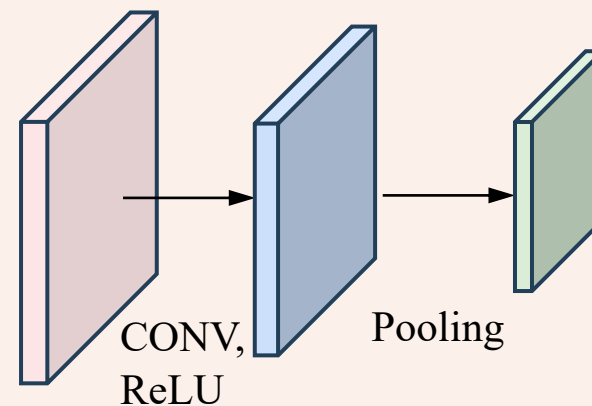
Fully Connected Net

- All-to-all connectivity
- Requires a large number of weights for large images
- Vulnerable to spatial variance



ConvNet

- Local connectivity
- Parameter sharing across regions (filter)
- Built-in capability of handling spatial variance



Recurrent Neural Networks

Learning from Sequential Observations

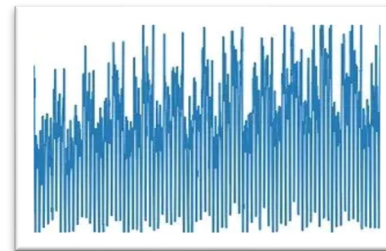
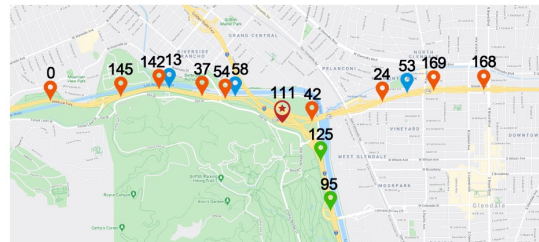


Agent

Natural language, Speech



Sensor Observations (Weather, Traffic, Environment, Industries)

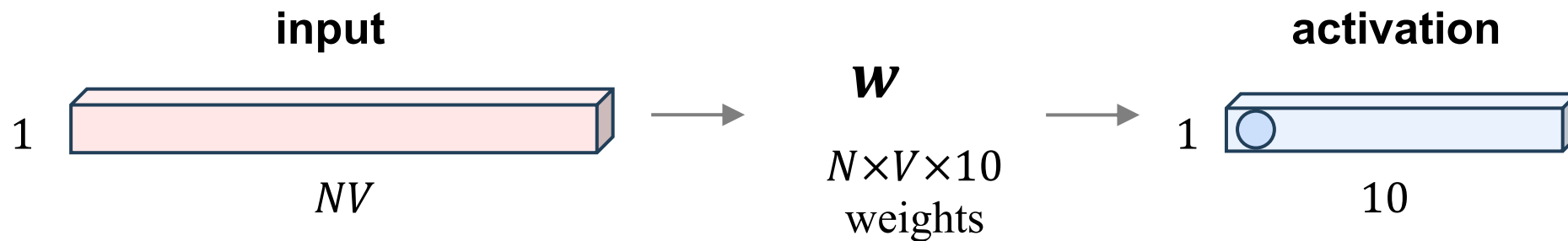


Stock market



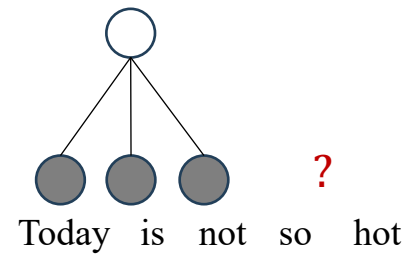
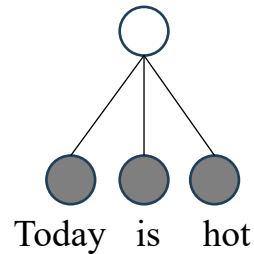
First attempt: Fully Connected Network

- Consider classifying a document to one of 10 topics/classes
- A document is a sequence of N words
- Suppose the vocabulary contains V words, each word is represented by a one-hot vector
- The document can be represented by a $N \times V$ tensor to $NV \times 1$ vector x
- Apply $y = \sigma(w^T x)$



First attempt: Fully Connected Network

- Problem of a simple fully connected network in handling sequences
 - **Fixed input/output size**: Cannot naturally handle variable-length sequences



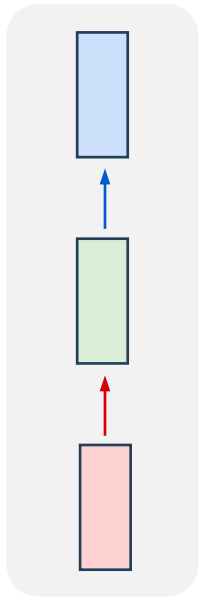
- **No sharing weights**: Each time step has a different weight, number of weights increases linearly to the length of input
- **No memory for the past**: It views each word independently, with no capability in handling temporal dependency.

Today is **not** so **hot**



Recurrent Neural Networks: Process Sequences

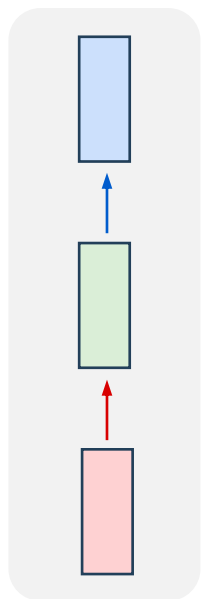
one to one



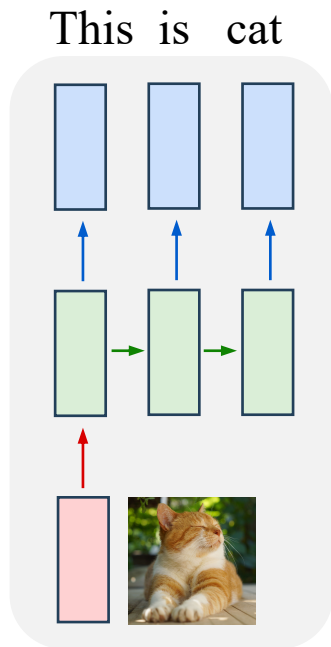
Vanilla Neural Networks

Recurrent Neural Networks

one to one

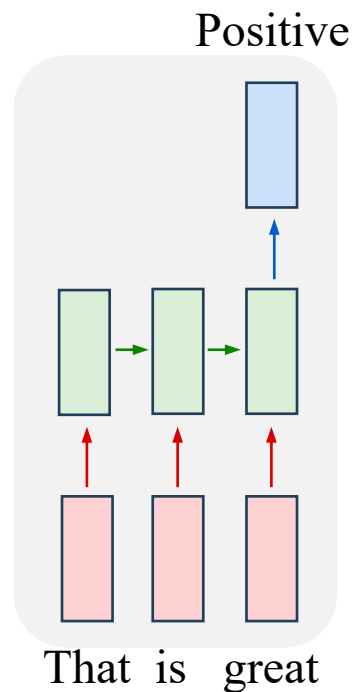


one to many



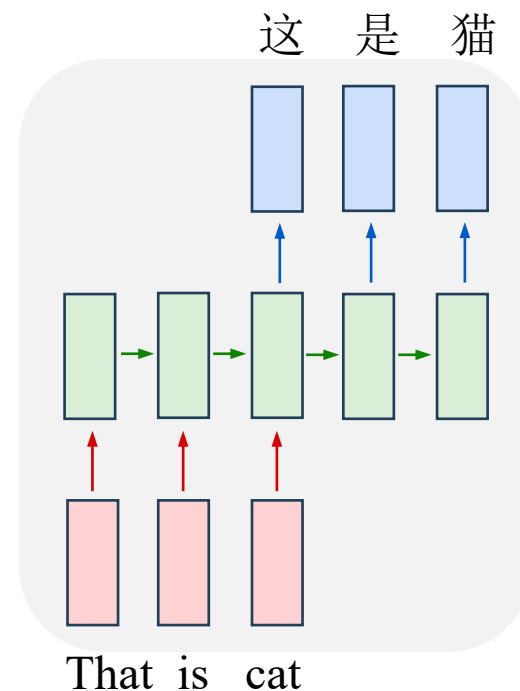
Sequence generation
e.g., image captioning
text generation

many to one



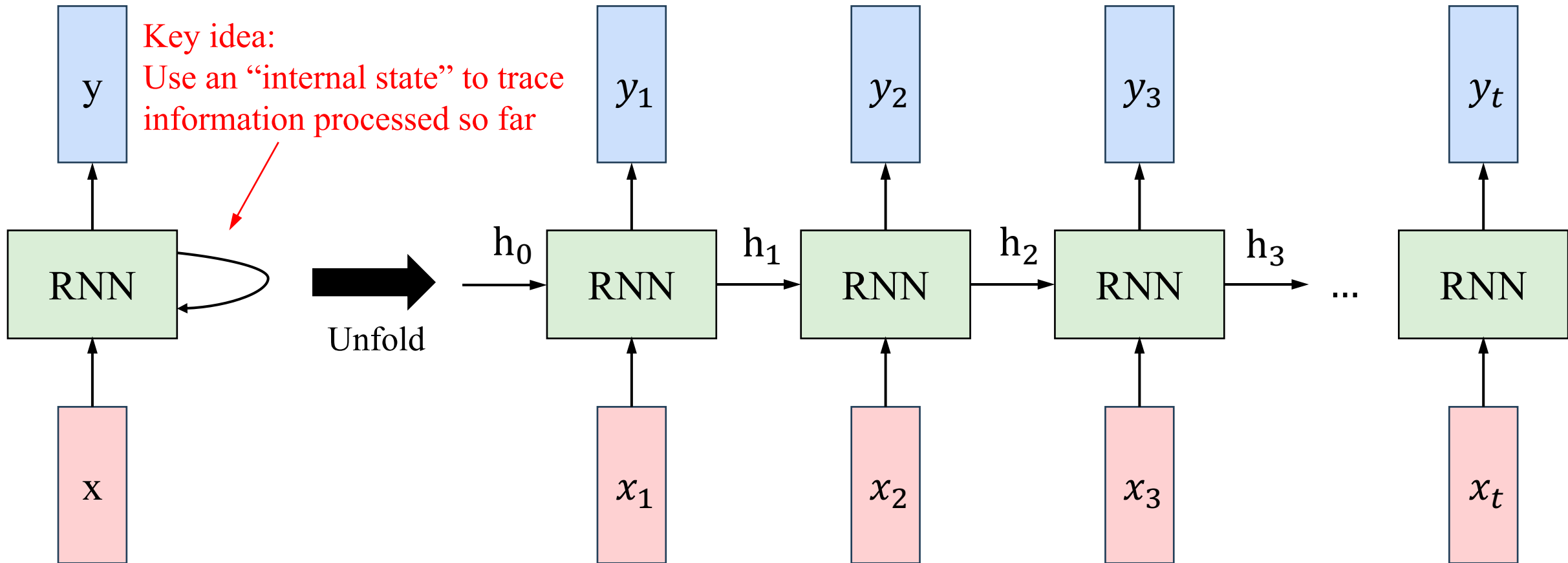
Sequence classification
e.g., document classification
Video/Audio classification

many to many



Translation
e.g., language translations
video to text

Recurrent Neural Networks



RNN hidden state update

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

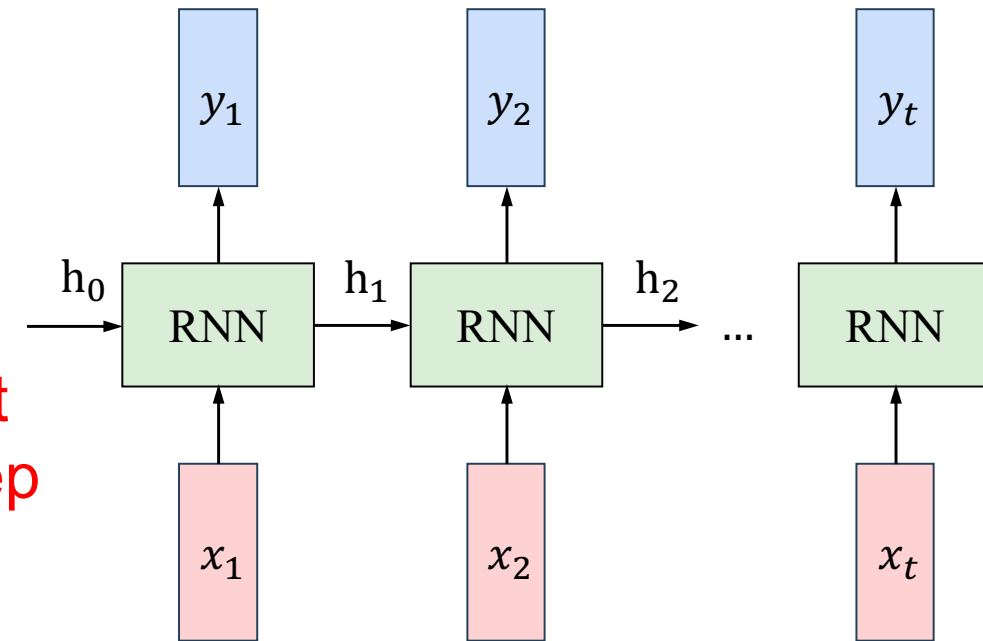
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

some function with weights W

old state

input vector at some time step



RNN output generation

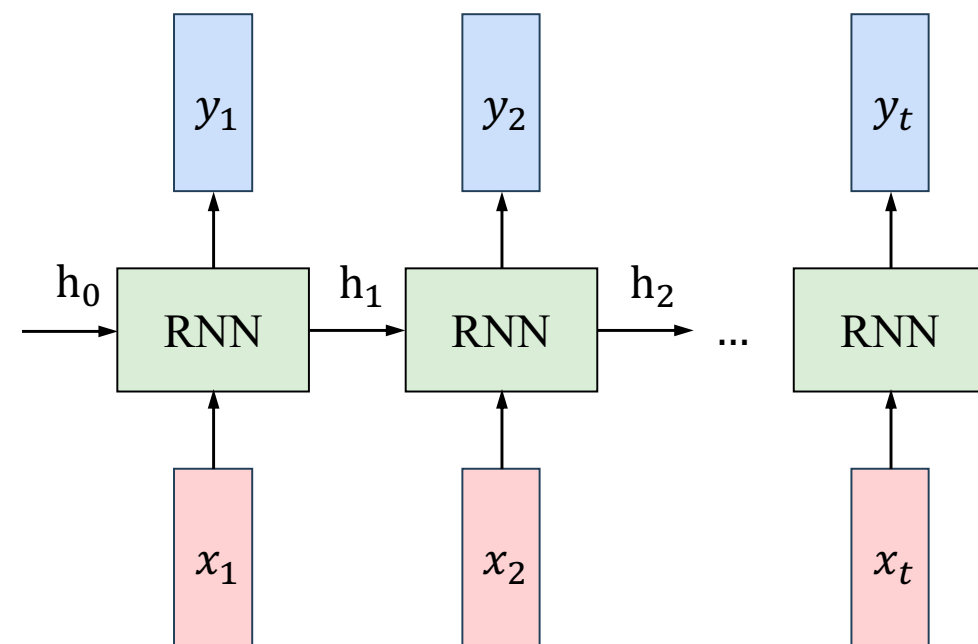
We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$\boxed{y_t} = \boxed{f_{W_{hy}}}(\boxed{h_t})$$

output

another function
with parameters W_0

new state



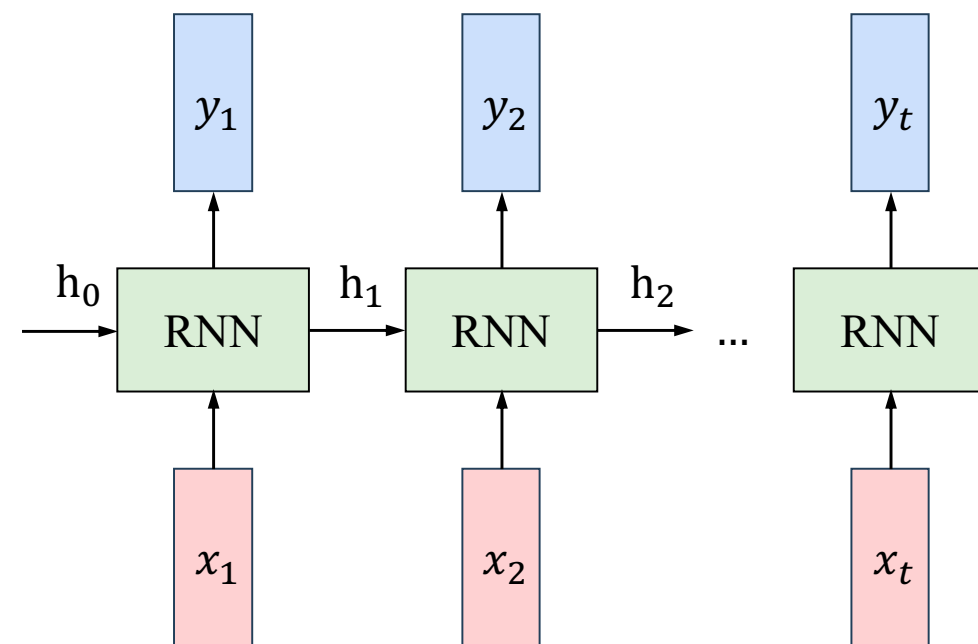
Recurrent Neural Network

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$h_t = f_W (h_{t-1}, x_t)$$

$$y_t = f_{W_{hy}} (h_t)$$

Notice: the same function and the same set of weights are used at every time step



(Vanilla) Recurrent Neural Network

The state consist of a single “*hidden*” vector h :

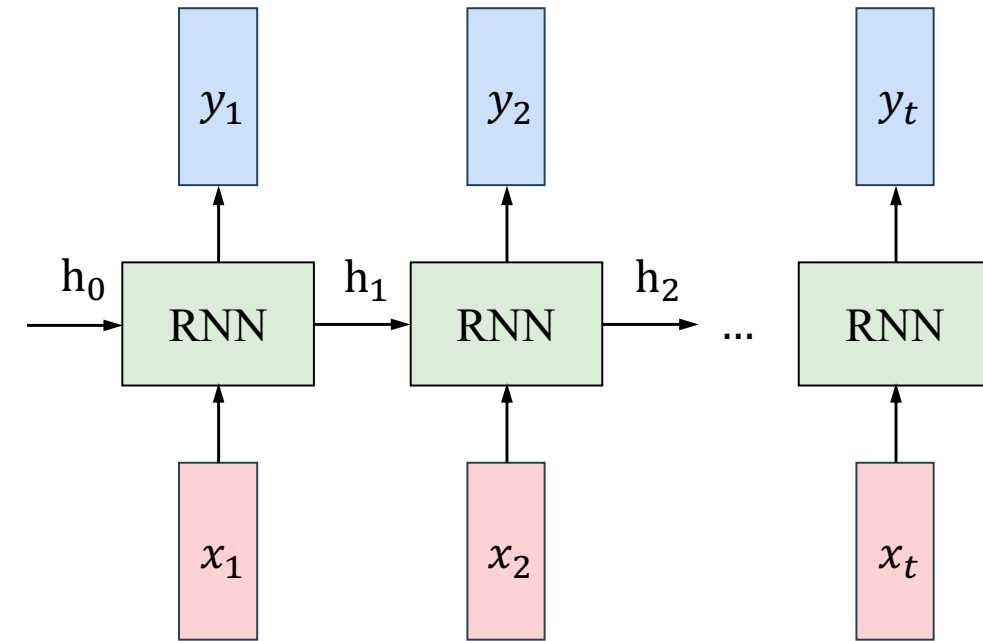
$$h_t = f_W (h_{t-1}, x_t)$$

$$y_t = f_{W_{hy}} (h_t)$$

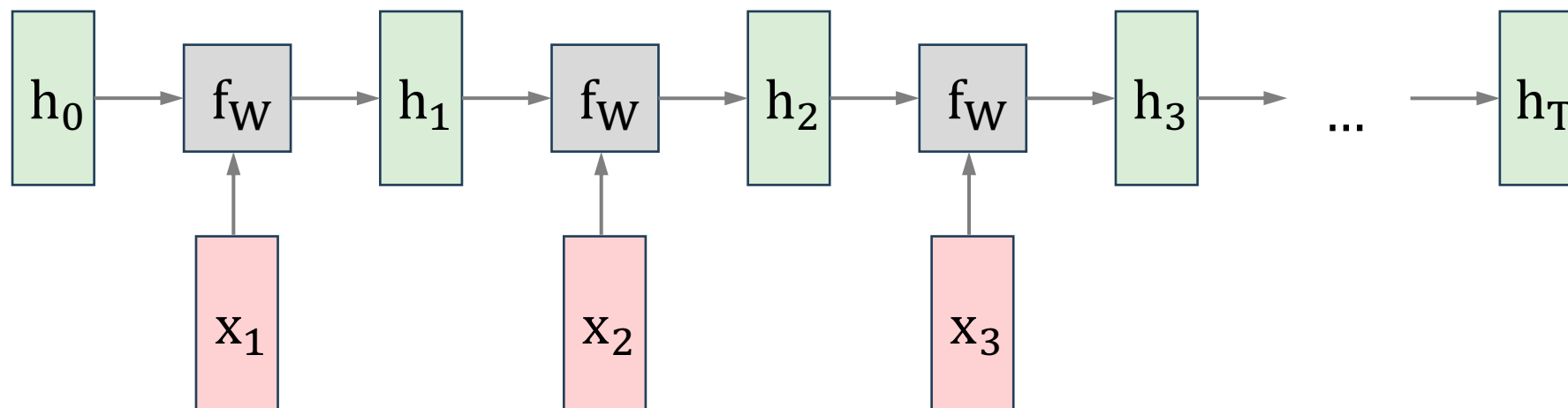


$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

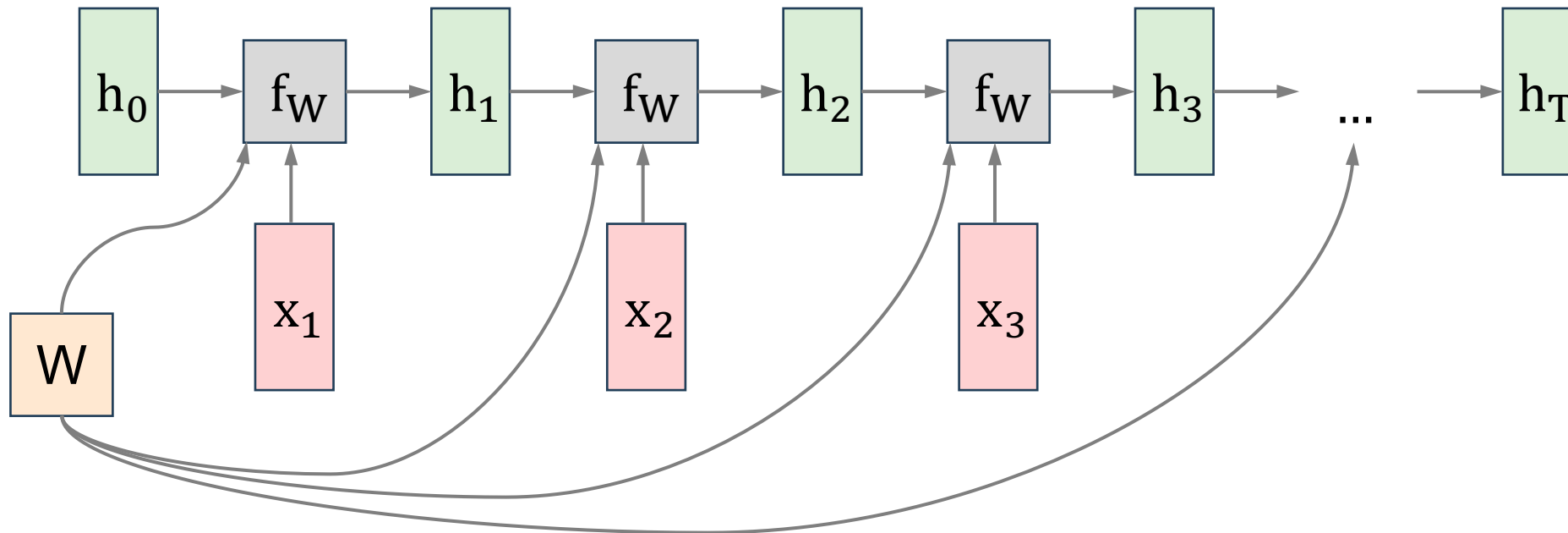


RNN: Computational Graph

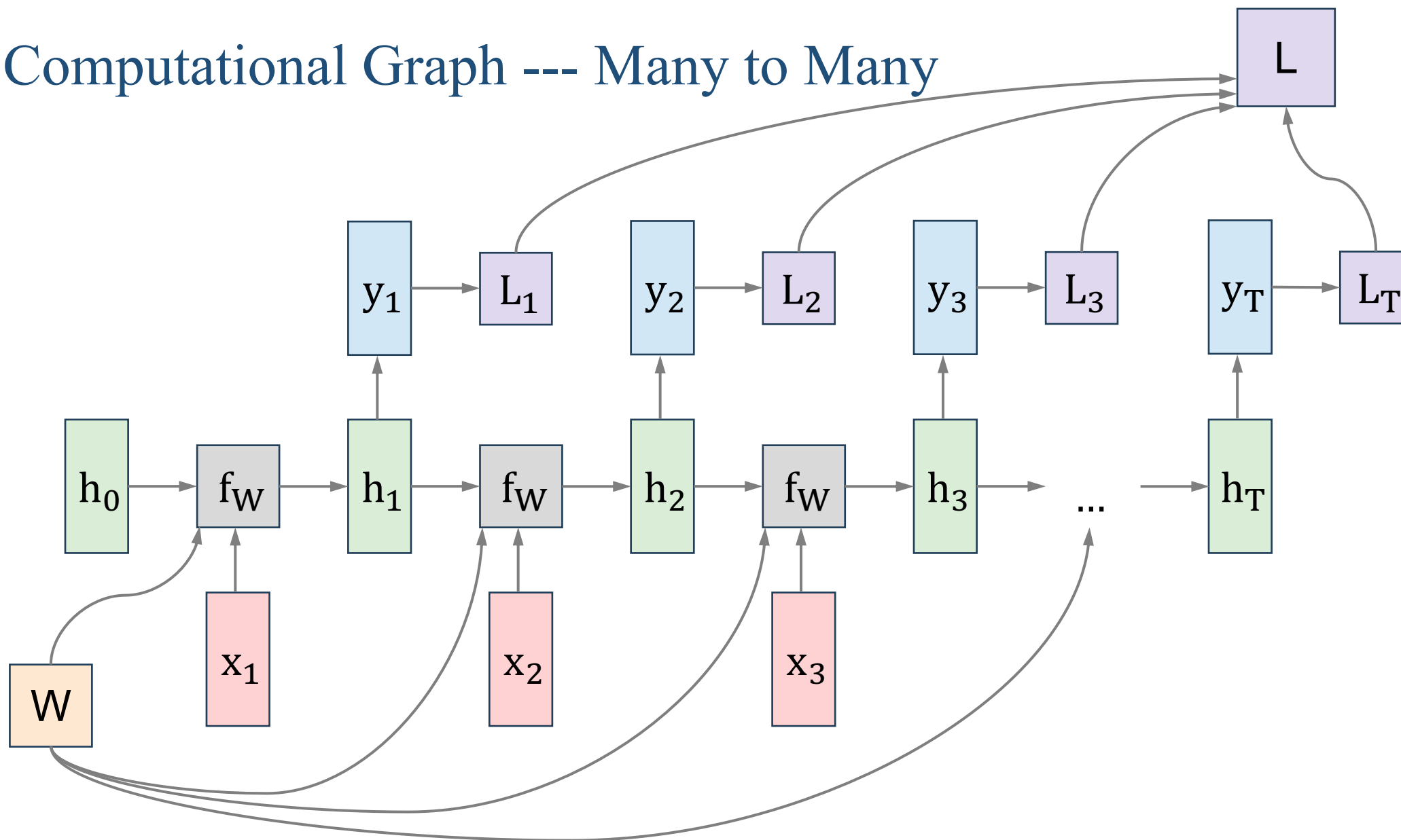


Computational Graph of RNNs

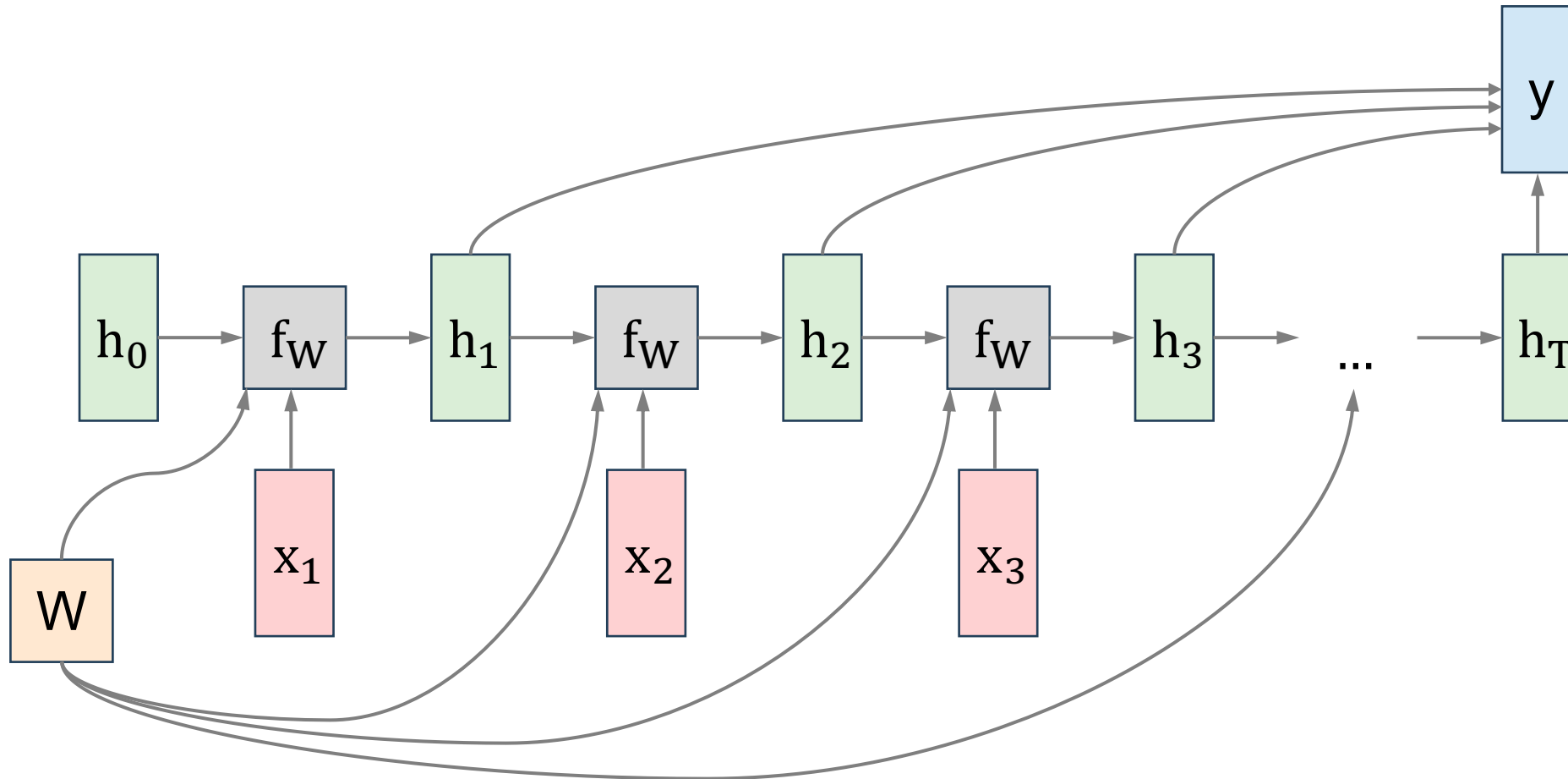
Re-use the same weight matrix at every time-step



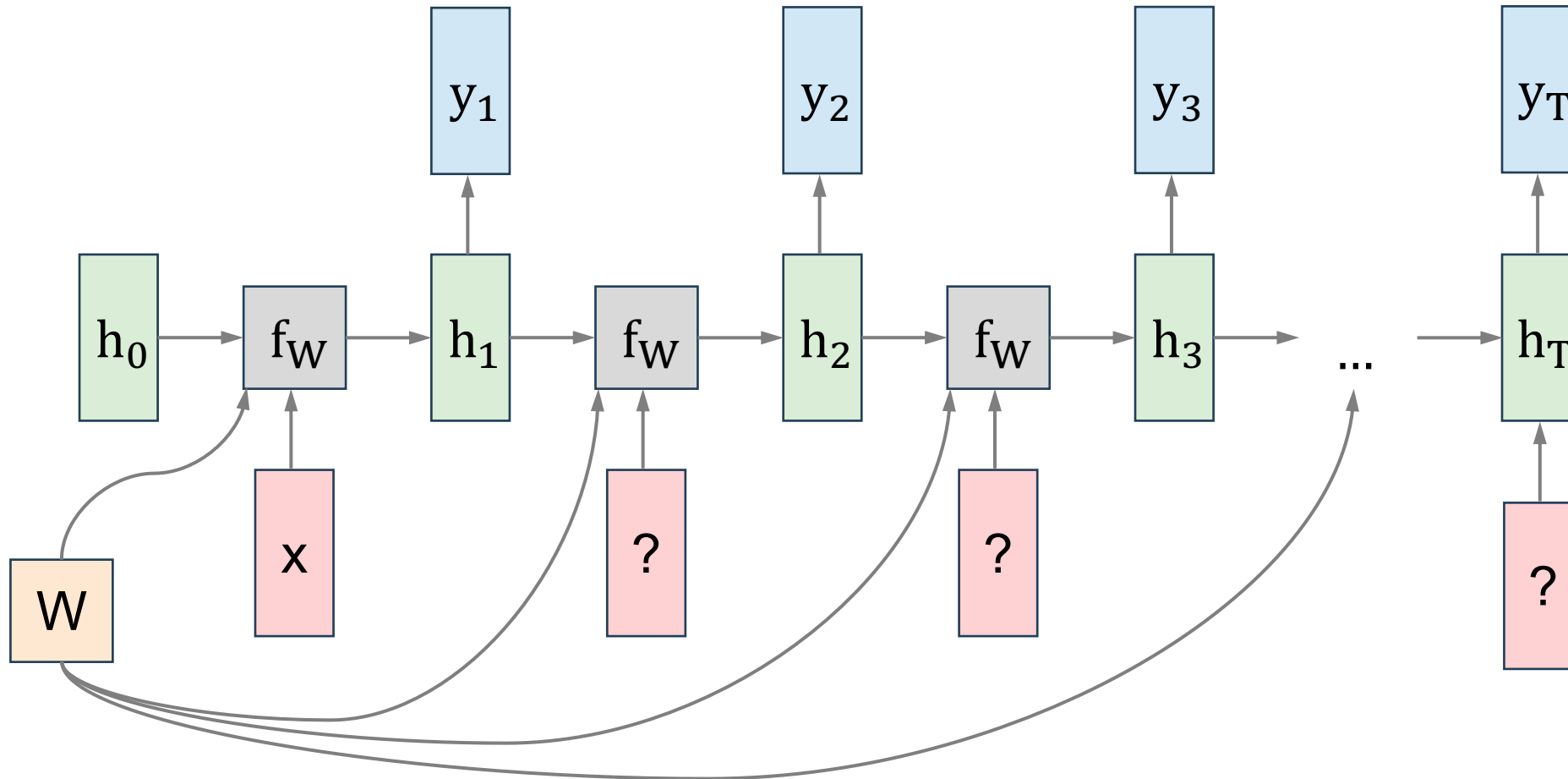
Computational Graph --- Many to Many



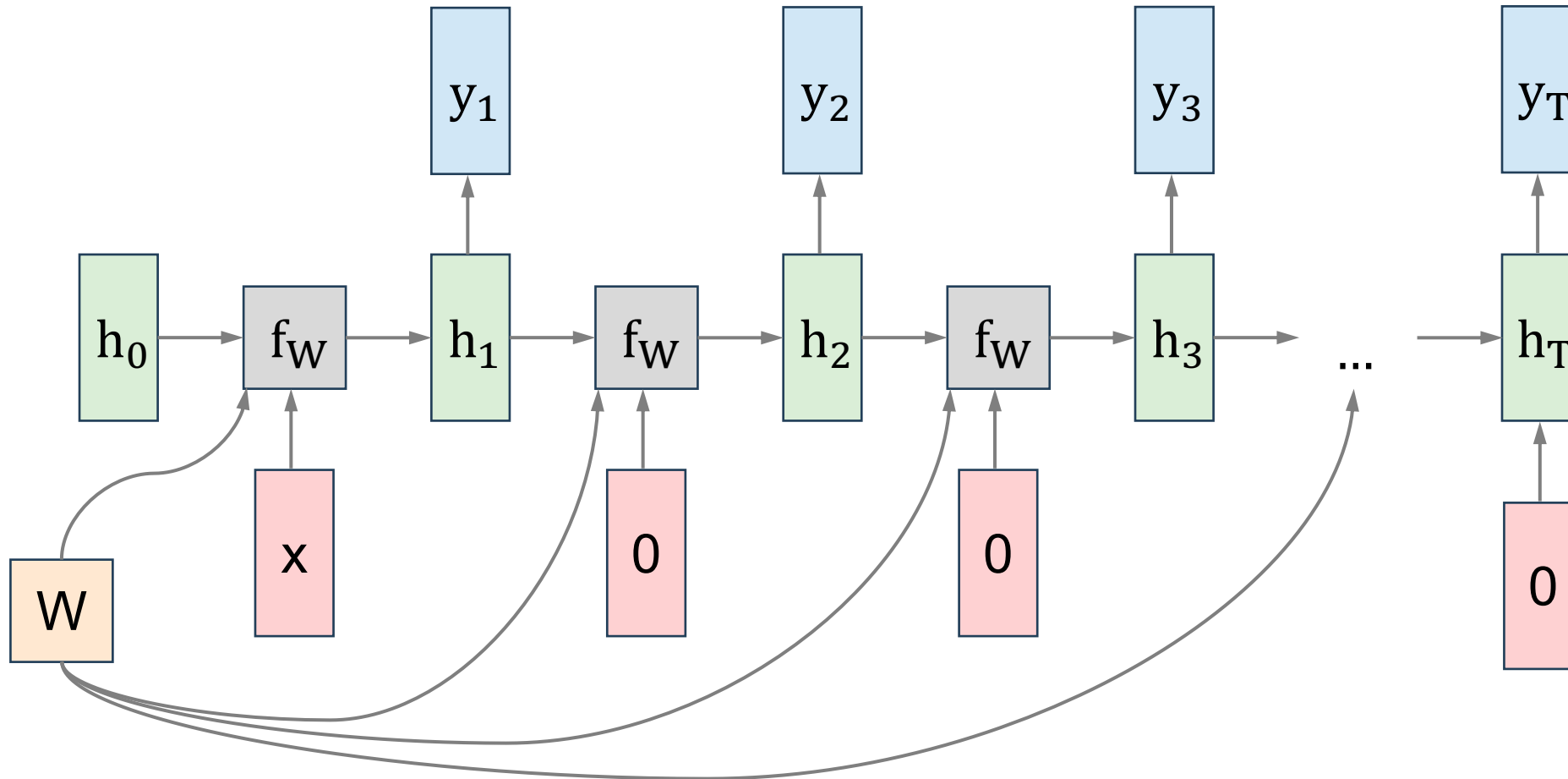
RNN: Computational Graph --- Many to One



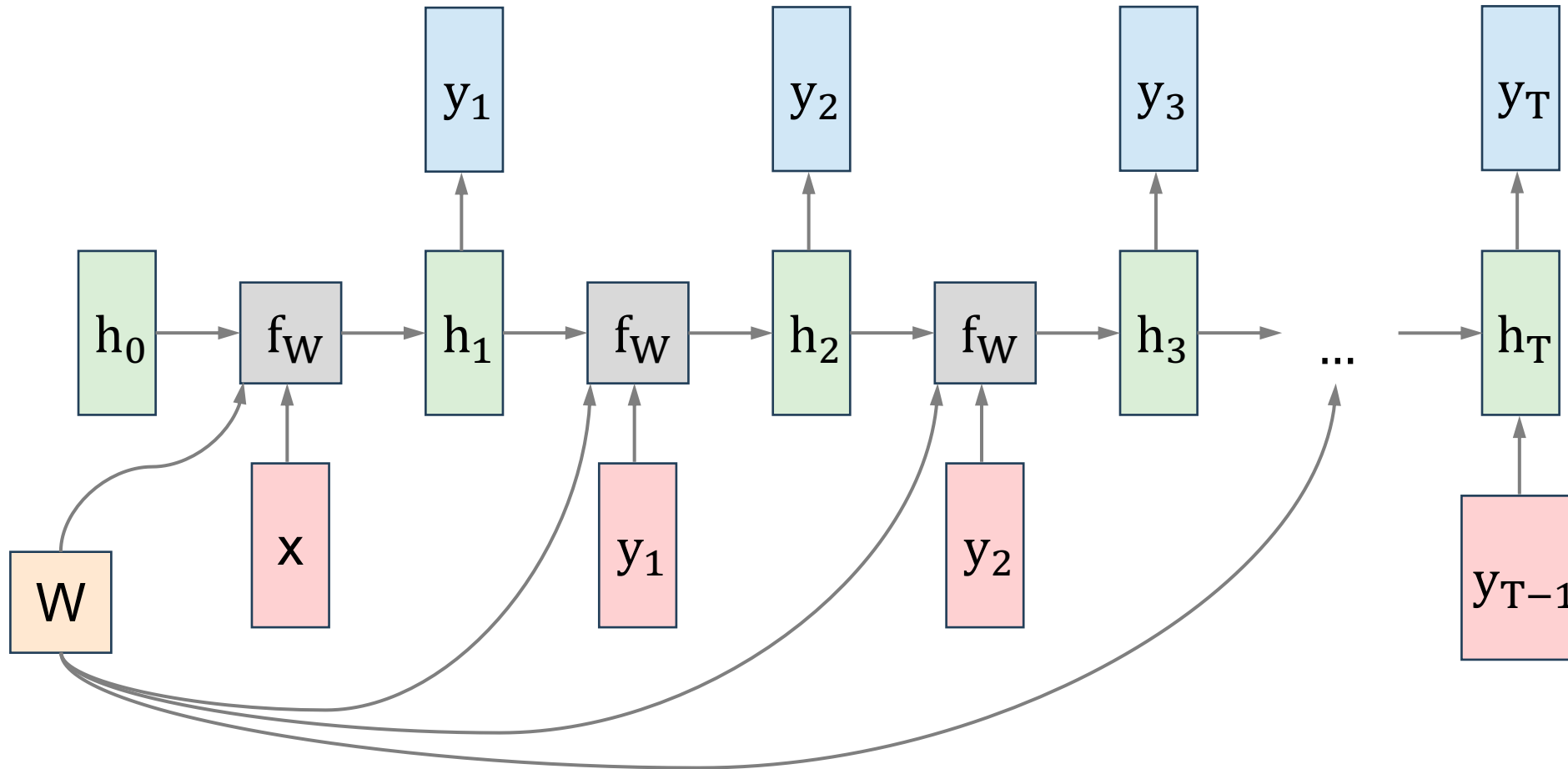
RNN: Computational Graph --- One to Many



RNN: Computational Graph --- One to Many

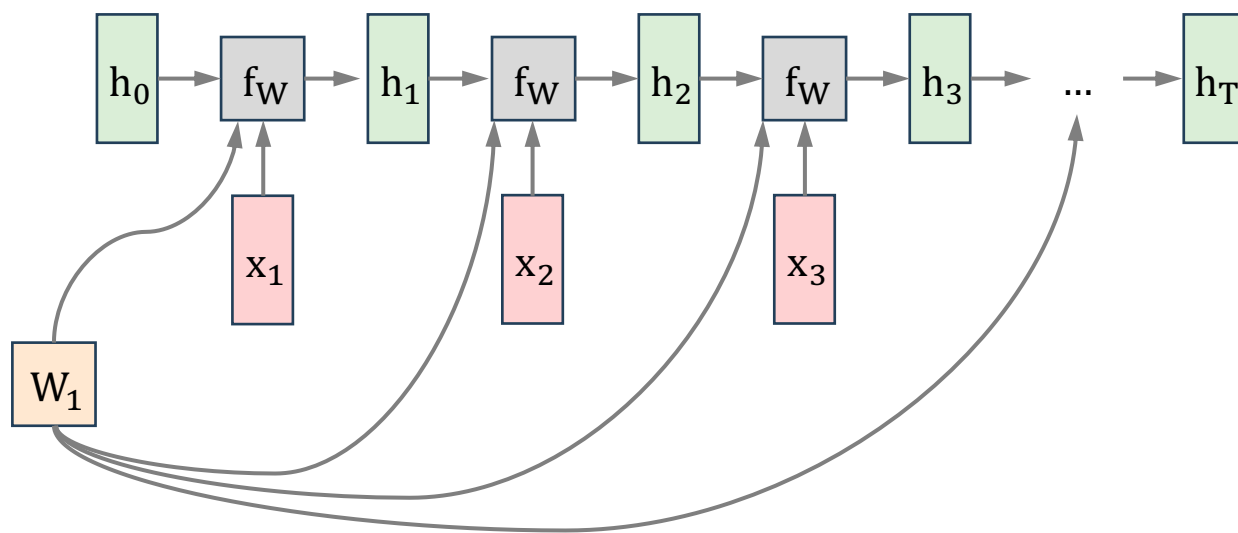


RNN: Computational Graph --- One to Many

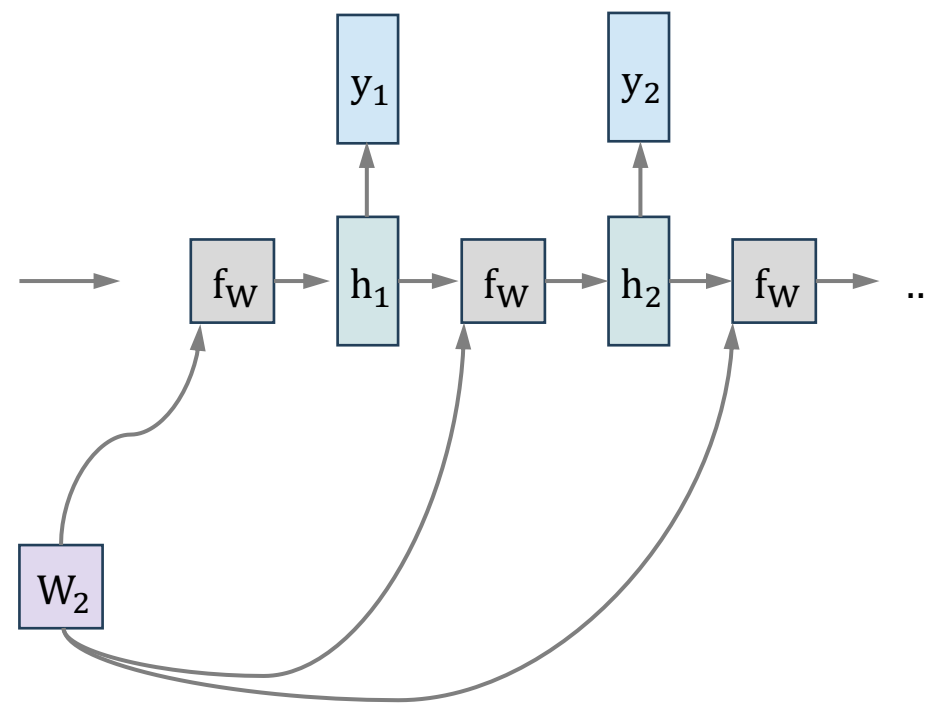


Sequence to Sequence --- many-to-one + one-to-many

many to one: Encode input sequence in a single vector

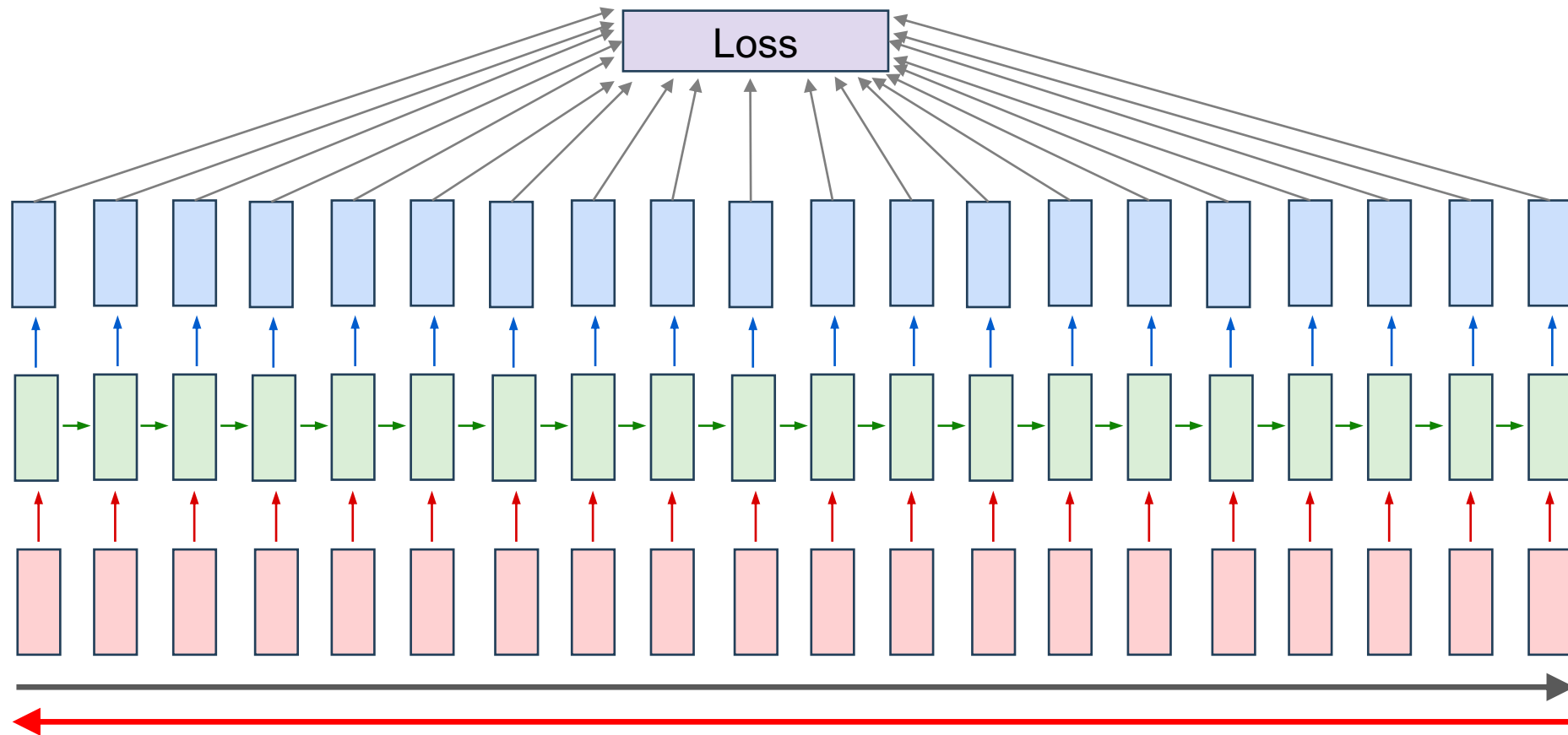


one to many: Produce output sequence from single input vector



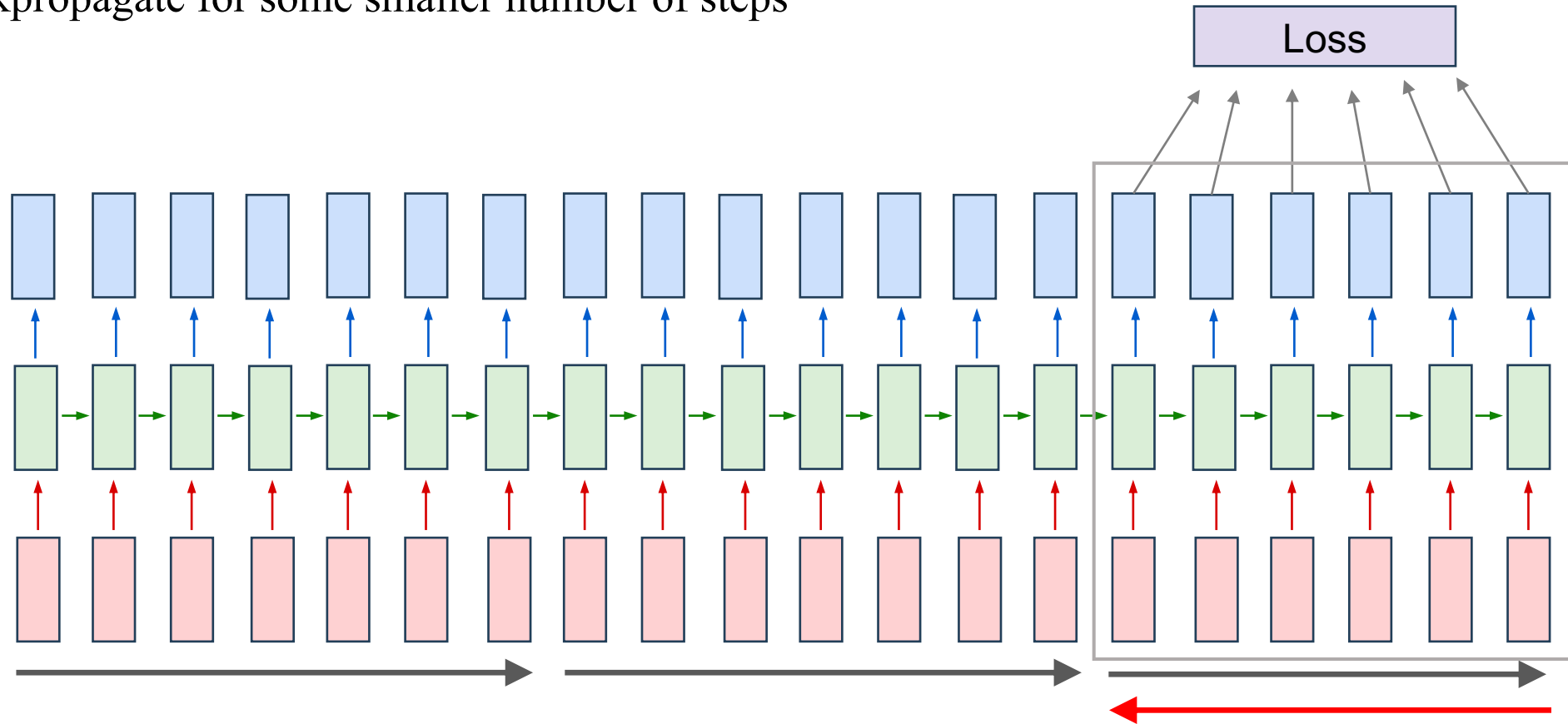
Backpropagation through Time

Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient



Truncated Backpropagation through Time

Carry hidden states forward in time forever, but only backpropagate for some smaller number of steps



Properties of RNN

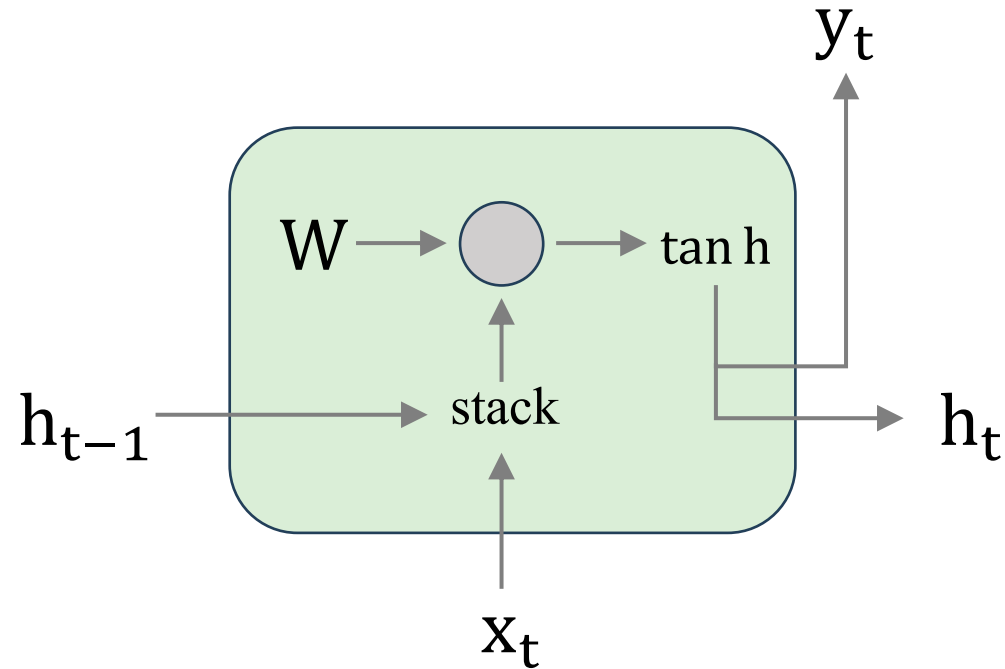
Advantages:

- Support variable input lengths
- Use the past information when calculating any step t
- Model size remain the same for different input lengths

Disadvantages:

- Recurrent computation is slow
- The **vanishing gradient problem** – forget distant information in the past

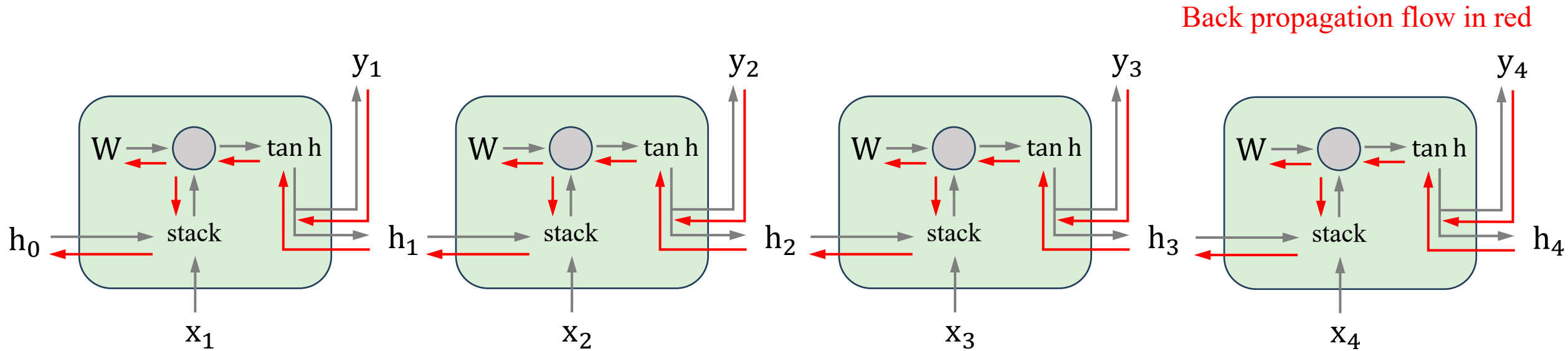
RNN – Vanishing Gradient



For simplicity, ignore W_{hy}

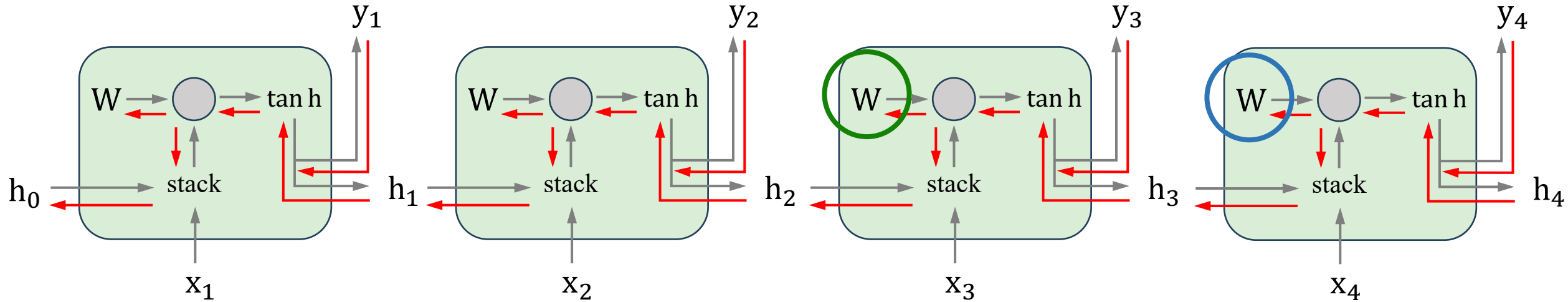
$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \ W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

RNN – Vanishing Gradient



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

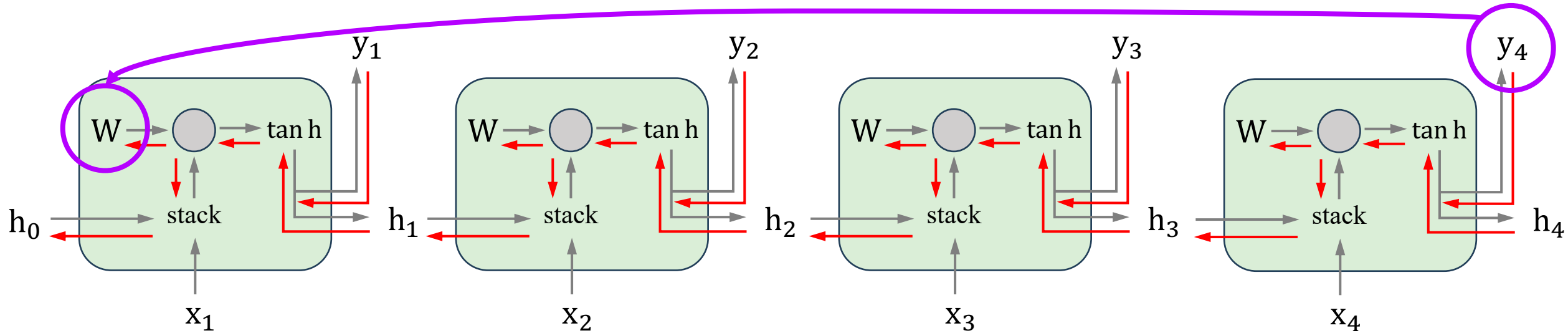
RNN – Vanishing Gradient



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \underbrace{\frac{\partial L_T}{\partial h_T} \cdot \frac{\partial h_T}{\partial W}}_{\text{Gradient from step T}} + \underbrace{\frac{\partial L_T}{\partial h_T} \cdot \frac{\partial h_T}{\partial h_{T-1}} \cdot \frac{\partial h_{T-1}}{\partial W}}_{\text{Gradient from step T-1}} + \dots$$

RNN – Vanishing Gradient



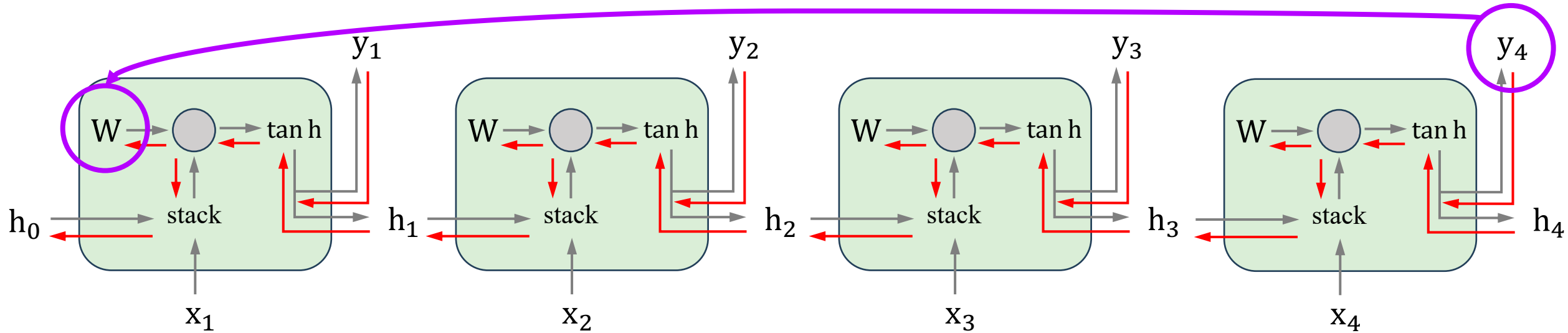
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \cdot \frac{\partial h_T}{\partial W} + \frac{\partial L_T}{\partial h_T} \cdot \frac{\partial h_T}{\partial h_{T-1}} \cdot \frac{\partial h_{T-1}}{\partial W} + \dots$$

Back to the first time step:

$$\frac{\partial L_T}{\partial h_T} \frac{\partial h_T}{\partial h_{T-1}} \dots \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

RNN – Vanishing Gradient

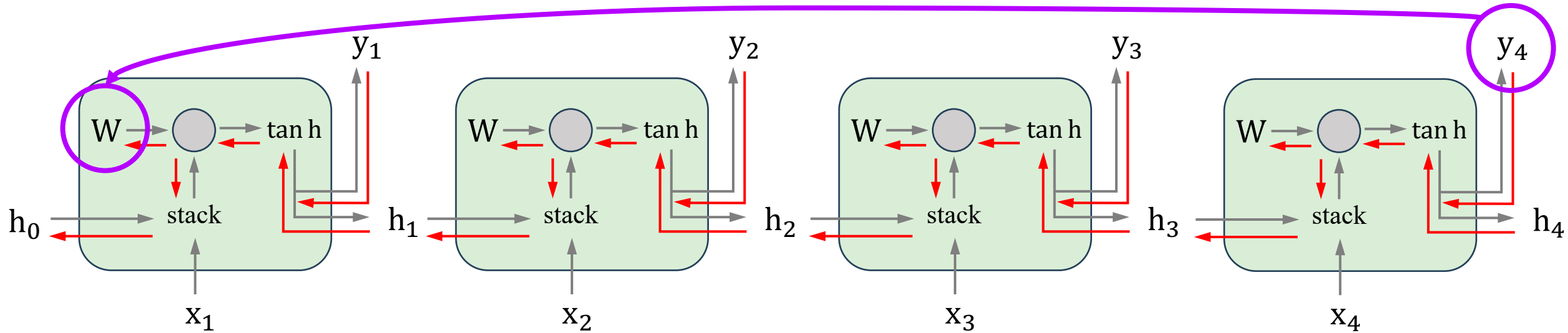


$$\frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

Back to the first time step:

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}$$

RNN – Vanishing Gradient



$\tanh'(x) \in (0,1]$
If $x \neq 0$, $\tanh'(x) < 1$

If largest singular value < 1 ,
for large T , $W_{hh}^{T-1} \rightarrow 0$

$$\frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \tanh'(W_{hh}h_{t-1} + W_{xh}x_t) \right) W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

The gradient will vanish
as T increases!

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

Four gates



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

Cell state



$$c_t = f \odot c_{t-1} + i \odot g$$

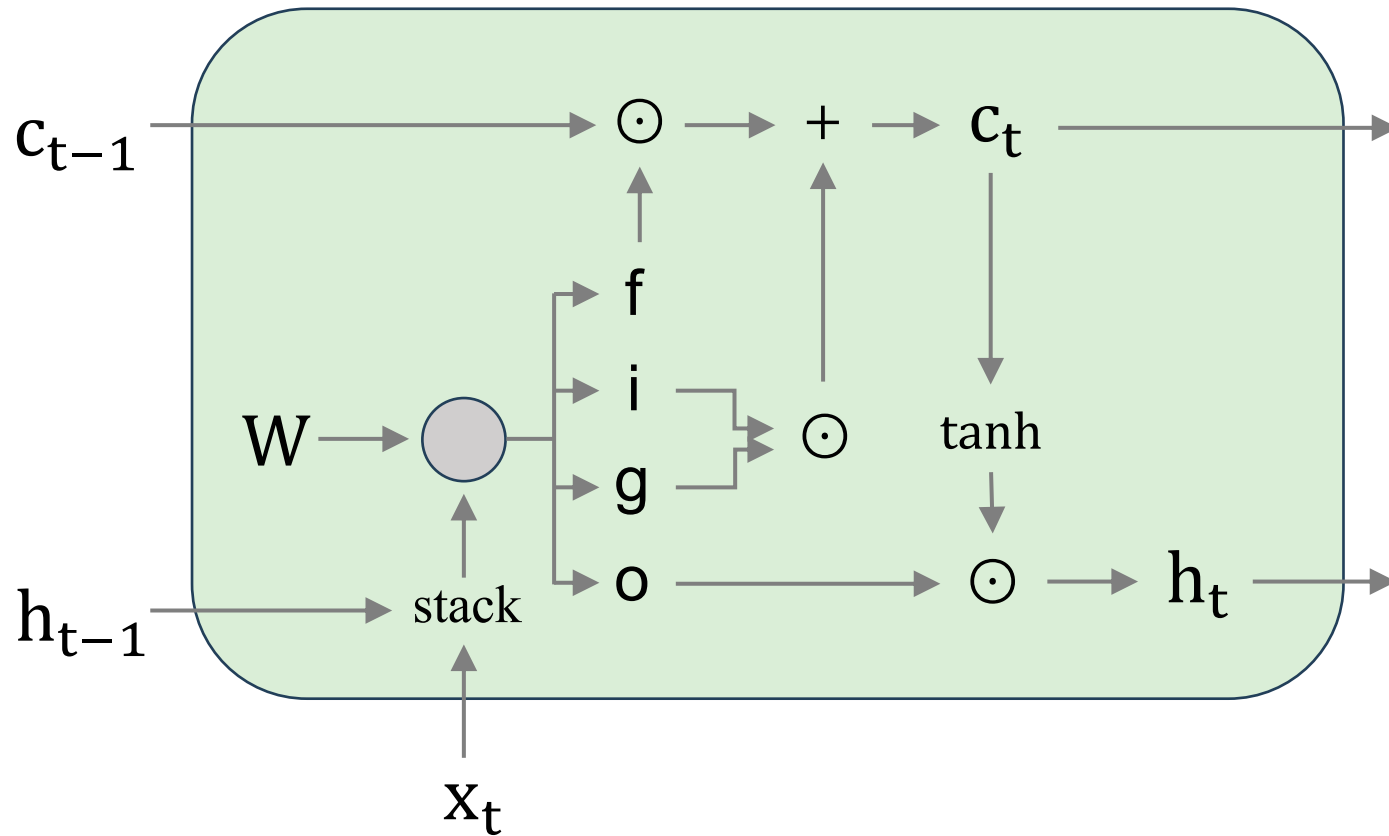
Hidden state



$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



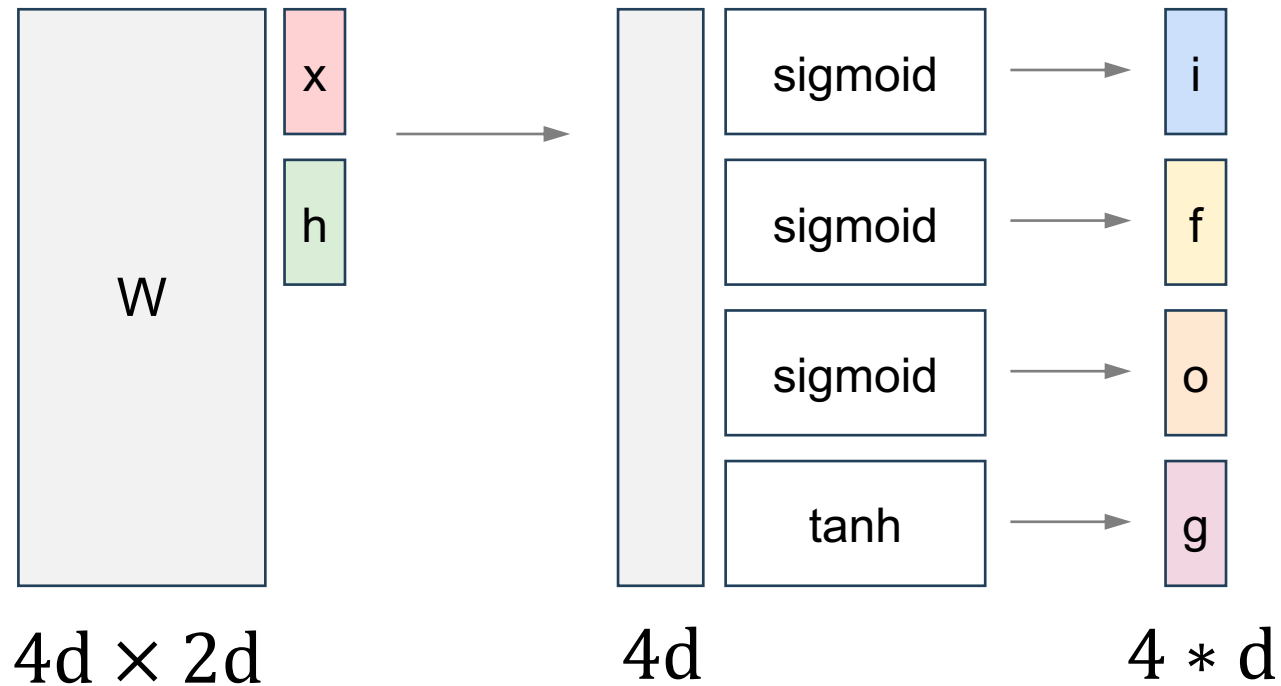
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

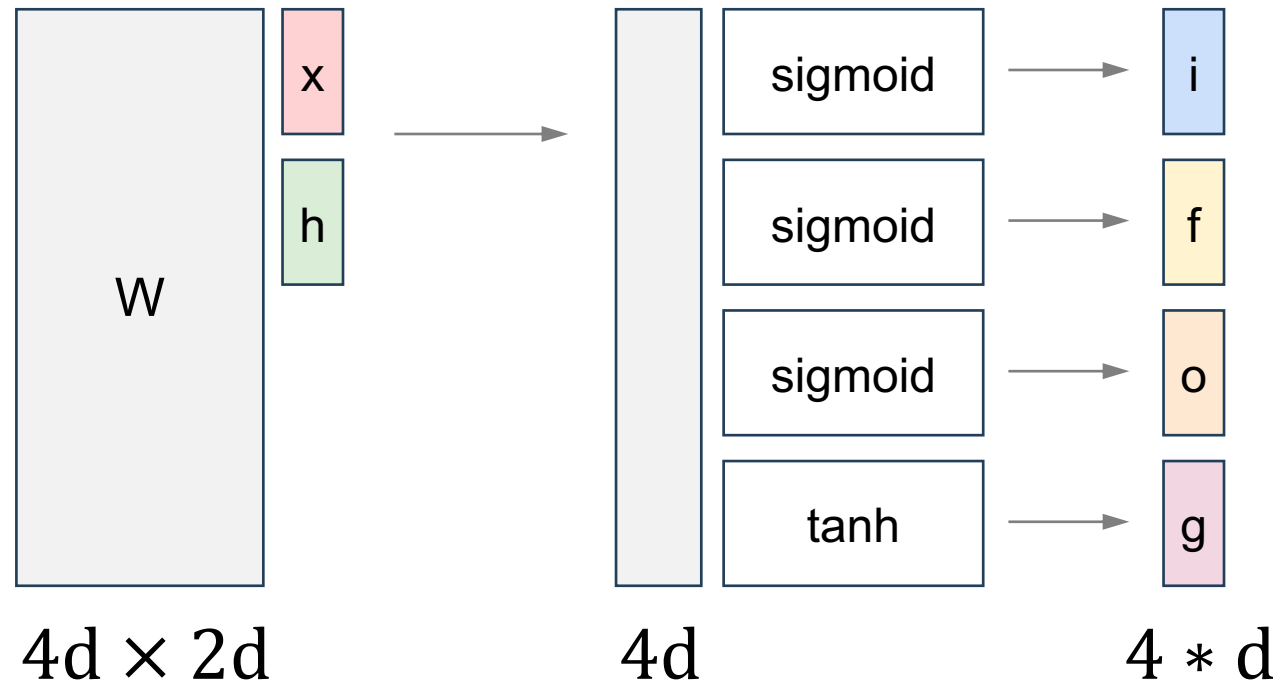
[Hochreiter et al., 1997]



Let d be the dimension of x and h

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



Let d be the dimension of x and h

g: The new information, identical to RNN

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

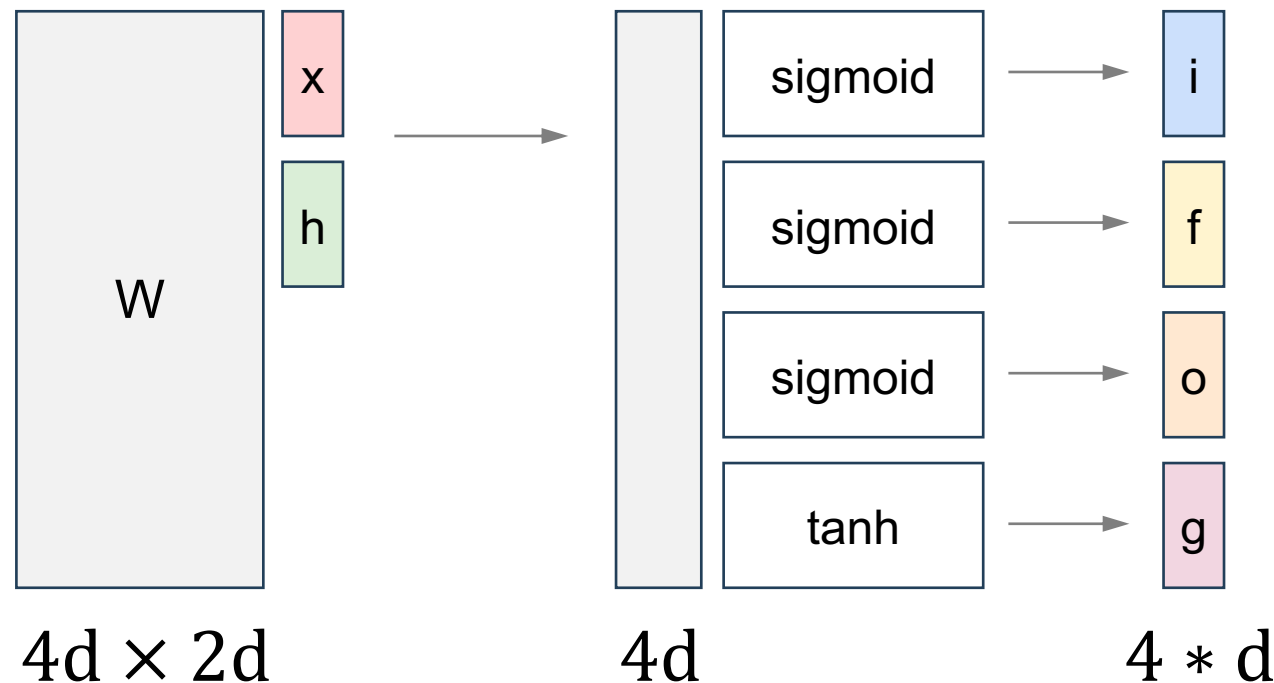
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]

i: Input gate, whether to write to cell



Let d be the dimension of x and h

g: The new information, identical to RNN

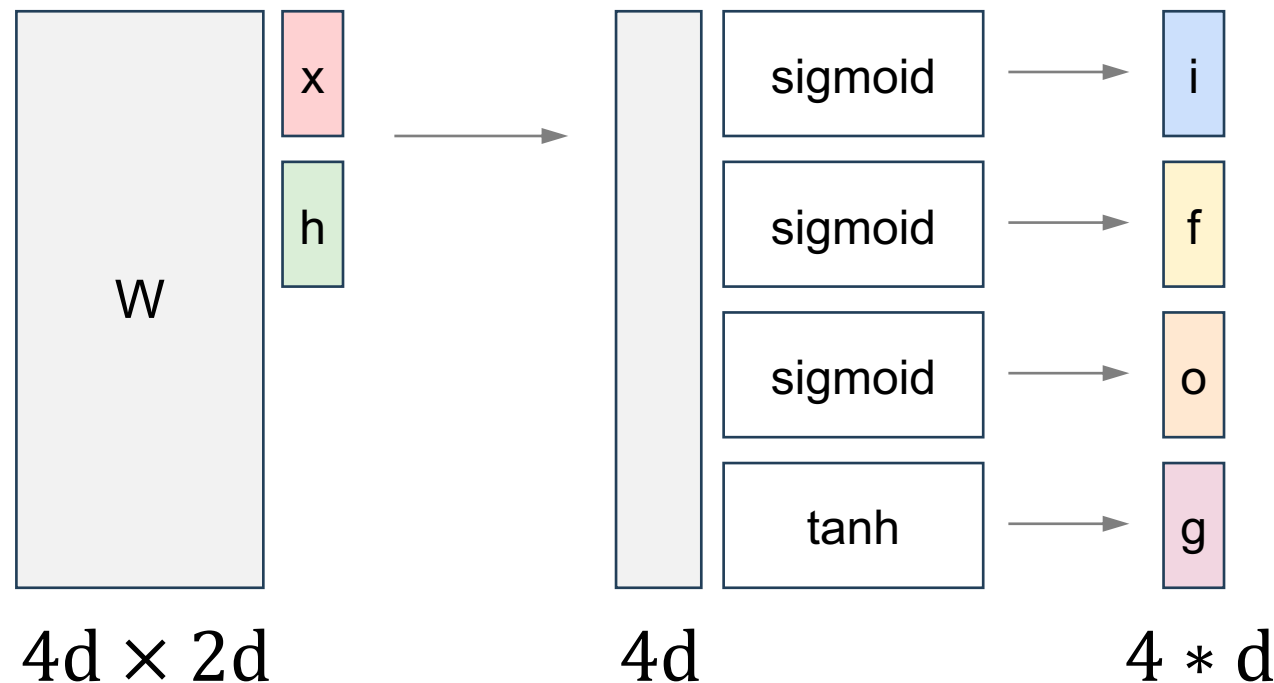
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



Let d be the dimension of x and h

i: Input gate, whether to write to cell

f: Forget gate, whether to erase cell

g: The new information, identical to RNN

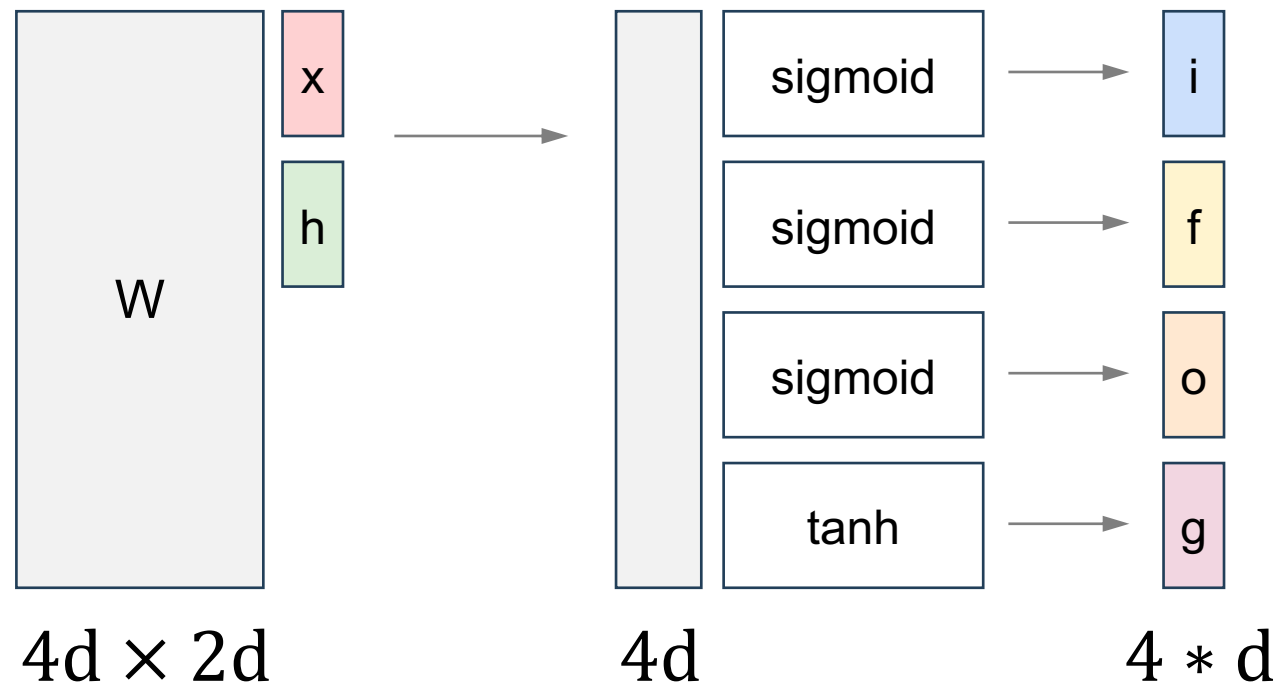
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



Let d be the dimension of x and h

i: Input gate, whether to write to cell

f: Forget gate, whether to erase cell

o: Output gate, whether to read from cell

g: The new information, identical to RNN

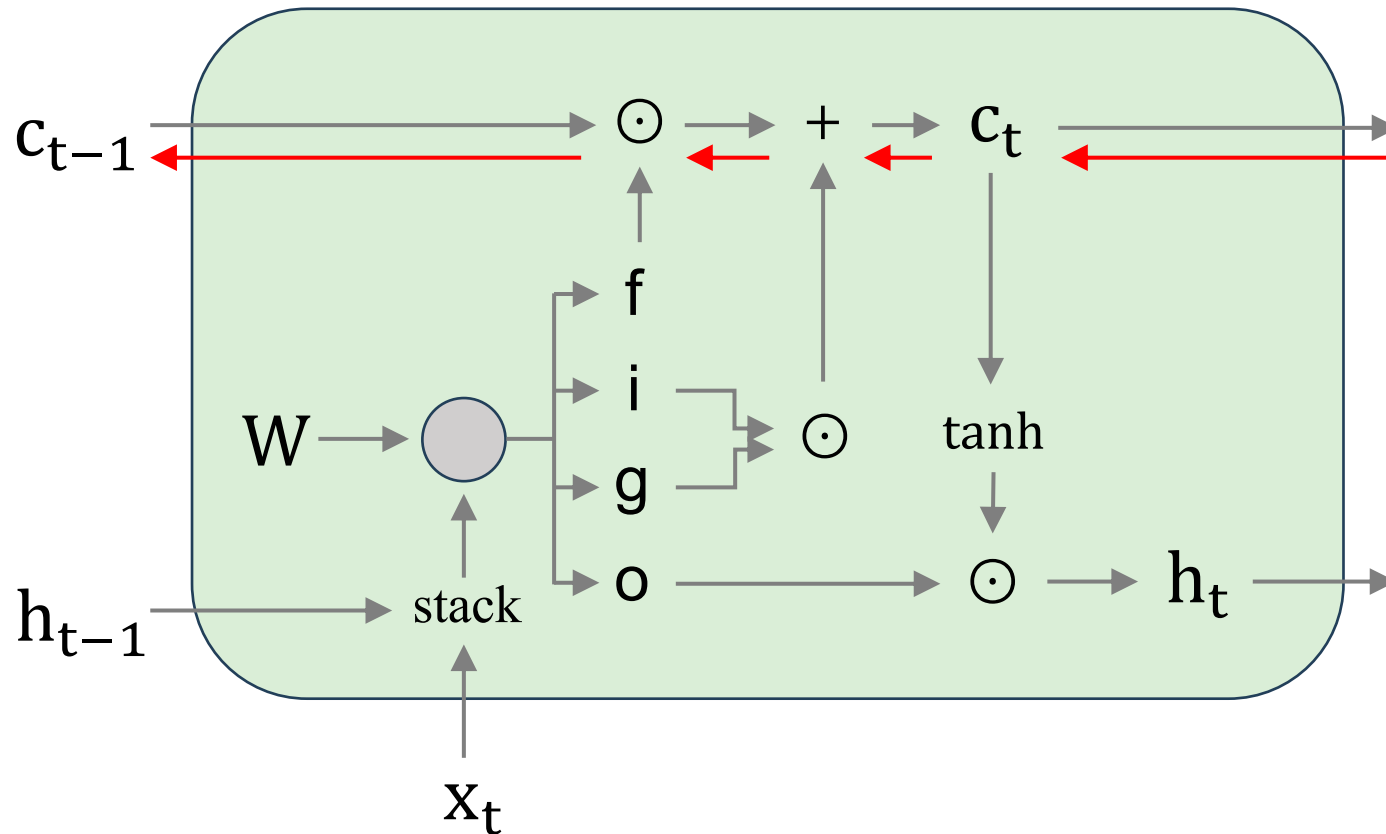
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



Backpropagation from c_t to c_{t-1} only elementwise multiplication by f , no matrix multiply by W

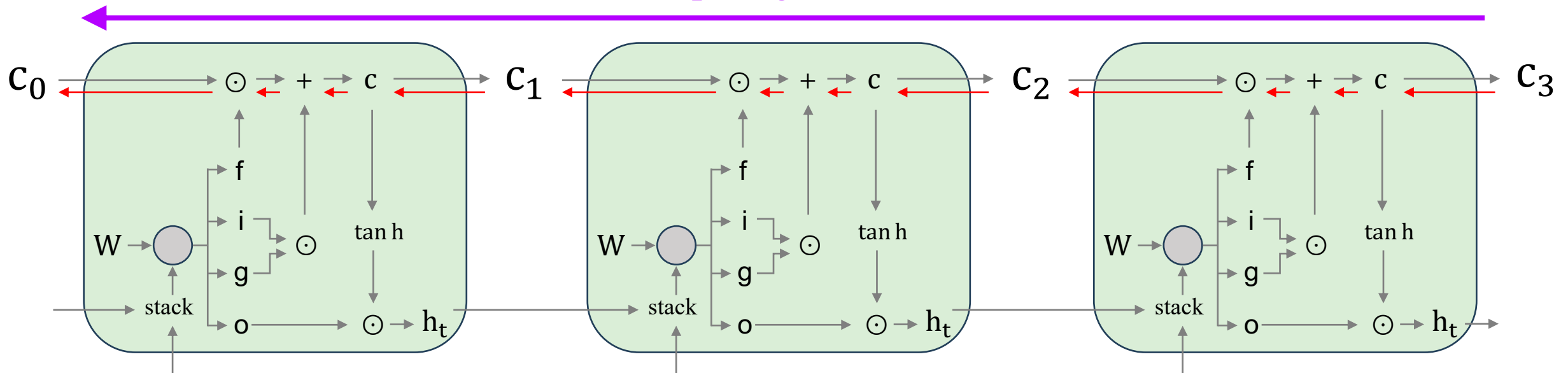
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

Uninterrupted gradient flow!



The f gate's values (e.g., activations) will determine whether to carry over the information along time

- If $f=1$, the gradient along c_i will not vanish:

$$\frac{\partial L_T}{\partial c_T} \frac{\partial c_T}{\partial c_{T-1}} \cdots \frac{\partial c_2}{\partial c_1} \frac{\partial c_1}{\partial W} = f^{T-1} \frac{\partial c_1}{\partial W} \approx \frac{\partial c_1}{\partial W}$$

- If a long memory is needed, the model will learn to set the f gate to a value very close to 1.

Other RNN Variants

Gated Recurrent Units (GRU) [Cho et al. 2014]

$$r_t = \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \quad \text{Reset gate}$$

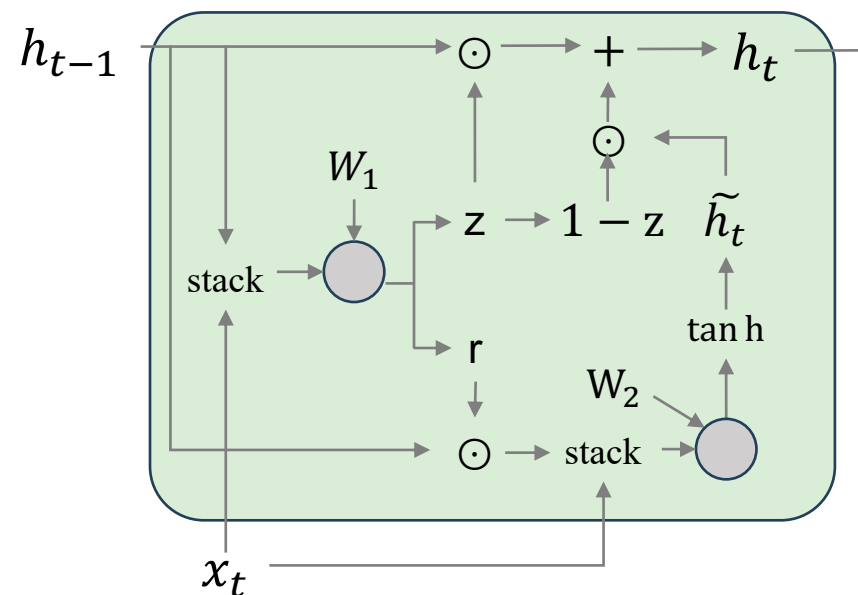
$$z_t = \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z) \quad \text{Update gate}$$

$$\tilde{h}_t = \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h)$$

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$

Backpropagation of GRU:

$$\begin{aligned} \frac{\partial h_t}{\partial h_{t-1}} &= z_t + (1 - z_t) \frac{\partial \tilde{h}_t}{\partial h_{t-1}} \\ &= z_t + (1 - z_t)(1 - \tilde{h}_t^2)(W_{hh}r_t) \end{aligned}$$



- If $z_t = 1$, no updates happen, $\frac{\partial h_t}{\partial h_{t-1}} = 1$
- Even if $r_t = 0$, forget all previous information, the gradient depends on the activation of the update gate
 $\frac{\partial h_t}{\partial h_{t-1}} = z_t$

Summary of RNNs

- RNNs design
 - Many-to-many
 - One-to-many
 - Many-to-one
- Vanilla RNNs– Vanishing gradient
- LSTM or GRU: learning from longer dependencies with better gradient flow compared to Vanilla RNNs

Attention and Transformers

Sequence to Sequence with RNNs

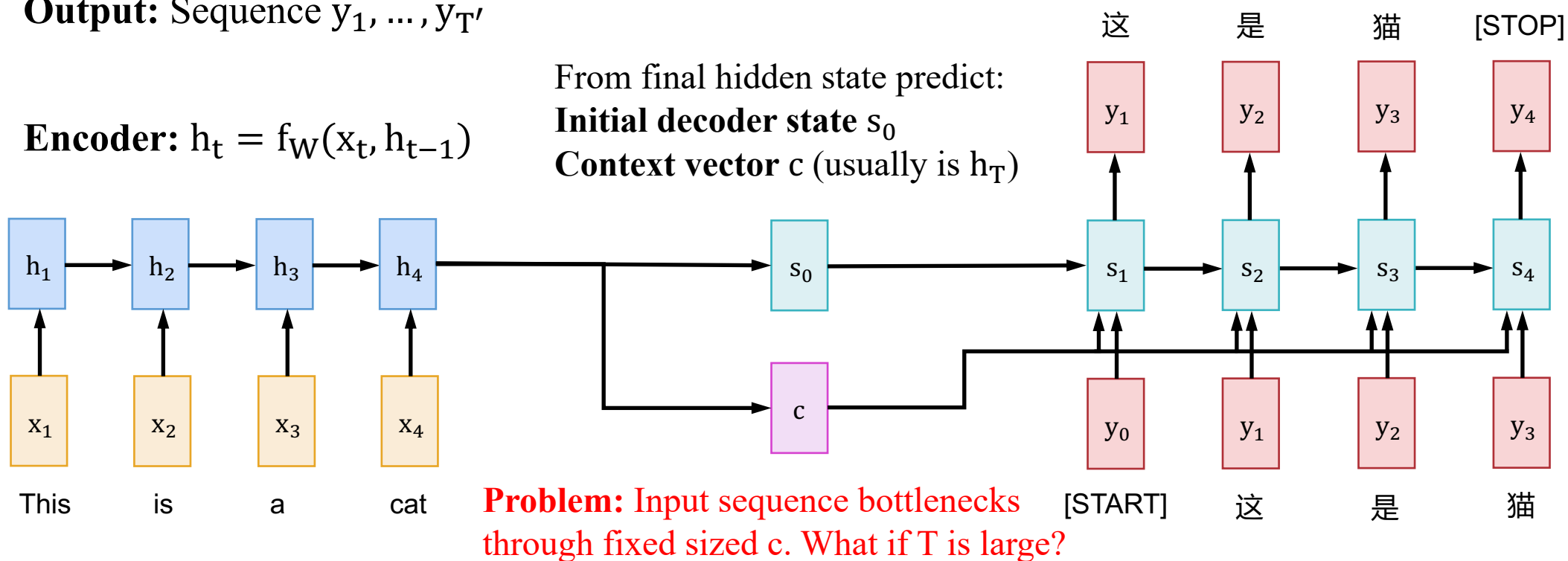
Machine translation: English $\rightarrow$ Chinese

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_{T'}$

Encoder: $h_t = f_W(x_t, h_{t-1})$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$



Sequence to Sequence with RNNs and Attention

Idea: Look at each past step, more attention on more relevant step

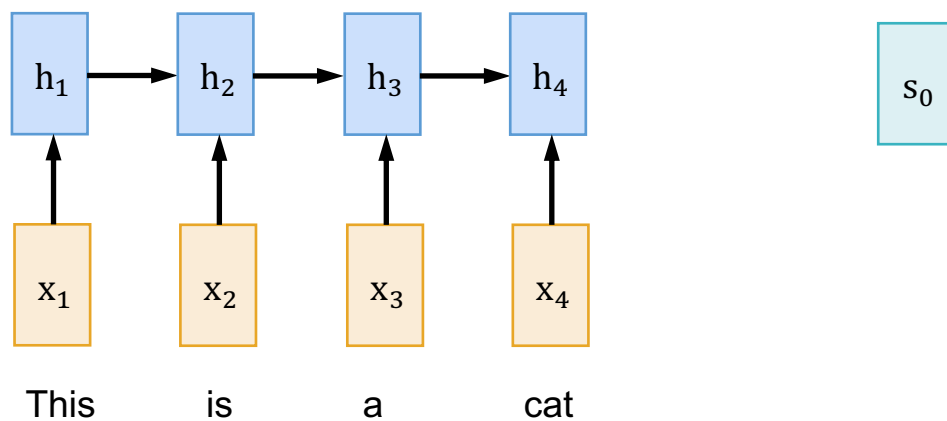
Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_{T'}$

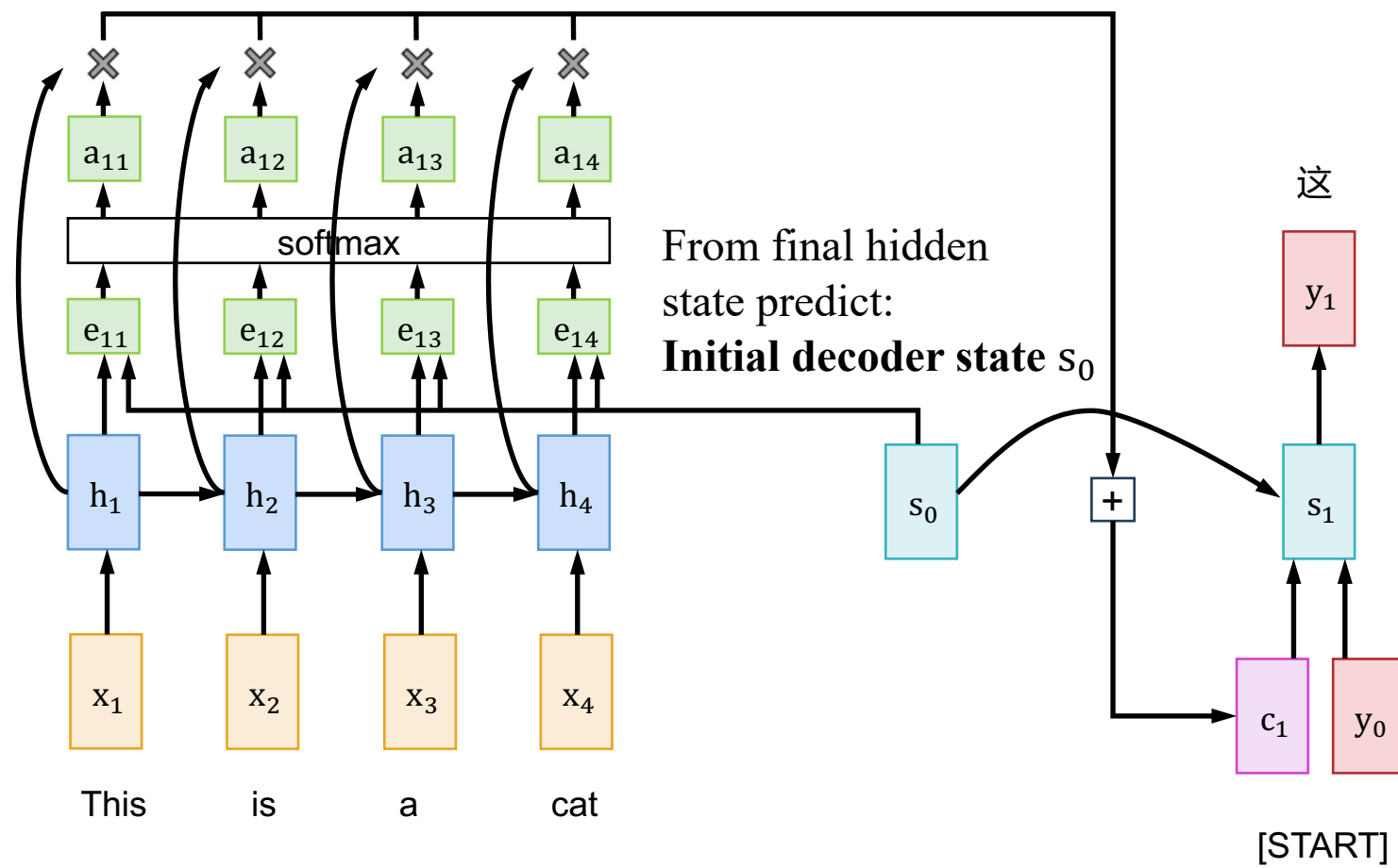
Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden
state predict:

Initial decoder state s_0



Sequence to Sequence with RNNs and Attention



Compute (scalar) **alignment scores**

$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \quad (f_{\text{att}} \text{ is a Linear Layer})$$

Normalize alignment scores to get **attention weights**

$$0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$$

Compute **context vector** as **weighted sum of hidden states**

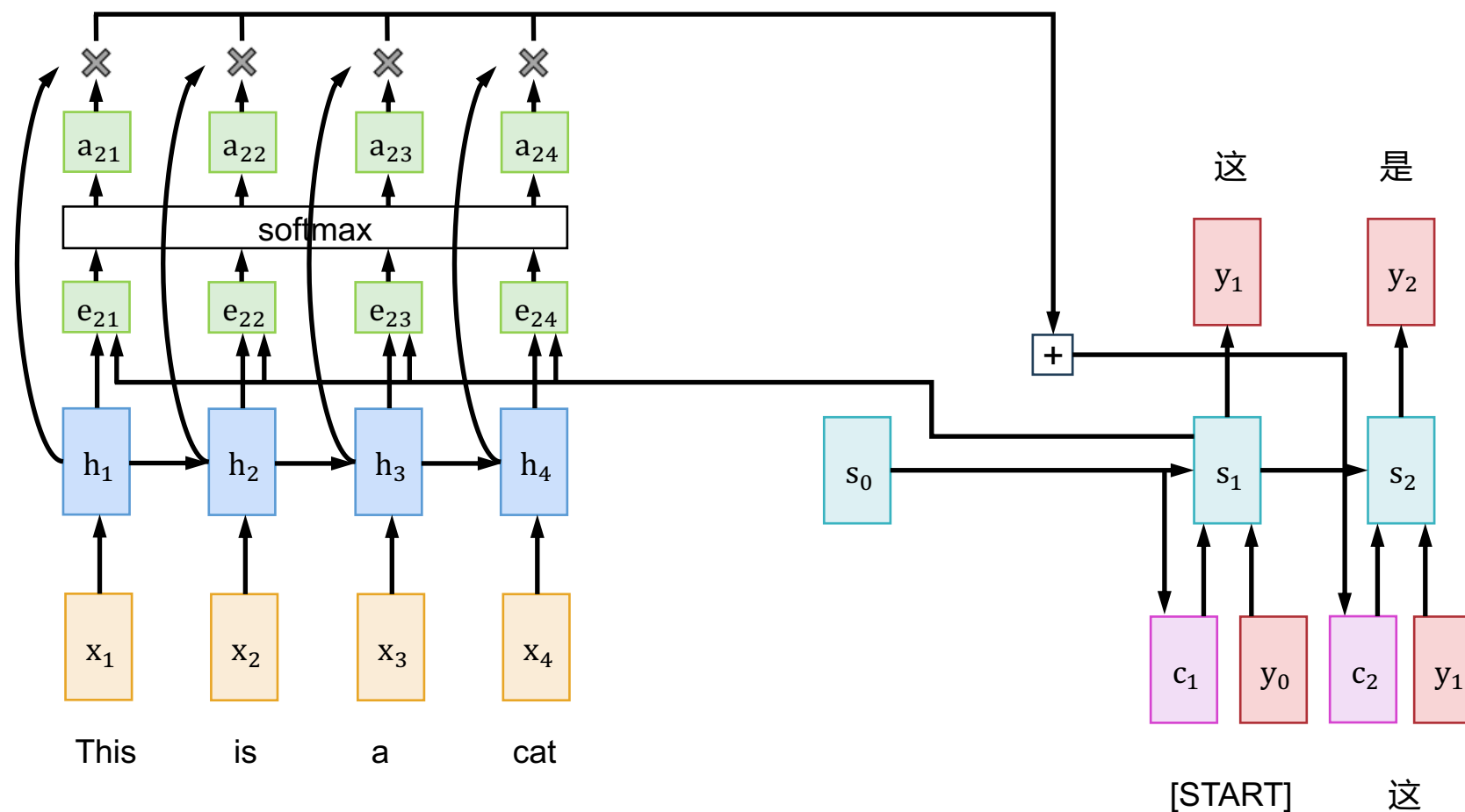
$$c_t = \sum_i a_{t,i} h_i$$

Use context vector in decoder:

$$s_t = g_U(y_{t-1}, s_{t-1}, c_t)$$

g_U is an RNN unit (e.g. LSTM, GRU)

Sequence to Sequence with RNNs and Attention



Repeat: Use s_1 to compute new context vector c_2

Compute new alignment scores $e_{2,i}$ and attention weights $a_{2,i}$

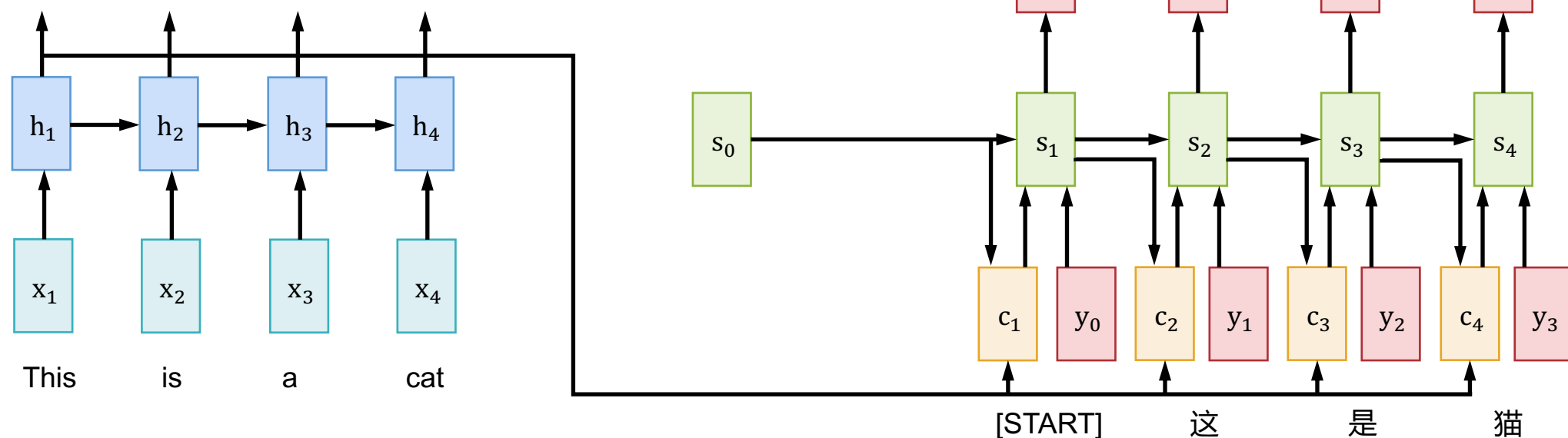
Sequence to Sequence with RNNs and Attention

Query vectors: decoder RNN states s

Data vectors: encoder RNN states h

Output vectors: context states c

Each **query** attends to all **data** vectors and gives one **output** vector



Attention Layer – General Formulation

Inputs:

Query vectors: Q [$N_Q \times D_Q$]

Data vectors: X [$N_X \times D_X$]

Key matrix: W_k [$D_X \times D_Q$]

Value matrix: W_v [$D_X \times D_V$]

Computation:

Keys: $K = XW_k$ [$N_X \times D_Q$]

Values: $V = XW_v$ [$N_X \times D_V$]

Similarities: $E = Q \cdot K^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ [$N_Q \times N_X$]

Output vector: $Y = AV$ [$N_Q \times D_X$]

$$Y_i = \sum_j A_{ij} V_j$$

X_1

X_2

X_3

Q_1

Q_2

Q_3

Q_4

Attention Layer – General Formulation

Inputs:

Query vectors: Q [$N_Q \times D_Q$]

Data vectors: X [$N_X \times D_X$]

Key matrix: W_k [$D_X \times D_Q$]

Value matrix: W_v [$D_X \times D_V$]

Computation:

Keys: $K = XW_k$ [$N_X \times D_Q$]

Values: $V = XW_v$ [$N_X \times D_V$]

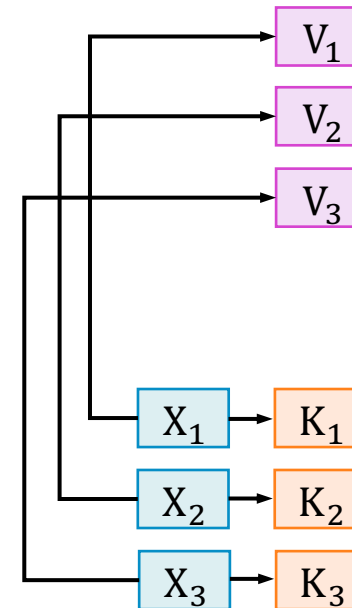
Similarities: $E = Q \cdot K^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ [$N_Q \times N_X$]

Output vector: $Y = AV$ [$N_Q \times D_X$]

$$Y_i = \sum_j A_{ij} V_j$$



Attention Layer – General Formulation

Inputs:

Query vector: Q [$N_Q \times D_Q$]

Data vectors: X [$N_X \times D_X$]

Key matrix: W_k [$D_X \times D_Q$]

Value matrix: W_v [$D_X \times D_V$]

Computation:

Keys: $K = XW_k$ [$N_X \times D_Q$]

Values: $V = XW_v$ [$N_X \times D_V$]

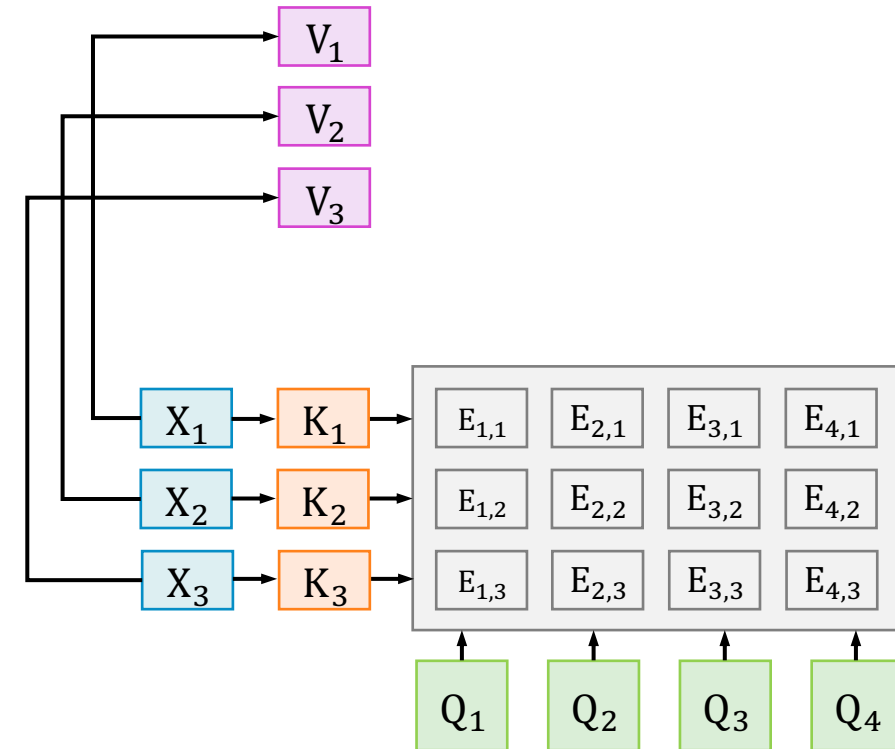
Similarities: $E = Q \cdot K^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ [$N_Q \times N_X$]

Output vector: $Y = AV$ [$N_Q \times D_X$]

$$Y_i = \sum_j A_{ij} V_j$$



Attention Layer – General Formulation

Inputs:

Query vector: Q [$N_Q \times D_Q$]

Data vectors: X [$N_X \times D_X$]

Key matrix: W_k [$D_X \times D_Q$]

Value matrix: W_v [$D_X \times D_V$]

Computation:

Keys: $K = XW_k$ [$N_X \times D_Q$]

Values: $V = XW_v$ [$N_X \times D_V$]

Similarities: $E = Q \cdot K^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

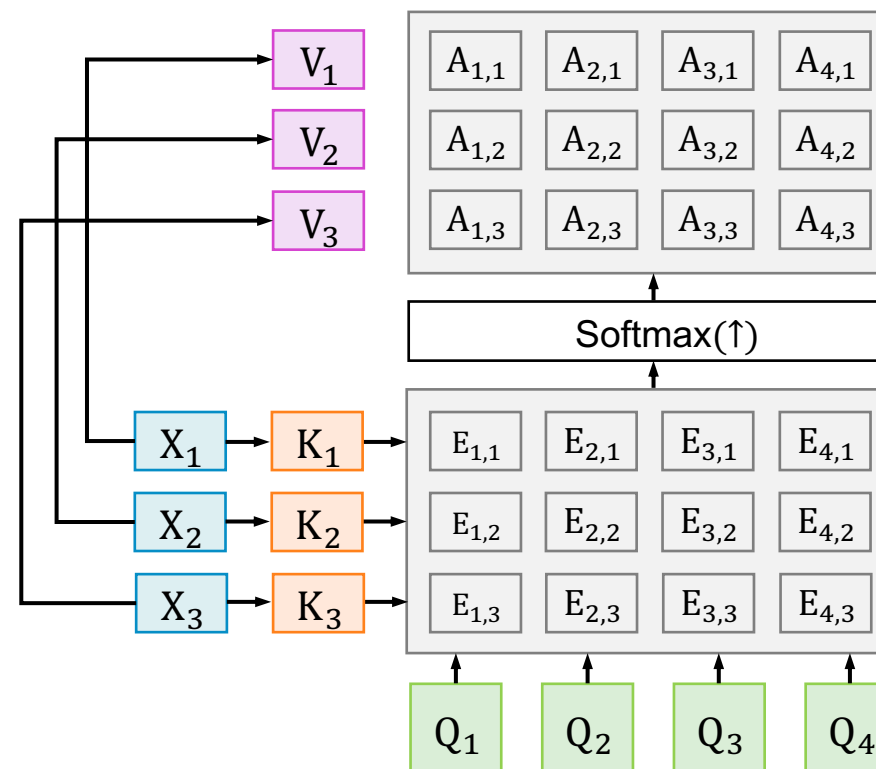
$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ [$N_Q \times N_X$]

Output vector: $Y = AV$ [$N_Q \times D_V$]

$$Y_i = \sum_j A_{ij} V_j$$

Softmax normalizes each column: each query predicts a distribution over the keys



Attention Layer – General Formulation

Inputs:

Query vector: Q [$N_Q \times D_Q$]

Data vectors: X [$N_X \times D_X$]

Key matrix: W_k [$D_X \times D_Q$]

Value matrix: W_v [$D_X \times D_V$]

Computation:

Keys: $K = XW_k$ [$N_X \times D_Q$]

Values: $V = XW_v$ [$N_X \times D_V$]

Similarities: $E = Q \cdot K^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

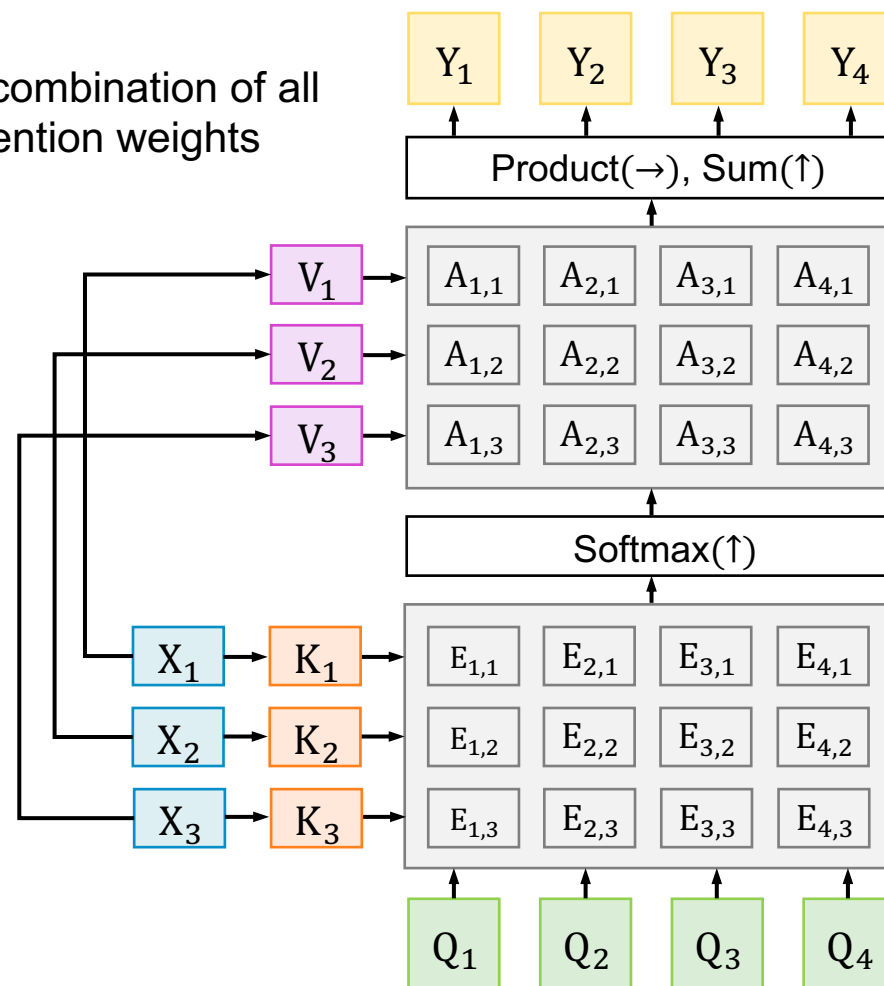
$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ [$N_Q \times N_X$]

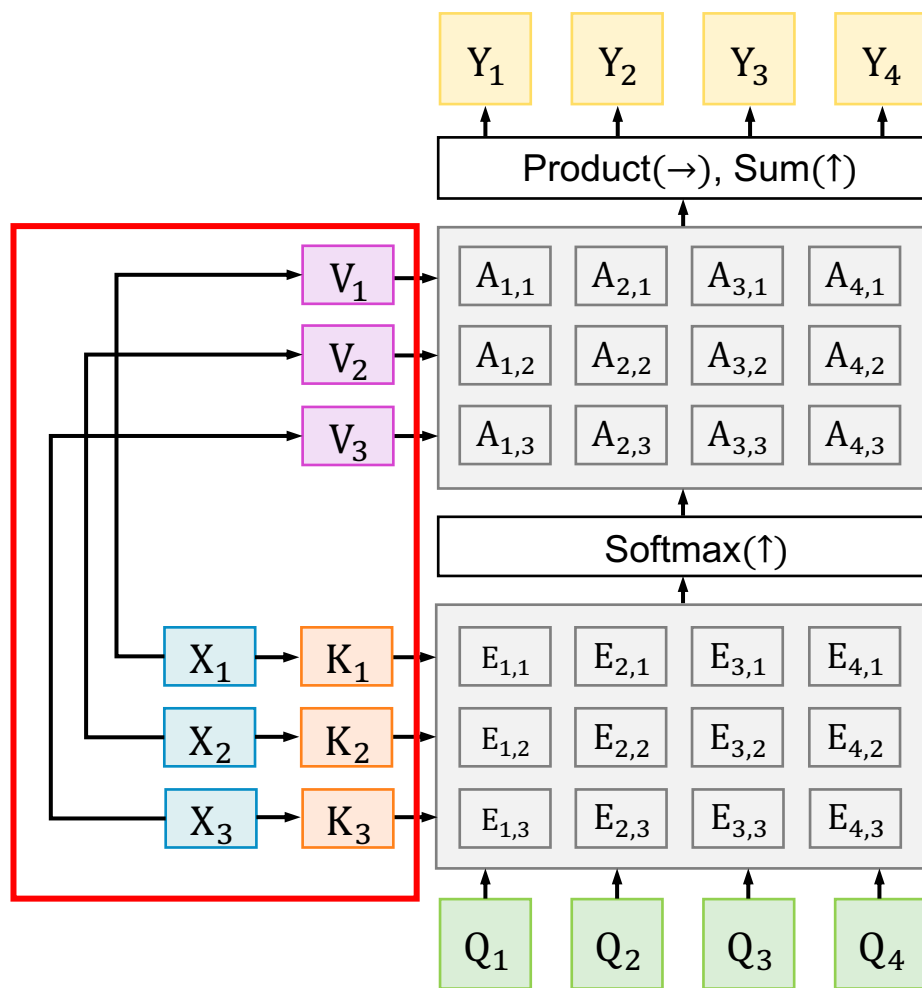
Output vector: $Y = A V$ [$N_Q \times D_X$]

$$Y_i = \sum_j A_{ij} V_j$$

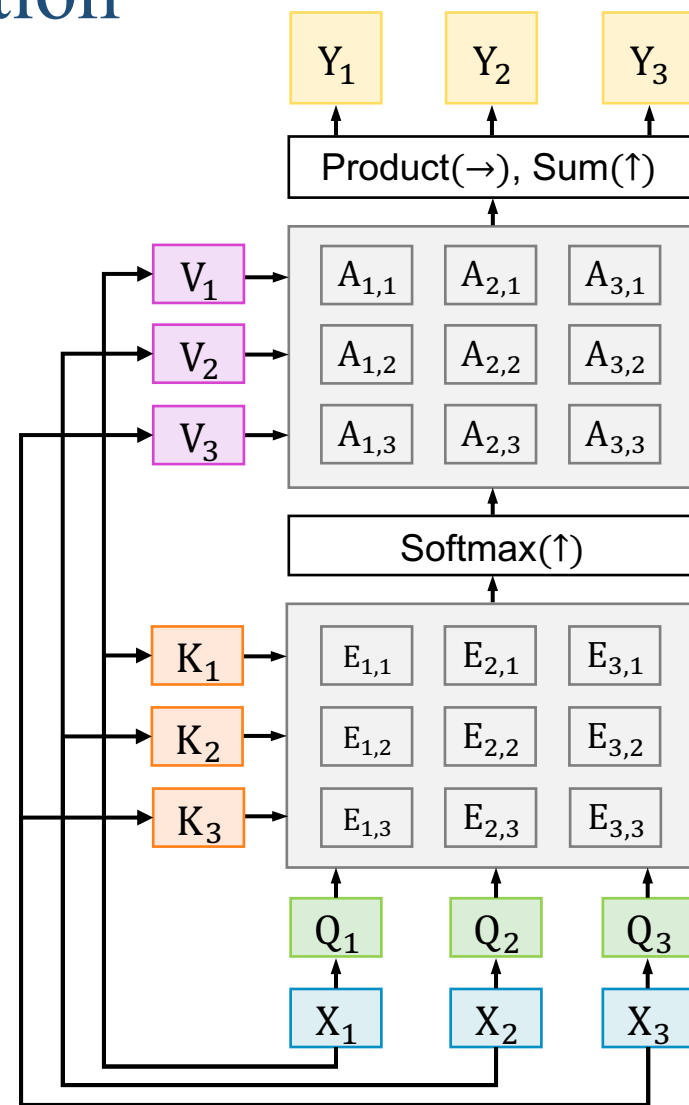
Each **output** is a linear combination of all **values**, weighted by attention weights



Cross-Attention Layer and Self-Attention



Cross-Attention



Self-Attention

Self-Attention Layer

Inputs:

Input vectors: X [$N \times D_{in}$]

Key matrix: W_k [$D_{in} \times D_{out}$]

Value matrix: W_v [$D_{in} \times D_{out}$]

Query matrix: W_Q [$N_{in} \times D_{out}$]

Computation:

Queries: $Q = XW_Q$ [$N \times D_{out}$]

Keys: $K = XW_k$ [$N \times D_{out}$]

Values: $V = XW_v$ [$N \times D_{out}$]

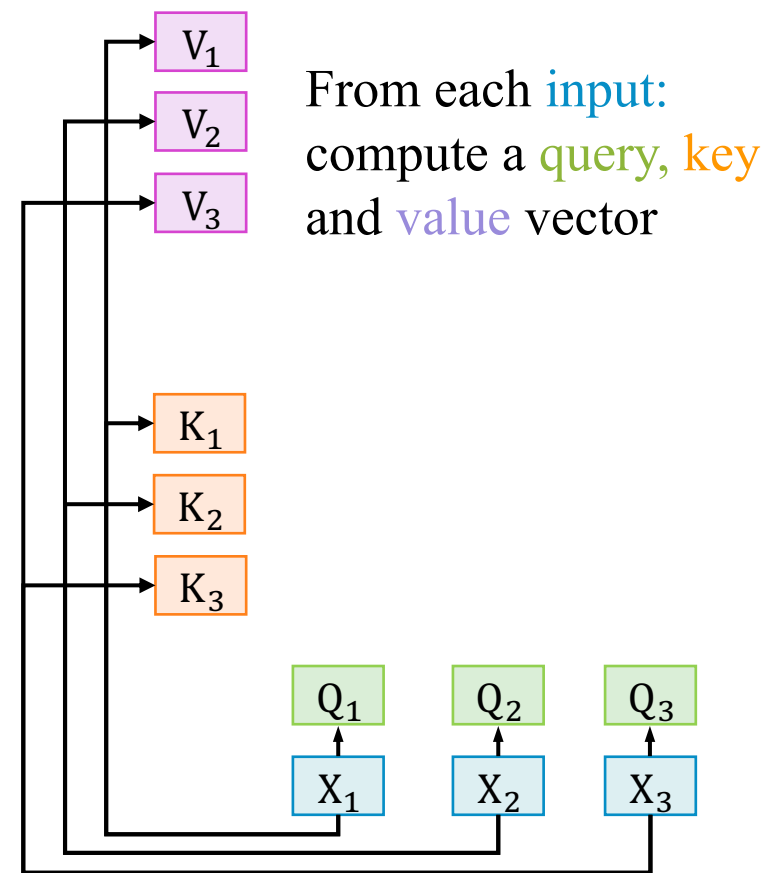
Similarities: $E = Q \cdot K^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ [$N \times N$]

Output vector: $Y = AV$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$



Self-Attention Layer

Inputs:

Input vectors: X [$N \times D_{in}$]

Key matrix: W_k [$D_{in} \times D_{out}$]

Value matrix: W_v [$D_{in} \times D_{out}$]

Query matrix: W_Q [$N_{in} \times D_{out}$]

Computation:

Queries: $Q = XW_Q$ [$N \times D_{out}$]

Keys: $K = XW_k$ [$N \times D_{out}$]

Values: $V = XW_v$ [$N \times D_{out}$]

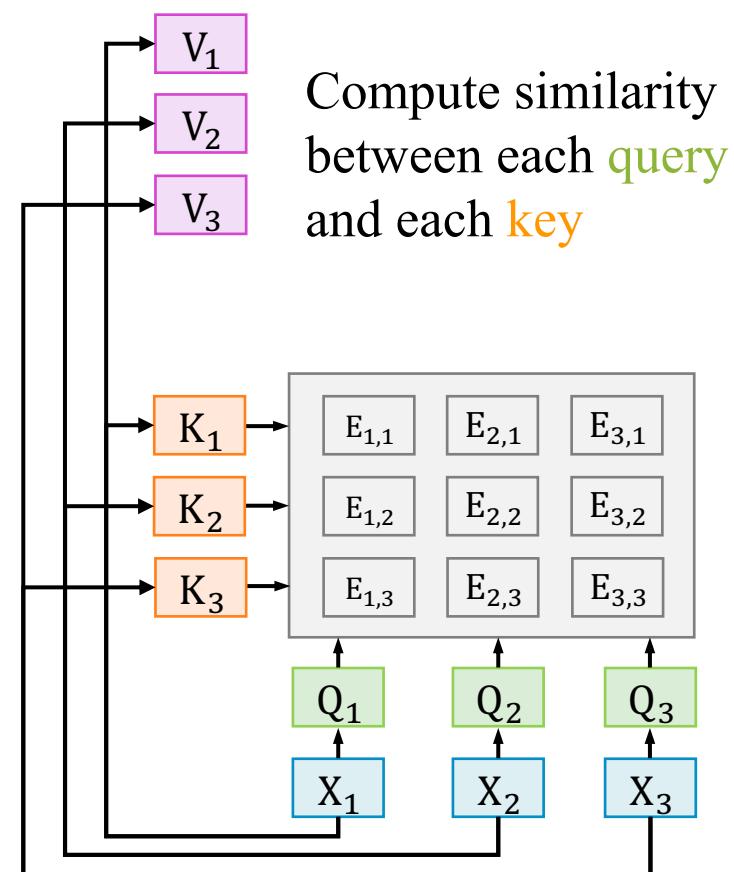
Similarities: $E = Q \cdot K^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ [$N \times N$]

Output vector: $Y = AV$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$



Self-Attention Layer

Inputs:

Input vectors: X [$N \times D_{in}$]

Key matrix: W_k [$D_{in} \times D_{out}$]

Value matrix: W_v [$D_{in} \times D_{out}$]

Query matrix: W_Q [$N_{in} \times D_{out}$]

Computation:

Queries: $Q = XW_Q$ [$N \times D_{out}$]

Keys: $K = XW_k$ [$N \times D_{out}$]

Values: $V = XW_v$ [$N \times D_{out}$]

Similarities: $E = Q \cdot K^T / \sqrt{D_Q}$ [$N \times N$]

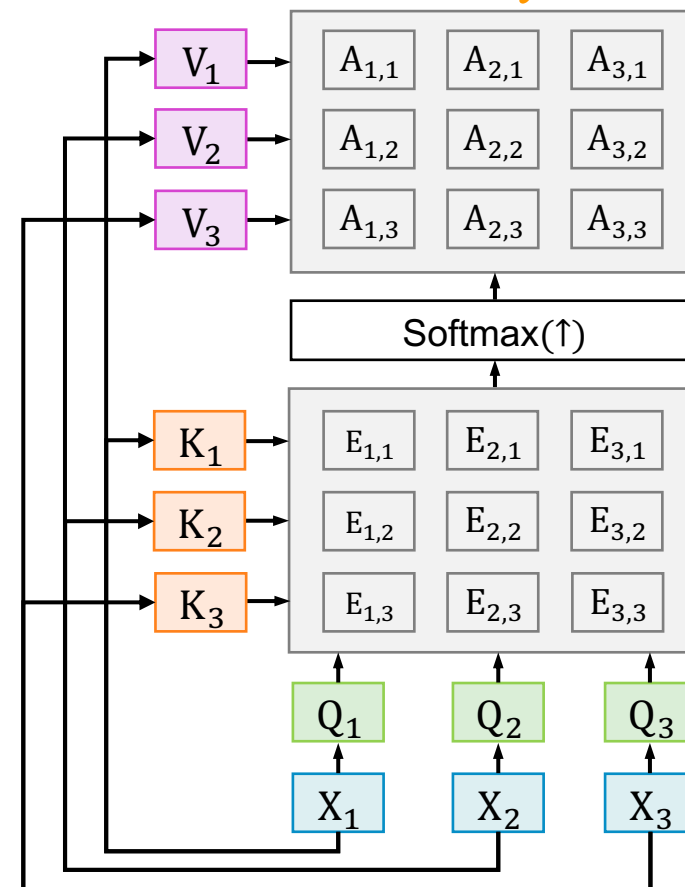
$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ [$N \times N$]

Output vector: $Y = AV$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$

Normalize over each column:
each **query** computes a
distribution over **keys**



Self-Attention Layer

Inputs:

Input vectors: X [$N \times D_{in}$]

Key matrix: W_k [$D_{in} \times D_{out}$]

Value matrix: W_v [$D_{in} \times D_{out}$]

Query matrix: W_Q [$N_{in} \times D_{out}$]

Computation:

Queries: $Q = XW_Q$ [$N \times D_{out}$]

Keys: $K = XW_k$ [$N \times D_{out}$]

Values: $V = XW_v$ [$N \times D_{out}$]

Similarities: $E = Q \cdot K^T / \sqrt{D_Q}$ [$N \times N$]

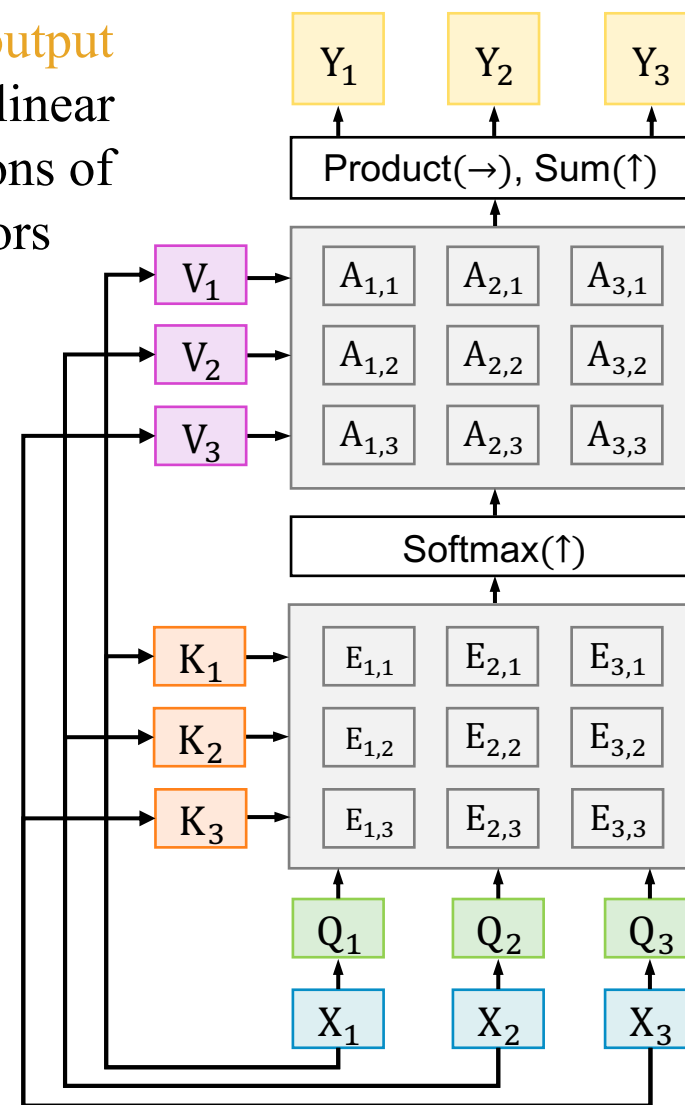
$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ [$N \times N$]

Output vector: $Y = AV$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$

Compute **output** vectors as linear combinations of **value** vectors



Self-Attention Layer

Inputs:

Input vectors: X [$N \times D_{in}$]

Key matrix: W_k [$D_{in} \times D_{out}$]

Value matrix: W_v [$D_{in} \times D_{out}$]

Query matrix: W_Q [$N_{in} \times D_{out}$]

Computation:

Queries: $Q = XW_Q$ [$N \times D_{out}$]

Keys: $K = XW_k$ [$N \times D_{out}$]

Values: $V = XW_v$ [$N \times D_{out}$]

Similarities: $E = Q \cdot K^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ [$N \times N$]

Output vector: $Y = AV$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$

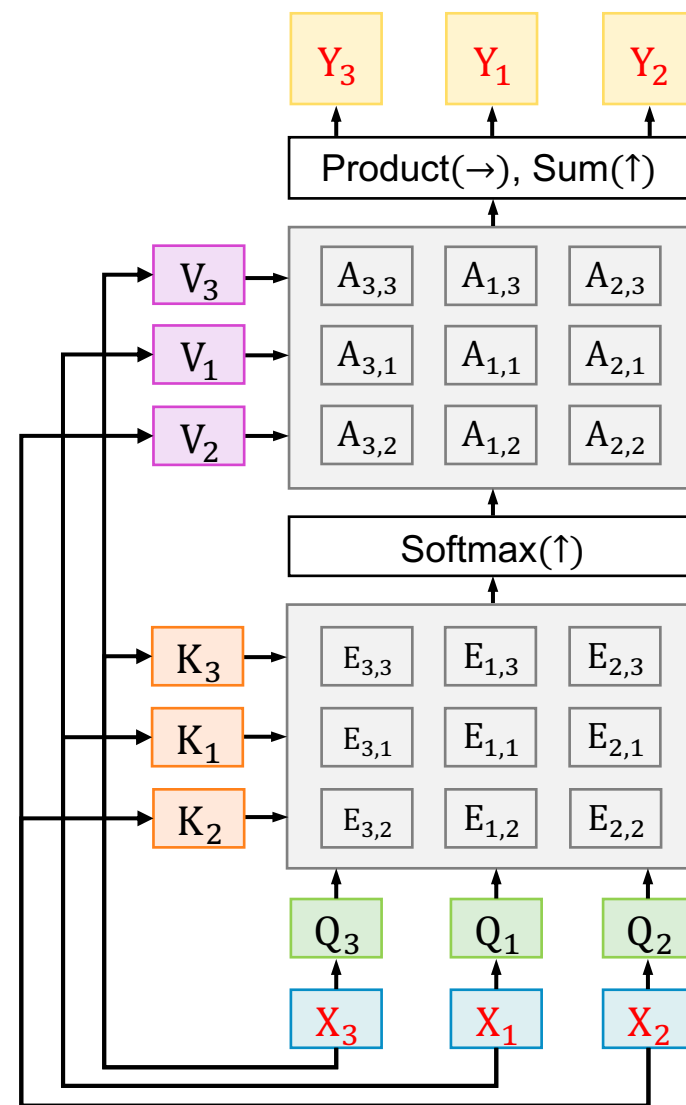
Consider permuting inputs:

Queries, keys, and values will be the same but permuted

Similarities E are the same but permuted

Attention weights A are the same but permuted

Outputs are the same but permuted



Self-Attention Layer

Inputs:

Input vectors: X [$N \times D_{in}$]

Key matrix: W_k [$D_{in} \times D_{out}$]

Value matrix: W_v [$D_{in} \times D_{out}$]

Query matrix: W_Q [$N_{in} \times D_{out}$]

Computation:

Queries: $Q = XW_Q$ [$N \times D_{out}$]

Keys: $K = XW_k$ [$N \times D_{out}$]

Values: $V = XW_v$ [$N \times D_{out}$]

Similarities: $E = Q \cdot K^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ [$N \times N$]

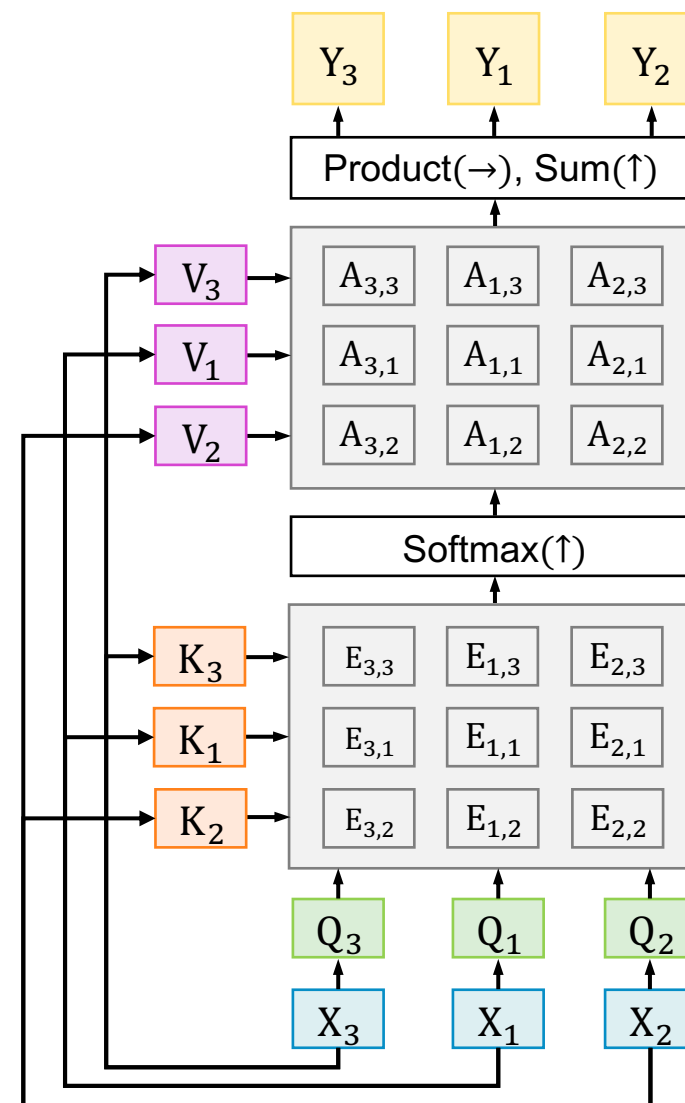
Output vector: $Y = AV$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$

Self-Attention is
permutation equivariant:

$$F(\sigma(X)) = \sigma(F(X))$$

This means that Self-Attention works on **set of vectors**



Self-Attention Layer

Inputs:

Input vectors: $X [N \times D_{in}]$

Key matrix: $W_k [D_{in} \times D_{out}]$

Value matrix: $W_v [D_{in} \times D_{out}]$

Query matrix: $W_Q [N_{in} \times D_{out}]$

Computation:

Queries: $Q = XW_Q [N \times D_{out}]$

Keys: $K = XW_k [N \times D_{out}]$

Values: $V = XW_v [N \times D_{out}]$

Similarities: $E = Q \cdot K^T / \sqrt{D_Q} [N \times N]$

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

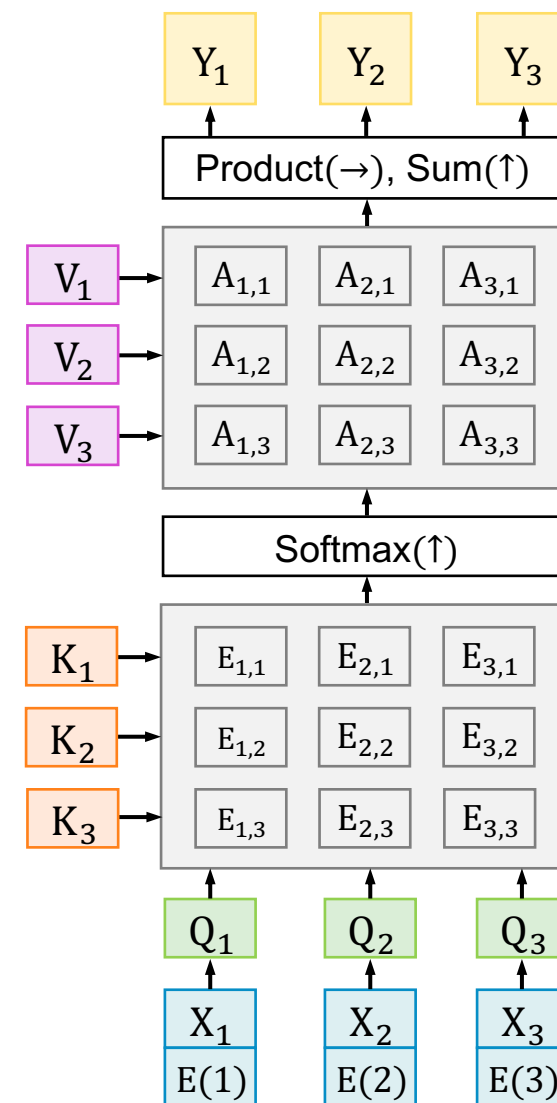
Attention weights: $A = \text{softmax}(E, \text{dim} = 1) [N \times N]$

Output vector: $Y = AV [N \times D_{out}]$

$$Y_i = \sum_j A_{ij} V_j$$

Problem: Self-Attention does not know the order of the sequence

Solution: Add positional encoding to each input; this is a vector that is a fixed function of the index



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: $X [N \times D_{in}]$

Key matrix: $W_k [D_{in} \times D_{out}]$

Value matrix: $W_v [D_{in} \times D_{out}]$

Query matrix: $W_Q [N_{in} \times D_{out}]$

Computation:

Queries: $Q = XW_Q [N \times D_{out}]$

Keys: $K = XW_k [N \times D_{out}]$

Values: $V = XW_v [N \times D_{out}]$

Similarities: $E = Q \cdot K^T / \sqrt{D_Q} [N \times N]$

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim} = 1) [N \times N]$

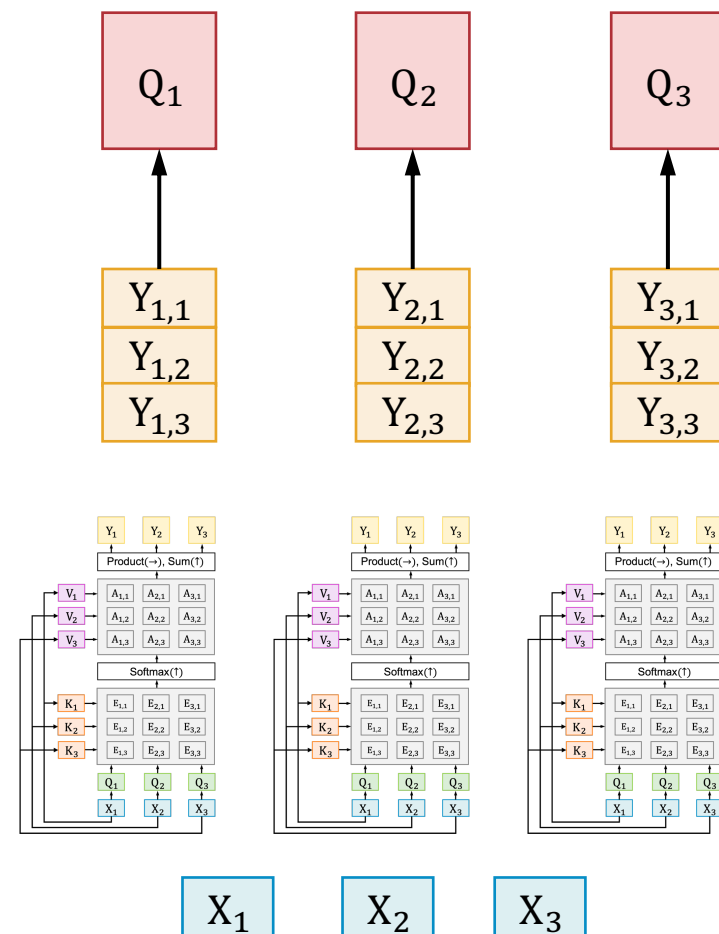
Output vector: $Y = AV [N \times D_{out}]$

$$Y_i = \sum_j A_{ij} V_j$$

Output projection fuses data from each head

Stack up the H independent outputs for each input X

H = 3 independent self-attention layers (called heads), each with their own weights



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: X [$N \times D$]

Key matrix: W_k [$D \times H D_H$]

Value matrix: W_v [$D \times H D_H$]

Query matrix: W_Q [$D \times H D_H$]

Output matrix: W_O [$H D_H \times D$]

Usually $D_H = D/H$, so inputs and outputs have the same dimension

Computation:

Queries: $Q = XW_Q$ [$H \times N \times D_H$]

Keys: $K = XW_k$ [$H \times N \times D_H$]

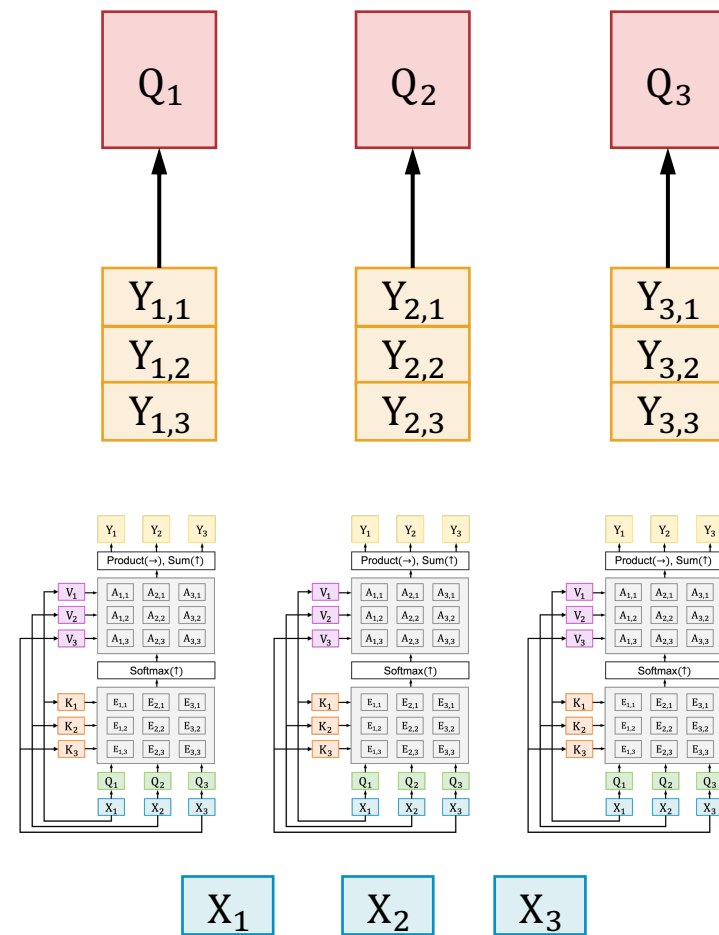
Values: $V = XW_v$ [$H \times N \times D_H$]

Similarities: $E = Q \cdot K^T / \sqrt{D_Q}$ [$H \times N \times N$]

Attention weights: $A = \text{softmax}(E, \text{dim} = 2)$ [$H \times N \times N$]

Head outputs: $Y = AV$ [$H \times N \times D_H$] $\Rightarrow$ [$N \times H D_H$]

Outputs: $O = YW_O$ [$N \times D$]



The Transformer

Transformer Block

Input: Set of vectors x

Output: Set of vectors y

Layer Normalization:

Latent vectors: $h_1, \dots, h_N$

Learnable projection parameters: γ, β

$$\mu_i = (\sum_j h_{i,j})/D, \quad \sigma_i = \left(\sum_j (h_{i,j} - \mu_i)^2 / D \right)^{1/2}$$

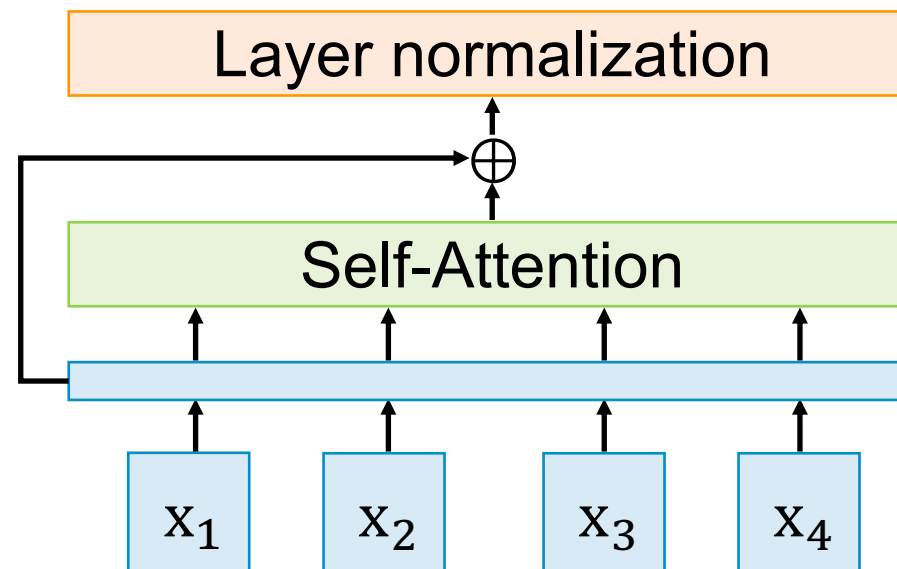
$$z_i = (h_i - \mu_i) / \sigma_i$$

$$y_i = \gamma * z_i + \beta$$

Layer normalization
normalizes all vectors

Residual connection

All vectors interact through
(multiheaded) Self-Attention



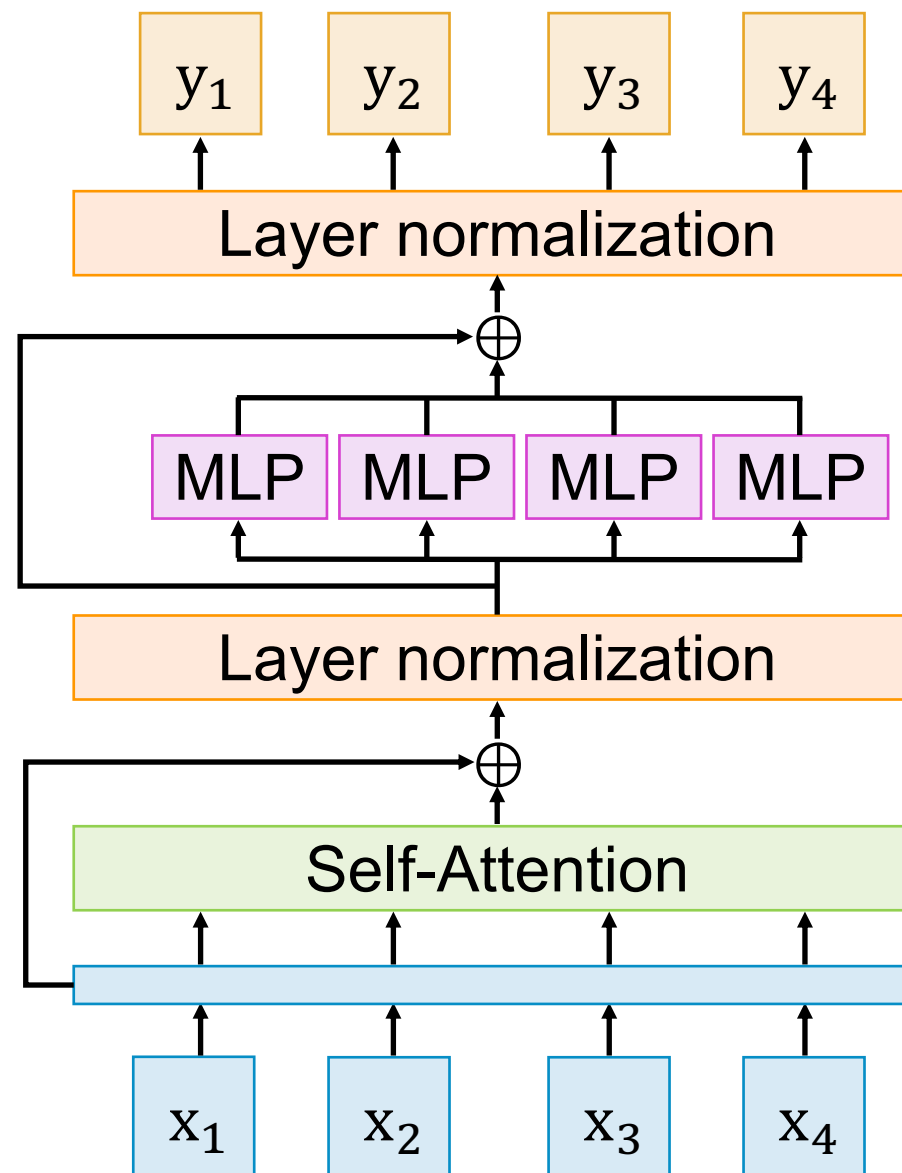
The Transformer

Transformer Block

Input: Set of vectors x

Output: Set of vectors y

- Another Layer Norm
- Residual connection
- MLP independently on each vector
- Layer normalization normalizes all vectors
- Residual connection
- All vectors interact through (multiheaded) Self-Attention



The Transformer

Transformer Block

Input: Set of vectors x

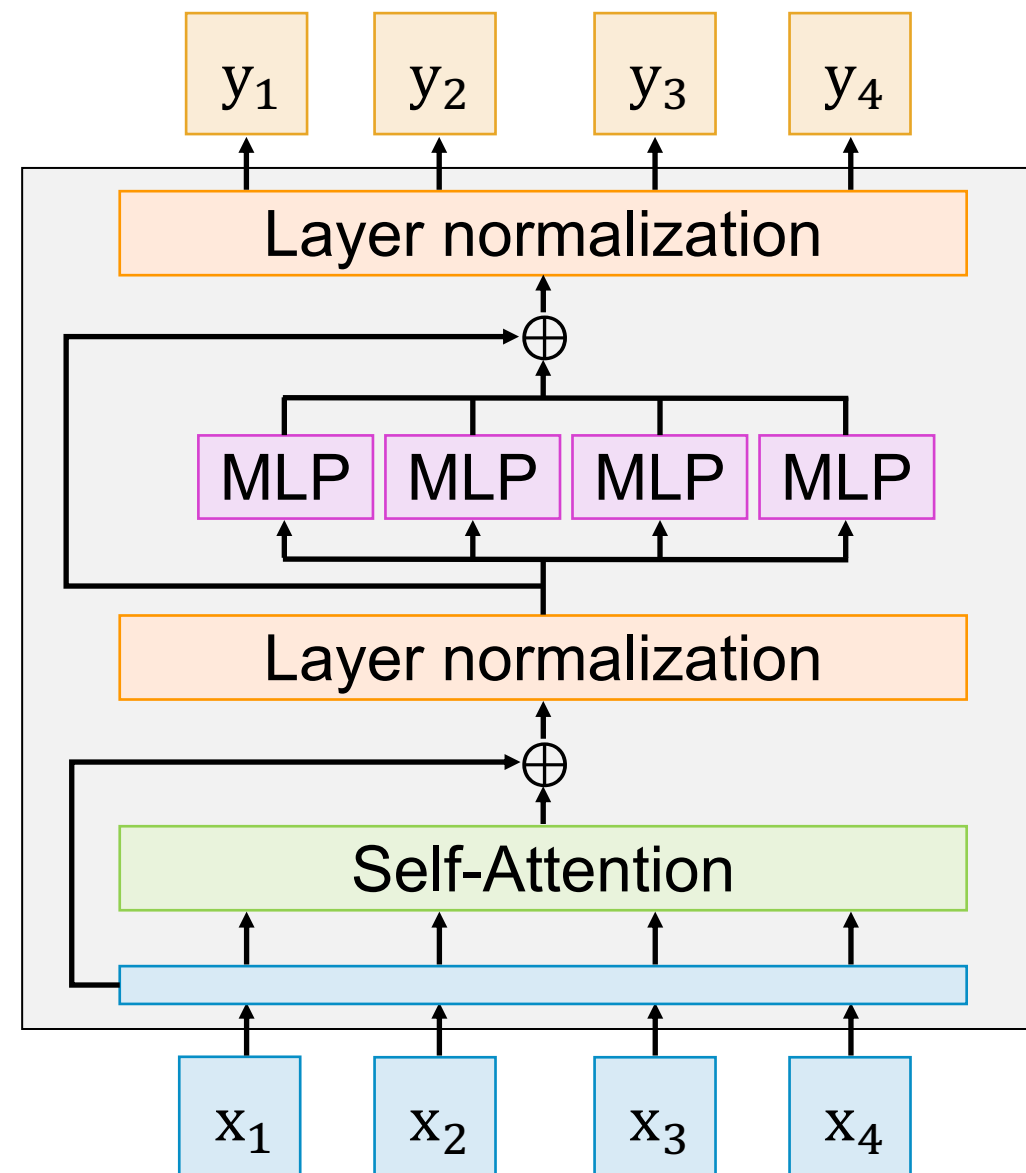
Output: Set of vectors y

Self-Attention is the only interaction between vectors

LayerNorm and MLP work on each vector independently

Highly scalable and parallelizable, most of the compute is just 6 matrix multiplications:

- 4 from Self-Attention
- 2 from MLP



The Transformer

A **Transformer** is just a stack of identical Transformer blocks!

Original: [Vaswani et al, 2017]

12 blocks, $D=1024$, $H=16$, $N=512$

213M parameters

GPT-2: [Radford et al, 2019]

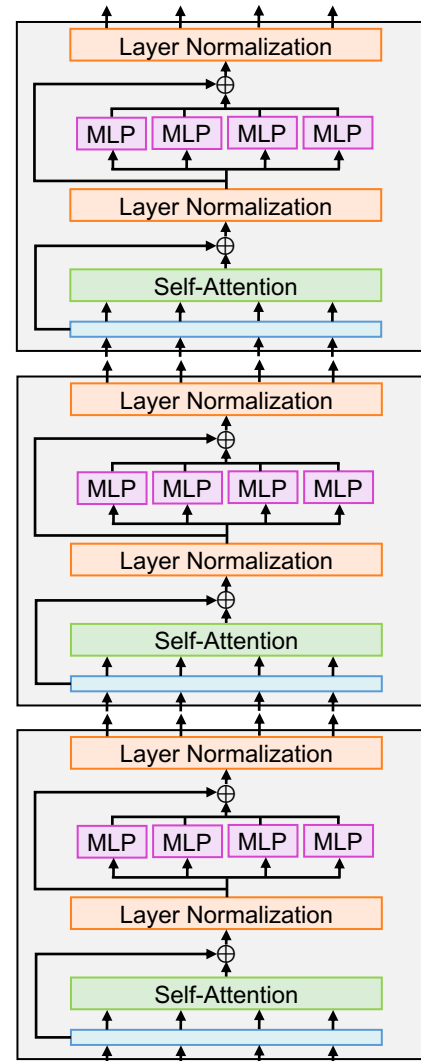
48 blocks, $D=1600$, $H=25$, $N=1024$

1.5B parameters

GPT-3: [Brown et al, 2020]

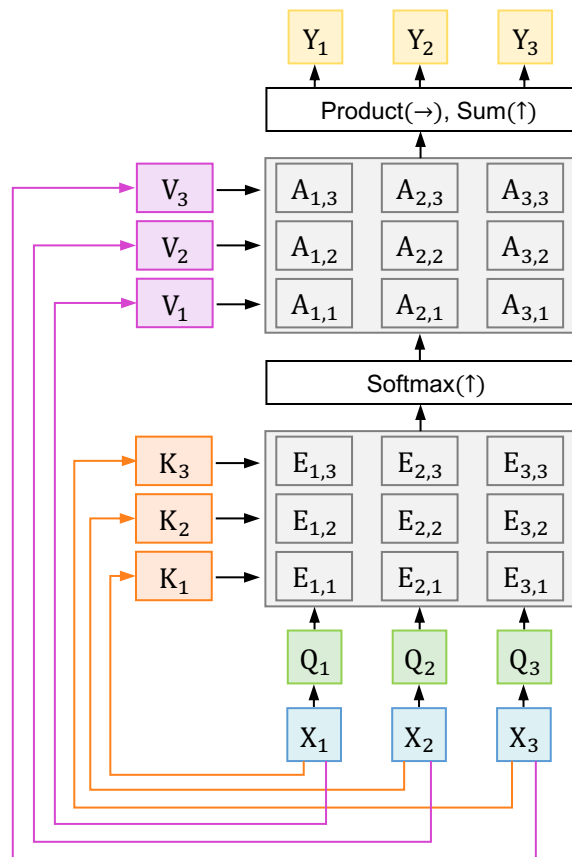
96 blocks, $D=12288$, $H=96$, $N=2048$

175B parameters



Summary: Attention + Transformers

Attention: Fuse information based on similarities



Transformer: A neural network architecture that uses attention

